

# HIBERNATE FRAMEWORK

## DAY-1

### What is Framework?

- Fr is a **soft arch** that provides a structure, guidelines and reusable component to facilitate the development of an application.
  - Fr handles the infrastructure so that you can focus on logic
  - Control Flow will be managed by framework itself
  - To simplify Development
- 
- GET THE FRAMEWORK
  - PUT THE DATA (id name add salary)
  - PUT YOUR CONFIGURATION (url user pass driver)
  - TELL WHAT TO DO
  - GET THE RESULT

### EXAMPLE:

- CREATE A TABLE
- SQL
- ResultSet
- Create Model Object
- Create a List

- Loop : Add Model into List
- return List

## **Hibernate:**

- get the List (Table, SQL, Model Object, Model->Setter, List, add(model),return List)
- return List

## **What is Hibernate?**

- Hibernate is a framework which provides an ORM tool to simplify the development (DAO)
- Hibernate ORM tool (Object Relational Mapping)
- To achieve Data Persistency
- GAVIN KING 2001
- Internally it uses JDBC API
- Third Party

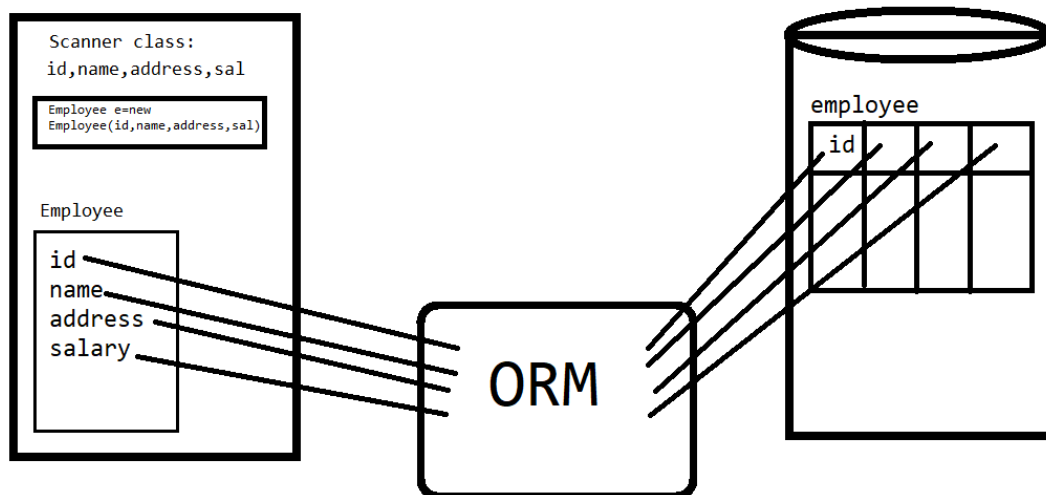
## **ADVANTAGE:**

- Fast Performance
- Pooling Mechanism
- SQL Independent (SQL,HQL,PURE JAVA)
- CACHING MECHANISM

- Auto Table Creation
- No Checked Exp
- PK Generation
- Easily Pagination
- Versioning
- EASY CRUD
- Simplify
- Job Opp

## ORM tool (Object Relational Mapping Tool)

- It is a programming technique that simplify interaction between Object and RDBMS



## DAY-2

In Hibernate we can Perform DB operation in two ways

## 1.SINGLE OPERATION

## 2.BULK OPERATION

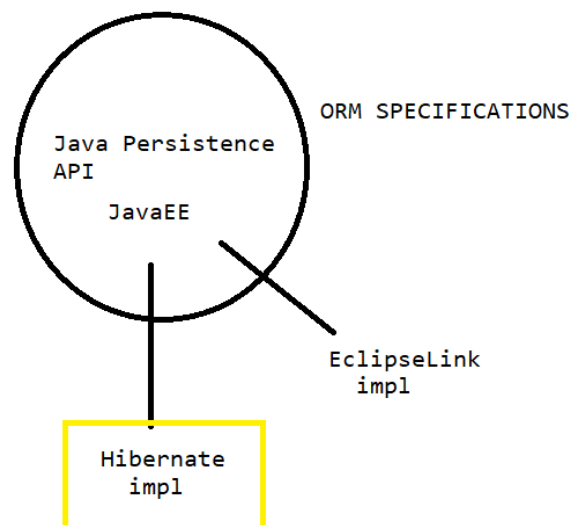
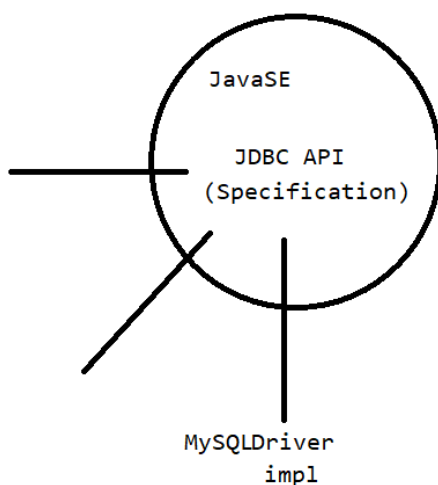
### SINGLE OPERATION (Work with PK)

To insert Data we should establish the Connection

#### 1.XML Approach

#### 2.Annotation Approach

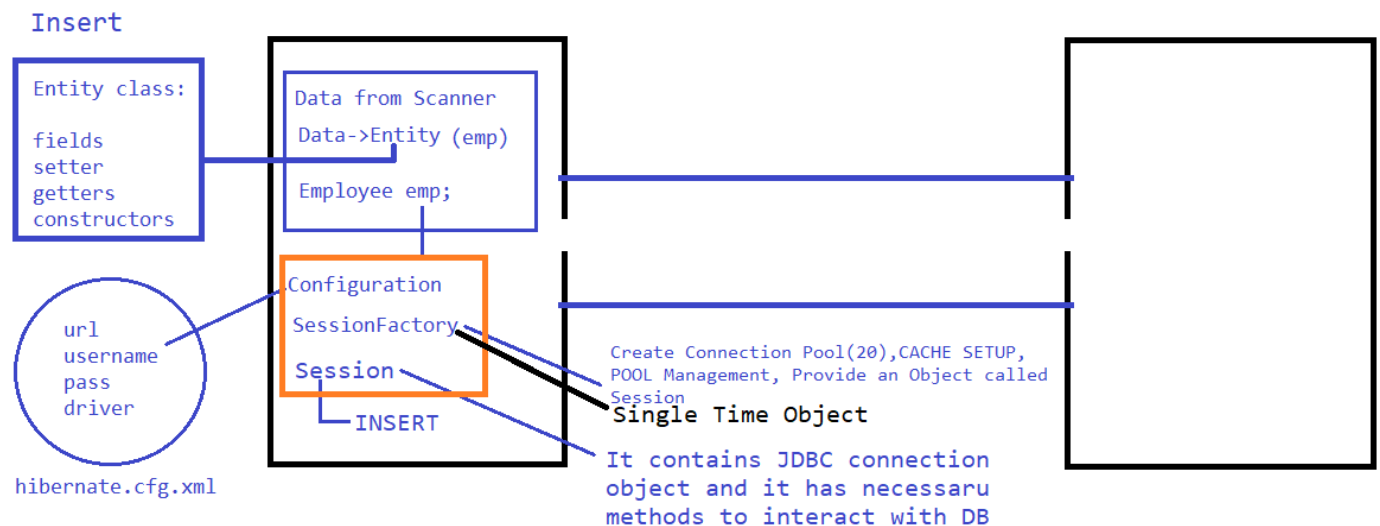
NOTE:



### XML Approach:

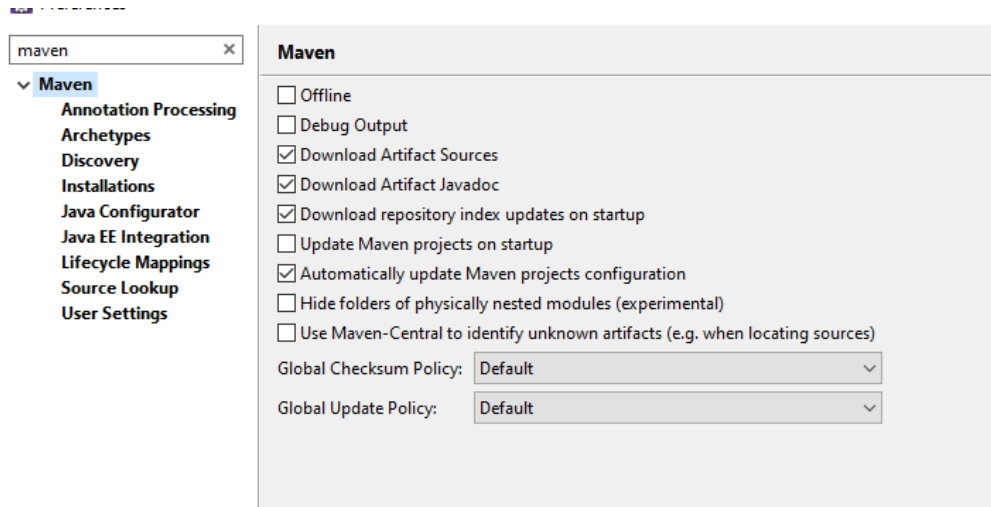
# HBN is a third party framework so we need its JARs

## Version: HBN5.X

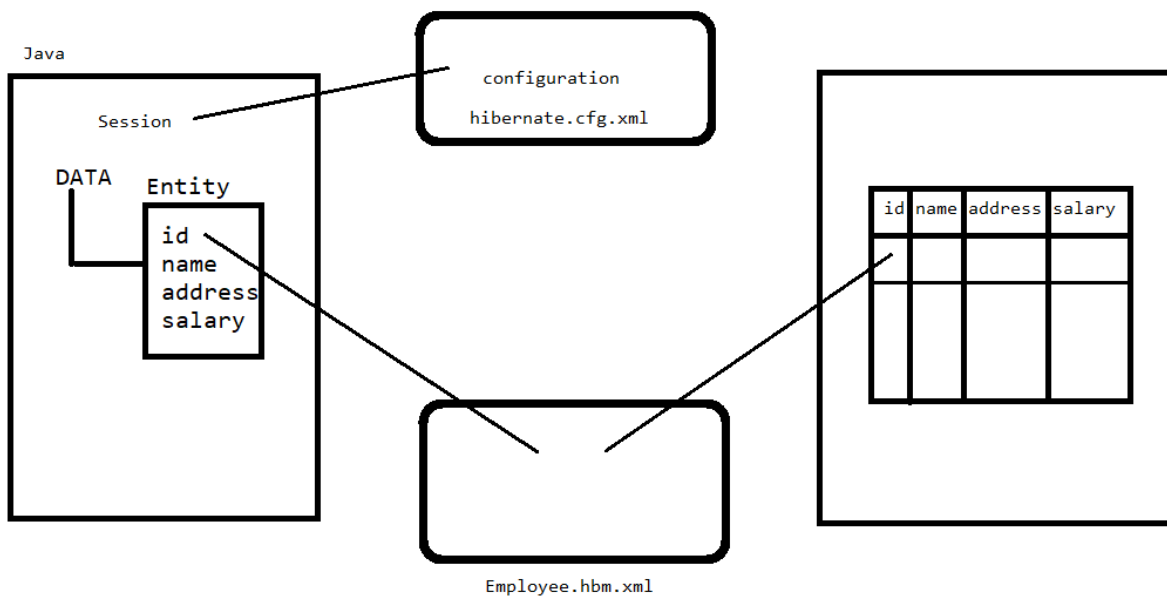


## XML: Language Agnostic

- It needs DOCTYPE DEFINITION (Xml Purpose, Validate, Error check)
- More line of code
- More files
- Configuration and Mapping decoupled (SEPERATION CONCERN)
- DYNAMIC UPDATE
- Legacy application



## CONVENTION OVER CONFIGURATION




---

```
package com.mainapp;
import java.io.Serializable;
import java.util.Scanner;

import org.hibernate.Session;
```

```

import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
public class Launch {

    public static void main(String[] args) {

        //LOADS CFG FILE
        Configuration configuration = new Configuration();
        configuration.configure();

        //SETUP READY
        SessionFactory sessionFactory = configuration.buildSessionFactory();

        //Now U can Perform CRUD (Inbuilt Connection)
        Session session = sessionFactory.openSession();

        //Transaction
        Transaction trs = session.beginTransaction();

        //DATA INSERT

        Scanner scanner = new Scanner(System.in);
        System.out.println("ENTER EID");
        int eid=scanner.nextInt();
        System.out.println("ENTER ENAME");
        String ename=scanner.next();
        System.out.println("ENTER EADDRESS");
        String eaddress=scanner.next();
        System.out.println("ENTER ESALARY");
        int esalary=scanner.nextInt();

        Employee employee = new Employee(eid, ename, eaddress, esalary);
        int pk = (int) session.save(employee);
        trs.commit();

        System.out.println(pk);

        session.close();
        //connection back to pool
        //cache memory free

    }
}

```

---

```

package com.mainapp;
//Entity Class
public class Employee {

    private int eid;
    private String ename;
    private String eaddress;
    private int esalary;

```

```

public Employee(int eid, String ename, String eaddress, int esalary) {
    super();
    this.eid = eid;
    this.ename = ename;
    this.eaddress = eaddress;
    this.esalary = esalary;
}

public int getId() {
    return eid;
}
public void setId(int eid) {
    this.eid = eid;
}
public String getName() {
    return ename;
}
public void setName(String ename) {
    this.ename = ename;
}
public String getAddress() {
    return eaddress;
}
public void setAddress(String eaddress) {
    this.eaddress = eaddress;
}
public int getSalary() {
    return esalary;
}
public void setSalary(int esalary) {
    this.esalary = esalary;
}
@Override
public String toString() {
    return "Employee [eid=" + eid + ", ename=" + ename + ", eaddress=" +
eaddress + ", salary=" + esalary + "]";
}
}

```

---

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

```

```

<hibernate-mapping>

```

```

    <class name="com.mainapp.Employee" table="employee">
        <id name="eid" column="id" ></id>
        <property name="ename" column="name"></property>
        <property name="eaddress" column="address"></property>
    </class>

```



```

        <property name="esalary" column="salary"></property>
    </class>

</hibernate-mapping>

```

---

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <property
name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3309/test</property>
        <property name="connection.username">root</property>
        <property name="connection.password"></property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <mapping resource="Employee.hbm.xml" />
    </session-factory>
</hibernate-configuration>

```

---

# DAY-3

## Single Operation

- Read Based on PK
- Hibernate will use Zero Parameterized Constructor to create object of your Entity class then it will call setter to set all values and you will get object

```
package com.mainapp;
```

```

import java.util.Scanner;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
public class Launch {

    public static void main(String[] args) {

        SessionFactory sf = getConnection();
        Scanner scanner = new Scanner(System.in);

        while (true) {

System.out.println("P1->Insert\nP2->Read\nP3->Update\nP4->Delete\nP5->InsertORUpdate\nP6->EXIT");

            System.out.print("Enter Choice:");
            int chocie = scanner.nextInt();//15 \n
            scanner.nextLine();
            if(chocie==6)
            {
                System.out.println("THANKS FOR USING");
                break;
            }
            switch (chocie) {
            case 1:
                insert(sf);
                break;
            case 2:
                read(sf);
                break;
            case 3:
                update(sf);
                break;
            case 4:
                delete(sf);
                break;
            case 5:
                insertOrUpdate(sf);
                break;
            default:
                break;
            } //end of switch
        } //end of while
    }

    private static void insertOrUpdate(SessionFactory sf) {

        Session session = sf.openSession();
        //Transaction
        Transaction trs = session.beginTransaction();
        //DATA INSERT
        Scanner scanner = new Scanner(System.in);
        System.out.println("ENTER EID"); //11
    }
}

```

```

        int eid=scanner.nextInt();
        System.out.println("ENTER ENAME");
        String ename=scanner.next();
        System.out.println("ENTER EADDRESS");
        String eaddress=scanner.next();
        System.out.println("ENTER ESALARY");
        int esalary=scanner.nextInt();

        Employee employee = new Employee(eid, ename, eaddress, esalary);
        session.saveOrUpdate(employee);
        trs.commit();
    }

```

```

private static void delete(SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();

    Employee employee = session.get(Employee.class, eid);
    if(employee!=null) {

        Transaction trs = session.beginTransaction();
        session.delete(employee);
        trs.commit();
        System.out.println("DATA DELETED");
    }
}

```

```

private static void update(SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();//15

    System.out.println("ENTER NEW SALARY");
    int newsalary=scanner.nextInt();

    Employee employee = session.get(Employee.class, eid);
    if(employee!=null) {

        employee.setEsalary(newsalary);
        Transaction trs = session.beginTransaction();
        session.update(employee);
        trs.commit();
        System.out.println("UPDATED");
    }else {
        System.out.println("NOT EXIST");
    }
}

```

```

}

private static void read (SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();

    Employee employee = session.get(Employee.class, eid);
    if(employee!=null)
        System.out.println(employee);
    else
        System.out.println("NOT EXIST");

    //FIRST LEVEL CACHE
    // employee = session.get(Employee.class, eid);
    // System.out.println(employee);

    session.close();
}

private static void insert(SessionFactory sf) {

    Session session = sf.openSession();
    //Transaction
    Transaction trs = session.beginTransaction();
    //DATA INSERT
    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();
    System.out.println("ENTER ENAME");
    String ename=scanner.next();
    System.out.println("ENTER EADDRESS");
    String eaddress=scanner.next();
    System.out.println("ENTER ESALARY");
    int esalary=scanner.nextInt();

    Employee employee = new Employee(eid, ename, eaddress, esalary);
    int pk = (int) session.save(employee);
    trs.commit();

    System.out.println(pk);

    session.close();
    //connection back to pool
    //cache memory free
}

private static SessionFactory getConnection() {
    //LOADS CFG FILE
    Configuration configuration = new Configuration();
    configuration.configure();
}

```

```

        //SETUP READY
        SessionFactory sessionFactory = configuration.buildSessionFactory();

        return sessionFactory;
    }
}

```

# CRUD PURE JAVA: Mapping and Configuration

```

package com.mainapp;
import java.util.Properties;
import java.util.Scanner;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.cfg.Environment;
public class Launch {

    public static void main(String[] args) {

        SessionFactory sf = getConnection();
        Scanner scanner = new Scanner(System.in);

        while (true) {

System.out.println("P1->Insert\nP2->Read\nP3->Update\nP4->Delete\nP5->InsertORUpdate\nP6->EXIT");

            System.out.print("Enter Choice:");
            int chocie = scanner.nextInt();//15 \n
            scanner.nextLine();
            if(chocie==6)
            {
                System.out.println("THANKS FOR USING");
                break;
            }
            switch (chocie) {
                case 1:
                    insert(sf);
                    break;
                case 2:
                    read(sf);
                    break;
                case 3:
                    update(sf);
                    break;
                case 4:
                    delete(sf);
                    break;
                case 5:

```

```

        insertOrUpdate(sf);
        break;
    default:
        break;
    } //end of switch
} //end of while
}

private static void insertOrUpdate(SessionFactory sf) {

    Session session = sf.openSession();
    //Transaction
    Transaction trs = session.beginTransaction();
    //DATA INSERT
    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID"); //11
    int eid=scanner.nextInt();
    System.out.println("ENTER ENAME");
    String ename=scanner.next();
    System.out.println("ENTER EADDRESS");
    String eaddress=scanner.next();
    System.out.println("ENTER ESALARY");
    int esalary=scanner.nextInt();

    Employee employee = new Employee(eid, ename, eaddress, esalary);
    session.saveOrUpdate(employee);
    trs.commit();
}

private static void delete(SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();

    Employee employee = session.get(Employee.class, eid);
    if(employee!=null) {

        Transaction trs = session.beginTransaction();
        session.delete(employee);
        trs.commit();
        System.out.println("DATA DELETED");
    }
}

private static void update(SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();//15

```

```

System.out.println("ENTER NEW SALARY");
int newsalary=scanner.nextInt();

Employee employee = session.get(Employee.class, eid);
if(employee!=null) {

    employee.setEsalary(newsalary);
    Transaction trs = session.beginTransaction();
    session.update(employee);
    trs.commit();
    System.out.println("UPDATED");

}else {
    System.out.println("NOT EXIST");
}
}

private static void read (SessionFactory sf) {

    Session session = sf.openSession();

    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();

    Employee employee = session.get(Employee.class, eid);
    if(employee!=null)
        System.out.println(employee);
    else
        System.out.println("NOT EXIST");

    //FIRST LEVEL CACHE
    // employee = session.get(Employee.class, eid);
    // System.out.println(employee);

    session.close();
}

private static void insert(SessionFactory sf) {

    Session session = sf.openSession();
    //Transaction
    Transaction trs = session.beginTransaction();
    //DATA INSERT
    Scanner scanner = new Scanner(System.in);
    System.out.println("ENTER EID");
    int eid=scanner.nextInt();
    System.out.println("ENTER ENAME");
    String ename=scanner.next();
    System.out.println("ENTER EADDRESS");
    String eaddress=scanner.next();
    System.out.println("ENTER ESALARY");
    int esalary=scanner.nextInt();

```

```

        Employee employee = new Employee(eid, ename, eaddress, esalary);
        int pk = (int) session.save(employee);
        trs.commit();

        System.out.println(pk);

        session.close();
        //connection back to pool
        //cache memory free
    }

    private static SessionFactory getConnection() {
        //LOADS CFG FILE
        Configuration configuration = new Configuration();

        Properties properties = new Properties();
        properties.put(Environment.URL, "jdbc:mysql://localhost:3309/test");
        properties.put(Environment.USER, "root");
        properties.put(Environment.PASS, "");
        properties.put(Environment.DRIVER, "com.mysql.cj.jdbc.Driver");
        properties.put(Environment.HBM2DDL_AUTO, "update");
        properties.put(Environment.SHOW_SQL, true);
        properties.put(Environment.FORMAT_SQL, true);

        configuration.setProperties(properties);
        configuration.addAnnotatedClass(Employee.class);

        //SETUP READY
        SessionFactory sessionFactory = configuration.buildSessionFactory();
        return sessionFactory;
    }
}

```

```

package com.mainapp;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
//Entity Class
@Entity
@Table(name="myemployee")
public class Employee {

    @Id
    @Column(name = "id")
    private int eid;

    @Column(name = "name" , length = 30 )
    private String ename;

    @Column(name = "address" , length = 500)
    private String eaddress;
}

```



```

@Column(name = "salary")
private int esalary;

public Employee() {
}

public Employee(int eid, String ename, String eaddress, int esalary) {
    super();
    this.eid = eid;
    this.ename = ename;
    this.eaddress = eaddress;
    this.esalary = esalary;
}

public int getEid() {
    return eid;
}
public void setEid(int eid) {
    this.eid = eid;
}
public String getEname() {
    return ename;
}
public void setName(String ename) {
    this.ename = ename;
}
public String getEaddress() {
    return eaddress;
}
public void setEaddress(String eaddress) {
    this.eaddress = eaddress;
}
public int getEsalary() {
    return esalary;
}
public void setEsalary(int esalary) {
    this.esalary = esalary;
}
@Override
public String toString() {
    return "Employee [eid=" + eid + ", ename=" + ename + ", eaddress=" +
eaddress + ", esalary=" + esalary + "]";
}
}

```

# DAY-4

## Complex Data

## Employee

- private int eid
- private String ename;
- private String eaddress;
- private int esalary;
- **private Account eaccount;**
- **private Education eeducation;**

## Account

- private int accNo;
- private String bankName;
- private int amount;

## Education

private int rollNo;

private String clgName;

private int percentage;

**TABLE: 10 COLUMNS**

# DAY-5

# Versioning

USER DATA: INSERT

| id | fname | lname | age | username | password |
|----|-------|-------|-----|----------|----------|
|----|-------|-------|-----|----------|----------|

version

|    |      |      |    |           |        |
|----|------|------|----|-----------|--------|
| 11 | raju | pqrs | 66 | raju@pqrs | 123456 |
|----|------|------|----|-----------|--------|

0

USER DATA: UPDATE

| id | fname | lname | age | username | password |
|----|-------|-------|-----|----------|----------|
|----|-------|-------|-----|----------|----------|

version

|    |      |      |    |          |        |
|----|------|------|----|----------|--------|
| 11 | raju | pqrs | 66 | raju@ops | 123456 |
|----|------|------|----|----------|--------|

3

getVersion->3

**if (version>2)**

**Don't Update**

**else**

**Update**

- BULK OPERATION:

- NATIVE SQL APPROACH
- PAGINATION
- HQL
- CRITERIA

## **DAY-6**

### **BULK OPERATION (READ UPDATE DELETE)**

1.SQL

2.HQL

3.PURE JAVA

### **SQL Approach OR NativeSQL approach**

**@NamedNativeQuery : we can separate out SQL query from the business logic (We can reuse our query)**

without add Entity:

- `List<Object[]>`

eid ename eaddress esalary

1

2

Row1: Object[obj1,obj2,obj3,obj4]

Row2: Object[obj1,obj2,obj3,obj4]

List<Object[]>

# DAY-7

## Pagination

20 Entries

- SetFirstResult=0
- Max result = 4

ROW: 1 2 3 4

- SetFirstResult=4
- Max result = 4

ROW: 5 6 7 8

# DAY-8

## HQL(Hibernate Query Language)

->To perform bulk operation (**READ DELETE UPDATE**)

->**We cannot perform insert operation in HQL**

->you can copy data from one table to another

## HQL is database independent

x employee

| x eid | ename |
|-------|-------|
|       |       |

|            |
|------------|
| SQL Oracle |
|------------|

|           |
|-----------|
| SQL Mysql |
|-----------|

create database xyz;

```
select * from employee
select * from employee where eid=:eid
delete from employee where eid=:eid
update employee set ename=:ename where eid=:eid
count
min
max
```

## SQL

- From table name

- select \* from emPloyEe;(Valid)

## HQL

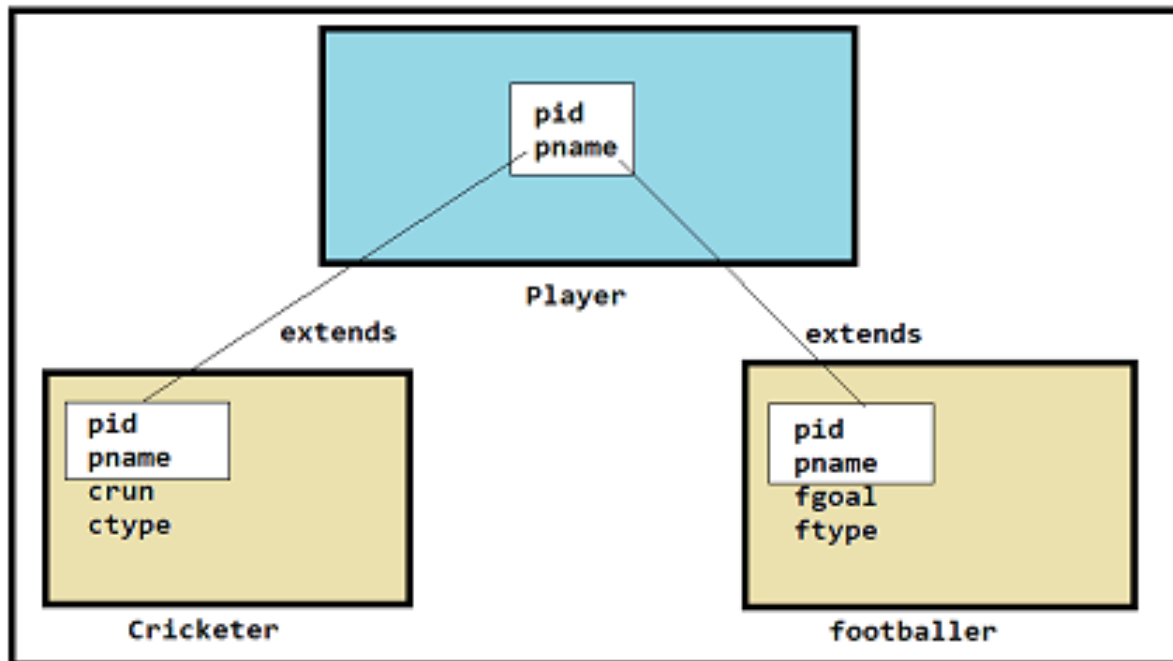
- From Entity Name
  - from Employee; (Valid)
  - from employee; (Invalid)
- 
- insert into tb(colms...) values(....)  
(INVALID)
  - into into tb1(colms...) select from tb2  
(VALID)

# DAY-9

## HQL Polymorphic query

### Subtopic: Hibernate Inheritance

```
@Inheritance(strategy = InheritanceType.JOINED)
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
```



1. Joined table
2. single table
3. table per class

| pid | pname | crun | ctype |
|-----|-------|------|-------|
| 11  | dho   | 98   |       |
| 7   | ron   | null |       |

# DAY-10

## HQL Supports Polymorphic query ( from Parent class -> Child classes access )

```

private static void readHQL(SessionFactory sf) {

    Session session = sf.openSession();
    String hql="from Player";

    Query query = session.createQuery(hql);
    List<Player> list = query.list();

    for(Player p : list) {

        if(p instanceof Cricketer) {

            Cricketer c=(Cricketer) p;
            System.out.println(c);
        }
    }
}
  
```



```

    }
    else {
        Footballer f=(Footballer) p;
        System.out.println(f);
    }
}

}

private static void readSQL(SessionFactory sf) {

    Session session = sf.openSession();

    String sql="select p.pid, p.pname, c.crun, c.ctype from
player p join cricketer c on p.pid=c.pid "
        + "UNION ALL "
        +"select p.pid, p.pname, f.fgoal, f.ftype from player
p join footballer f on p.pid=f.pid";

    NativeQuery nativeQuery = session.createNativeQuery(sql);
    List<Object[]> list = nativeQuery.getResultList();

    for(Object[] orr : list) {

        for(Object obj: orr) {
            System.out.print(obj+" ");
        }
        System.out.println();
    }
}

```

# DAY-11

Hibernate Mapping : it defines how java objects map to the Relational database tables

Object [ Object ] → Separate Table

- One to One Mapping  
->Unidirectional ->Bidirectional
- One to Many Mapping
- Many to One Mapping
- Many to Many Mapping

```
package com.mainapp;
import java.util.Properties;
import java.util.Scanner;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.cfg.Environment;
public class Launch {

    public static void main(String[] args) {

        SessionFactory sf = getConnection();
        Scanner scanner = new Scanner(System.in);

        while (true) {

System.out.println("P1->Insert\nP2->ReadFromPlayer\nP3->Read
FromAdhaar\nP4->Delete\nP5->EXIT");
            System.out.print("Enter Choice:");
            int chocie = scanner.nextInt();
            scanner.nextLine();
            if (chocie == 5) {
                System.out.println("THANKS FOR USING");
                break;
            }
            switch (chocie) {
            case 1:
                insert(sf);
```

```

        break;
    case 2:
        readFromPlayer(sf);
        break;

    case 3:
        readFromAhdaar(sf);
        break;

    default:
        break;
    }
}
}

```

```

private static void readFromAhdaar(SessionFactory sf) {

    Session session = sf.openSession();
    Adhaar adhaar = session.get(Adhaar.class, 121212);

    System.out.println(adhaar.getAdhaarNo());
    System.out.println(adhaar.getAdhaarName());
    System.out.println(adhaar.getPlayer().getPid());
    System.out.println(adhaar.getPlayer().getPname());

}

```

```

private static void readFromPlayer(SessionFactory sf) {
    Session session = sf.openSession();
    Player player = session.get(Player.class, 11);
    System.out.println(player);
}

```

```

private static void insert(SessionFactory sf) {
    Session session = sf.openSession();
    Transaction t = session.beginTransaction();

    Adhaar adhaar = new Adhaar(121212, "xyz oo");
    Player p = new Player(11, "xyz", adhaar);
}

```

```

        adhaar.setPlayer(p);

        session.save(p);

        t.commit();
    }

    private static SessionFactory getConnection() {
        Configuration configuration = new Configuration();
        Properties properties = new Properties();
        properties.put(Environment.URL,
"jdbc:mysql://localhost:3309/mapping");
        properties.put(Environment.USER, "root");
        properties.put(Environment.PASS, "");
        properties.put(Environment.DRIVER,
"com.mysql.cj.jdbc.Driver");
        properties.put(Environment.HBM2DDL_AUTO, "create");
        properties.put(Environment.SHOW_SQL, true);
        properties.put(Environment.FORMAT_SQL, true);
        configuration.setProperties(properties);
        configuration.addAnnotatedClass(Player.class);
        configuration.addAnnotatedClass(Adhaar.class);
        SessionFactory sessionFactory =
configuration.buildSessionFactory();
        return sessionFactory;
    }
}

```

```

package com.mainapp;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.OneToOne;
import javax.persistence.Table;
@Entity
@Table(name = "adhaar")

```

```

public class Adhaar {

    @Id
    @Column(name = "adhaarno")
    private int adhaarNo;

    @Column(name = "adhaarname")
    private String adhaarName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "pid")
    private Player player;

    public Adhaar() {
        // TODO Auto-generated constructor stub
    }

    public Adhaar(int adhaarNo, String adhaarName) {
        super();
        this.adhaarNo = adhaarNo;
        this.adhaarName = adhaarName;
    }

    public Adhaar(int adhaarNo, String adhaarName, Player
player) {
        super();
        this.adhaarNo = adhaarNo;
        this.adhaarName = adhaarName;
        this.player = player;
    }

    public int getAdhaarNo() {
        return adhaarNo;
    }

    public void setAdhaarNo(int adhaarNo) {
        this.adhaarNo = adhaarNo;
    }
}

```

```

    public String getAdhaarName() {
        return adhaarName;
    }

    public Player getPlayer() {
        return player;
    }

    public void setPlayer(Player player) {
        this.player = player;
    }

    public void setAdhaarName(String adhaarName) {
        this.adhaarName = adhaarName;
    }

    // @Override
    // public String toString() {
    //     return "Adhaar [adhaarNo=" + adhaarNo + ",
    //     adhaarName=" + adhaarName + "];
    // }

    // @Override
    // public String toString() {
    //     return "Adhaar [adhaarNo=" + adhaarNo + ",
    //     adhaarName=" + adhaarName + ", player=" + player + "];
    // }

}

```

```

package com.mainapp;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.OneToOne;
import javax.persistence.Table;

```

```

@Entity
@Table(name = "player")
public class Player {

    @Id
    @Column(name = "pid")
    private int pid;

    @Column(name = "pname")
    private String pname;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "adhaarno")
    private Adhaar adhaar;

    public Player() {
        // TODO Auto-generated constructor stub
    }

    public Player(int pid, String pname, Adhaar adhaar) {
        super();
        this.pid = pid;
        this.pname = pname;
        this.adhaar = adhaar;
    }

    public int getPid() {
        return pid;
    }

    public void setPid(int pid) {
        this.pid = pid;
    }

    public String getPname() {
        return pname;
    }

    public void setPname(String pname) {

```

```

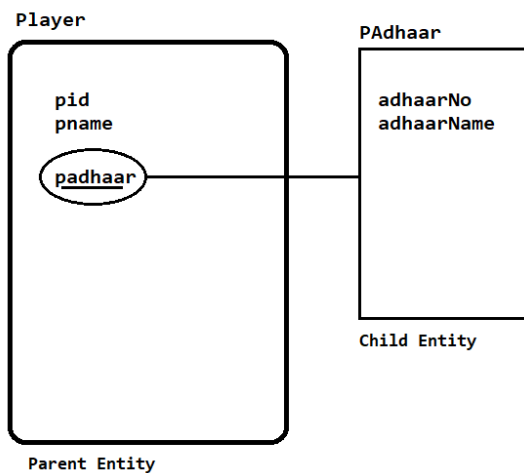
        this.pname = pname;
    }

    public Adhaar getAdhaar() {
        return adhaar;
    }

    public void setAdhaar(Adhaar adhaar) {
        this.adhaar = adhaar;
    }

    @Override
    public String toString() {
        return "Player [pid=" + pid + ", pname=" + pname +
            ", adhaar=" + adhaar + "]";
    }
}

```



| id | name | adhaarNo |
|----|------|----------|
| 11 | xyz  | 121212   |
|    |      |          |

Player

| adhaarNo | adhaarName |
|----------|------------|
| 121212   | xyz oo     |
|          |            |

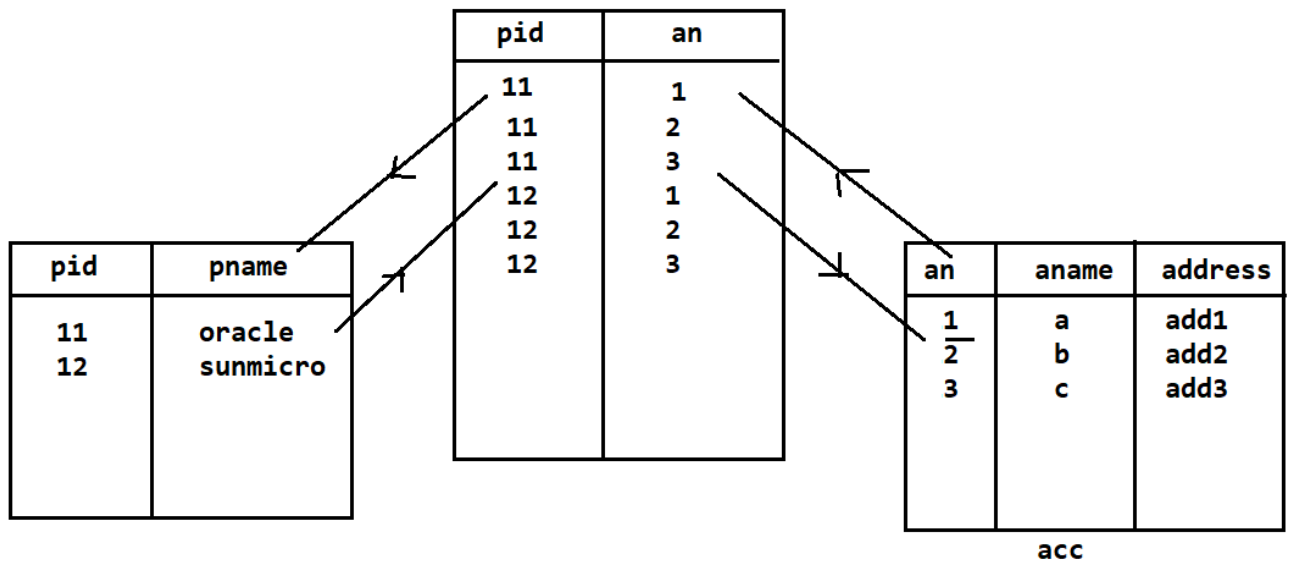
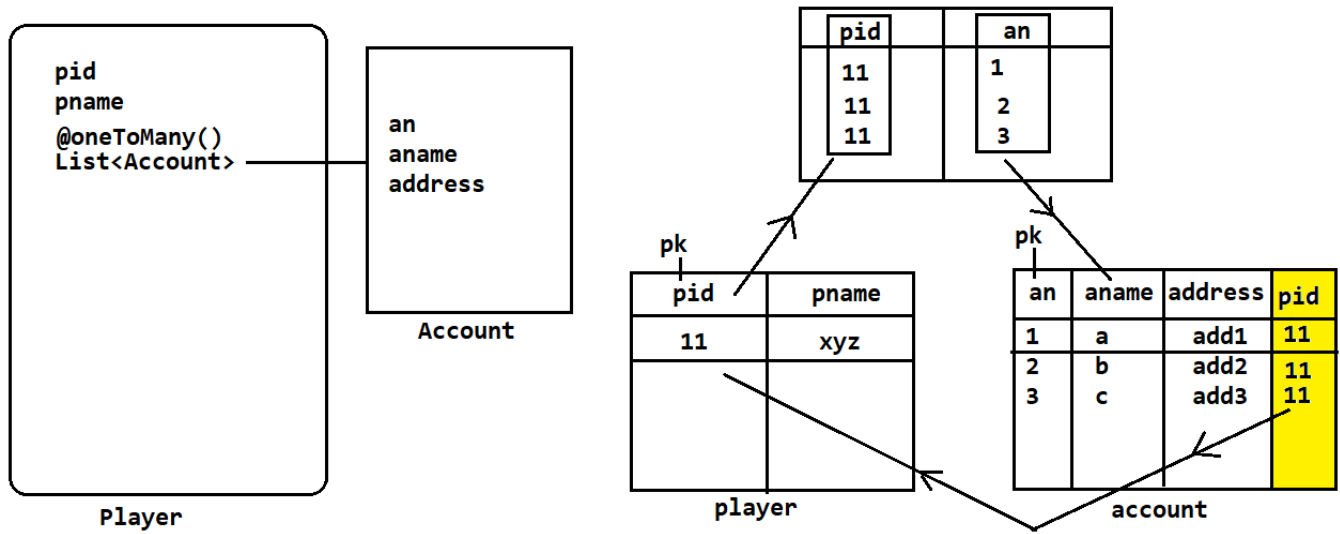
Adhaar

# DAY-12

- ONE TO MANY
- MANY TO ONE



- MANY TO MANY (UNI/BI)



# DAY-13

## CRITERIA API

->It is a feature provided by JPA to construct dynamic and database independent query to interact with databases

->this is pure Java approach

->You can perform BULK Operation (READ UPDATE DELETE)

->ClassName and field Names

```
package com.mainapp;
import java.util.List;
import java.util.Properties;
import java.util.Scanner;
import javax.persistence.criteria.CriteriaBuilder;
import javax.persistence.criteria.CriteriaDelete;
import javax.persistence.criteria.CriteriaQuery;
import javax.persistence.criteria.CriteriaUpdate;
import javax.persistence.criteria.Predicate;
import javax.persistence.criteria.Root;
import org.hibernate.Criteria;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.cfg.Environment;
import org.hibernate.criterion.ProjectionList;
import org.hibernate.criterion.Projections;
import org.hibernate.criterion.Restrictions;
import org.hibernate.criterion.SimpleExpression;
import org.hibernate.query.Query;
public class Launch {

    public static void main(String[] args) {

        SessionFactory sf = getConnection();
        Scanner scanner = new Scanner(System.in);

        while (true) {

            System.out.println("P1->Insert\nP2->Read\nP3->Update\nP4->Delete\nP5->
Test");

            System.out.print("Enter Choice:");
            int chocie = scanner.nextInt();
            scanner.nextLine();
            if (chocie == 6) {
                System.out.println("THANKS FOR USING");
                break;
            }
        }
    }
}
```

```

        switch (chocie) {
        case 1:
            insert(sf);
            break;
        case 2:
            read(sf);
            break;

        case 3:
            update(sf);
            break;

        case 4:
            delete(sf);
            break;

        case 5:
            test(sf);
            break;

        default:
            break;
        }
    }
}

```

```

private static void test(SessionFactory sf) {
    Session session = sf.openSession();
    Criteria criteria = session.createCriteria(Player.class);

    //pid,pname-> Projection

    ProjectionList projectionList =
Projections.projectionList();
    projectionList.add(Projections.property("pid"));
    projectionList.add(Projections.property("pname"));

    SimpleExpression lt = Restrictions.lt("pid",4);
    criteria.setProjection(projectionList);
    criteria.add(lt);

    List<Object[]> list = criteria.list();
    for(Object orr[] : list) {
        for(Object o : orr) {
            System.out.println(o);
        }
    }
}

```

```

    }
}

private static void insert(SessionFactory sf) {
    Session session = sf.openSession();

    for(int i=1;i<=10;i++) {
        Transaction t = session.beginTransaction();
        Player player = new Player("raju"+i, "CSK"+i);
        session.save(player);
        t.commit();
    }
}

private static void read(SessionFactory sf) {
    Session session = sf.openSession();

    //IT WILL PROVIDE YOU CRITERIA EX. READ UPDATE DELETE
    CriteriaBuilder criteriaBuilder =
session.getCriteriaBuilder();
    CriteriaQuery<Player> readCriteria =
criteriaBuilder.createQuery(Player.class); //READ OPERATION
    Root<Player> root = readCriteria.from(Player.class); //YOU
CAN SELECT PID PNAME PADDRESS

    //select id,name from criteriaplayer where id<6 and name
like '%2'

    readCriteria.multiselect(root.get("pid"),root.get("pname"));
//select id,name from criteriaplayer

    Predicate c1 = criteriaBuilder.lessThan(root.get("pid"), 6);
//id<6

    Predicate c2 = criteriaBuilder.like(root.get("pname"), "%2");
//name like '%2'

    readCriteria.where(criteriaBuilder.and(c1,c2)); //where id<6
and name like '%2'

    Query<Player> query = session.createQuery(readCriteria);
    List<Player> list = query.list();
    System.out.println(list);
}

private static void update(SessionFactory sf) {

```

```

        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        //update criteriaplayer set name='kaju' where id<5

        CriteriaBuilder criteriaBuilder =
session.getCriteriaBuilder();
        CriteriaUpdate<Player> updateCriteria =
criteriaBuilder.createCriteriaUpdate(Player.class);
        Root<Player> root = updateCriteria.from(Player.class);

        updateCriteria.set(root.get("pname"), "kaju"); // set
name='kaju'
        updateCriteria.where(criteriaBuilder.lessThan(
root.get("pid"), 5));

        Query<Player> query = session.createQuery(updateCriteria);
        query.executeUpdate();
        t.commit();
    }
    private static void delete(SessionFactory sf) {
        Session session = sf.openSession();

        Transaction t = session.beginTransaction();
        //delete from criteriaplayer where id>6

        CriteriaBuilder criteriaBuilder =
session.getCriteriaBuilder();
        CriteriaDelete<Player> criteriaDelete =
criteriaBuilder.createCriteriaDelete(Player.class);
        Root<Player> root = criteriaDelete.from(Player.class);
        criteriaDelete.where(criteriaBuilder.greaterThan(
root.get("pid"), 6));

        Query<Player> query = session.createQuery(criteriaDelete);
        query.executeUpdate();
        t.commit();
    }

    private static SessionFactory getConnection() {
        Configuration configuration = new Configuration();
        Properties properties = new Properties();
        properties.put(Environment.URL,
"jdbc:mysql://localhost:3309/mapping");
        properties.put(Environment.USER, "root");
    }

```

```

        properties.put(Environment.PASS, "");
        properties.put(Environment.DRIVER,
"com.mysql.cj.jdbc.Driver");
        properties.put(Environment.HBM2DDL_AUTO, "update");
        properties.put(Environment.SHOW_SQL, true);
        //properties.put(Environment.FORMAT_SQL, true);
        configuration.setProperties(properties);
        configuration.addAnnotatedClass(Player.class);
        SessionFactory sessionFactory =
configuration.buildSessionFactory();
        return sessionFactory;
    }
}

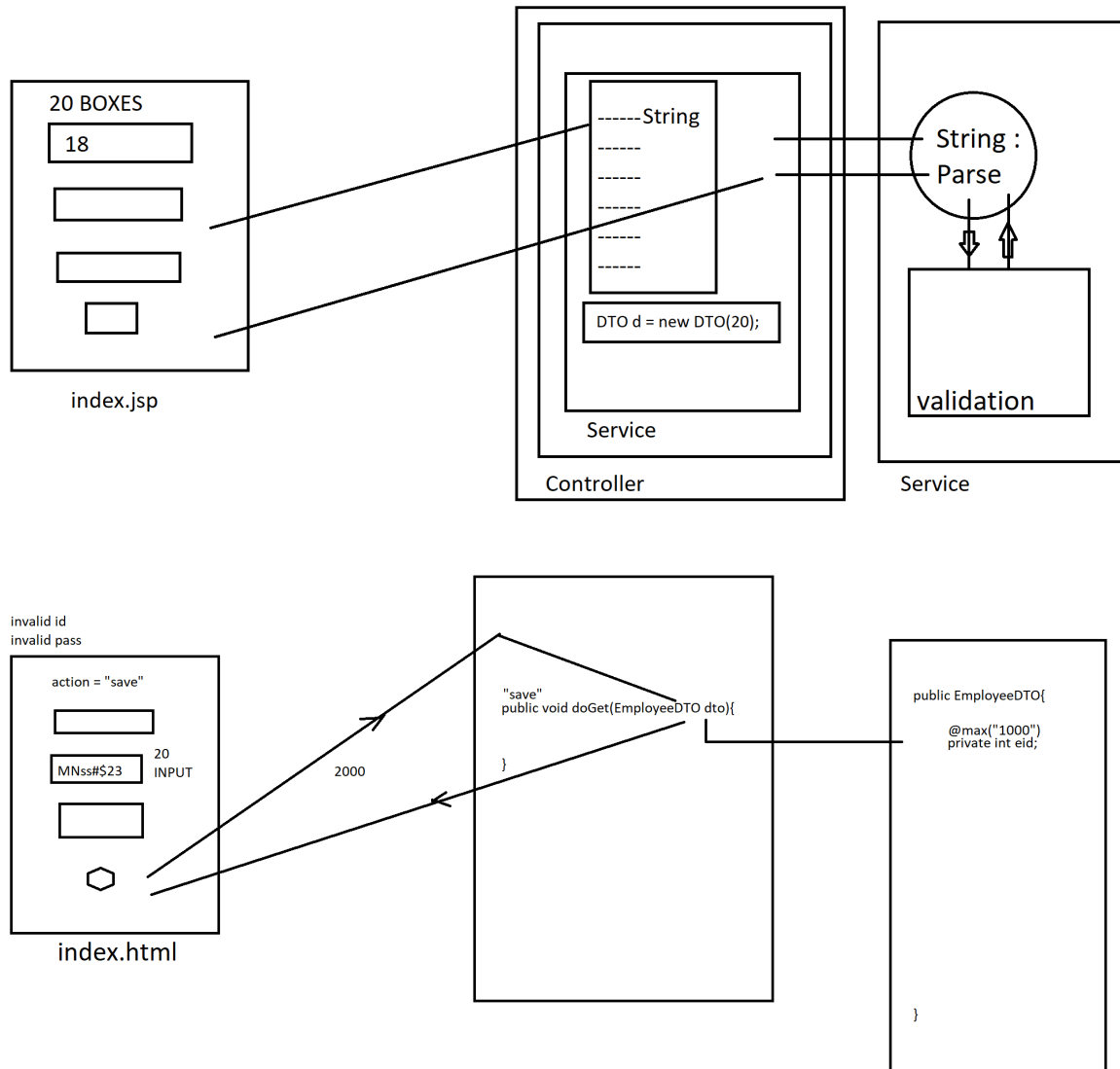
```

# SPRING FRAMEWORK

## DAY 1:

### Why we should learn Spring Framework?

- >To simply development
- >We can develop application rapidly
- >Spring works for every layer
- >it reduces line of code
- >easy to maintance
- >Supports Robustness
- >Best Suitable for enterprise application development



## WEB APPLICATION Ex. Amazon

## DESKTOP GUI APPLICATION Ex. PAINT

## STANDALONE APPLICATION Ex. CONSOLE BASED APPLICATION

## CLOUD APPLICATION Ex. AWS GDRIVE

## DISTRIBUTED APPLICATION Ex. Amazon

## EMBEDDED APPLICATION Ex. C/C++ embedded application/shadi wala board

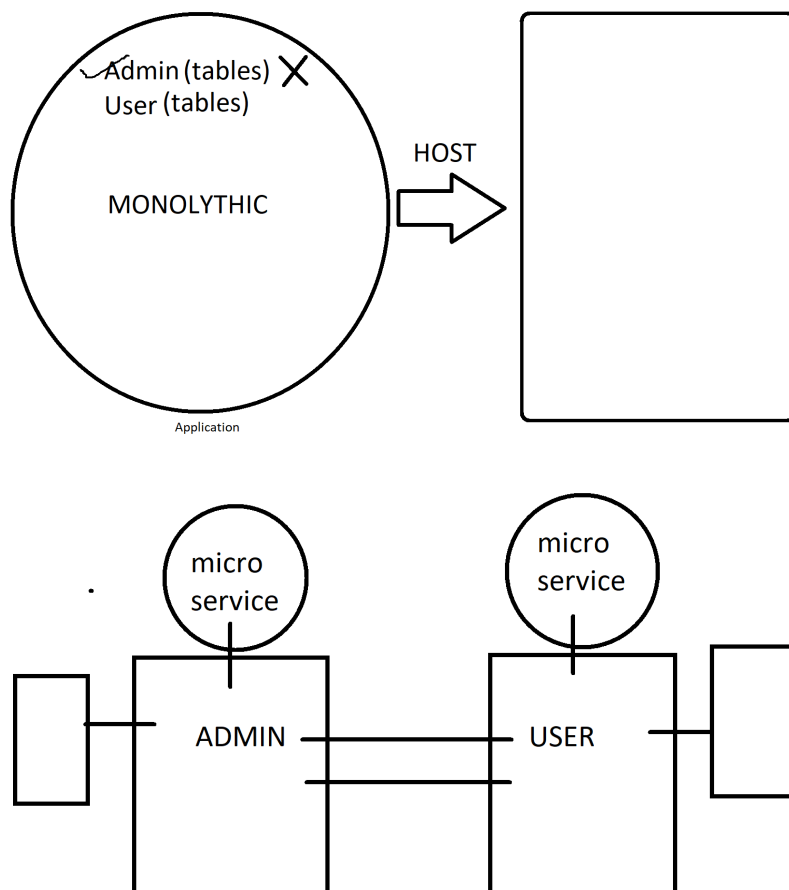
IOT APPLICATION Ex. HOIUSE HOLD SMART DEVICES

**PRESENTATION LAYER->BUSINESS LAYER->DATA  
STORAGE LAYER**

**View->Controller->Service->Dao->Storage**

**PYTHON----->JAVA(DATABASE) RESTAPI**

**MICROSERVICES (MULTIPLE SERVERS)**



**What is Framework?**



->it is a software arch that provides a structure, guidelines and reusable component to facilitate the development

->Less Control

->It uses libraries

## What is Spring Framework?

Spring is an Open Source Framework

Spring is a java platform that provides complete infrastructure to support Java development; spring handles the infrastructure so that u can focus on main business logic

Spring is a lightweight framework

Spring supports **pojo based** programming

### Spring Modules:

1. Spring Core
2. Spring JDBC
3. Spring MVC

# SPRING FRAMEWORK

## DAY 2:

### Spring Core

- >It is a module of spring framework
- >it teaches how to manage objects

## **OBJECT MANAGEMENT**

- >Object Creation
- >Object life cycle
- >Object injection/Dependency Injection

Note: new keyword creates tight coupling

Launch----->Demo ( BEAN )  
(main)

## **Spring IOC(Inversion of control) Container**

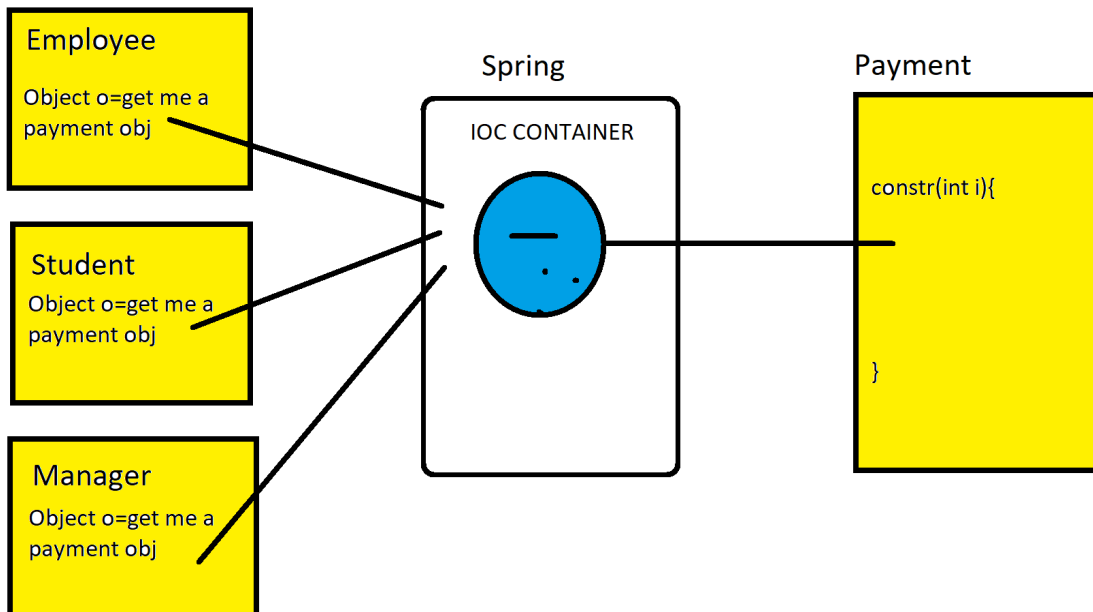
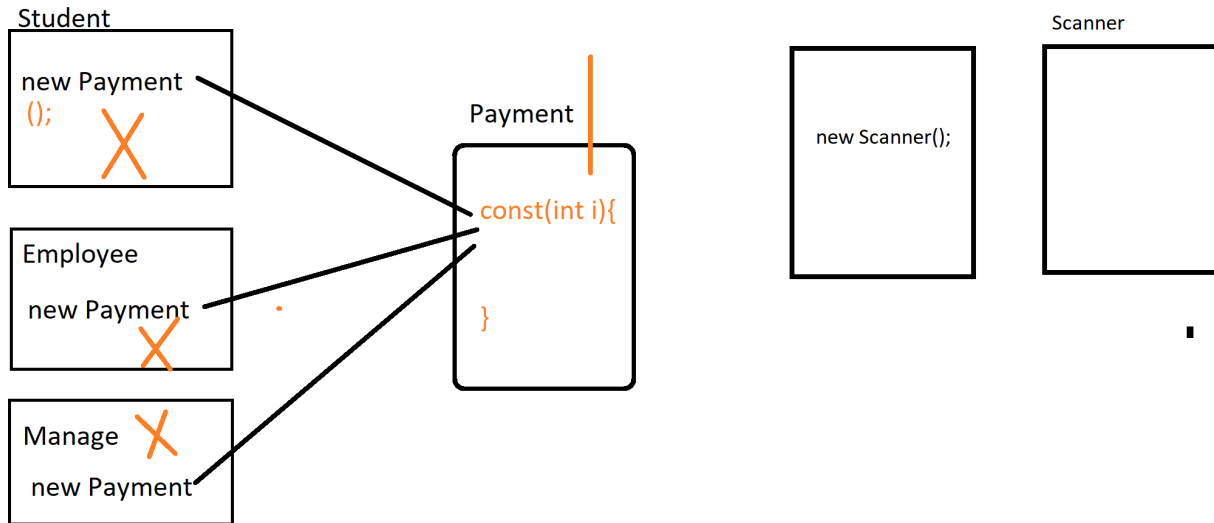
- >it is a core component of Spring Core
- >it is used to manage objects often referred as bean
- >it acts like a boss

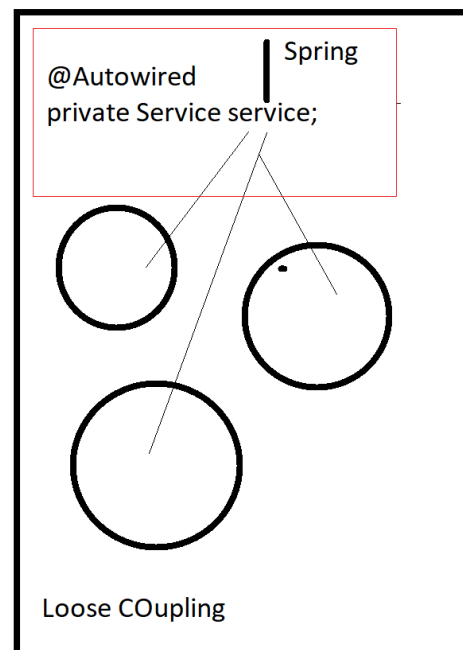
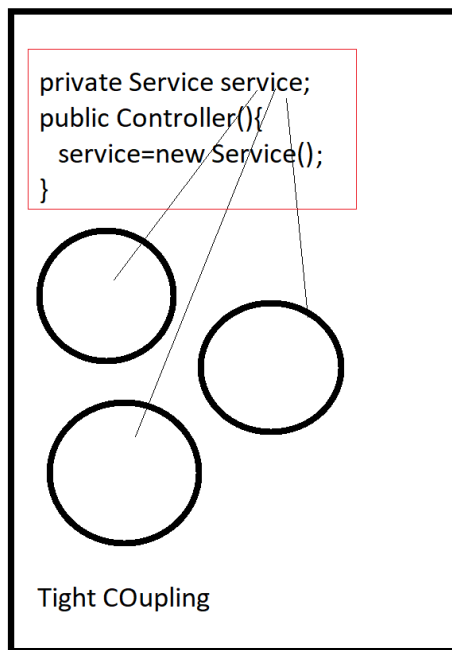
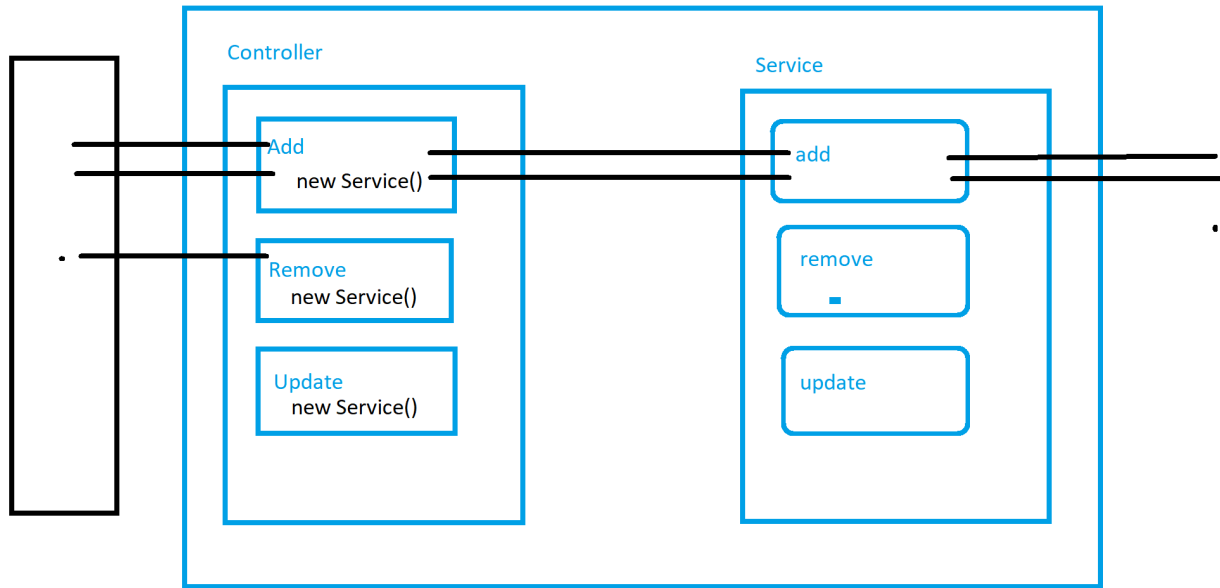
## **Types of IOC container**

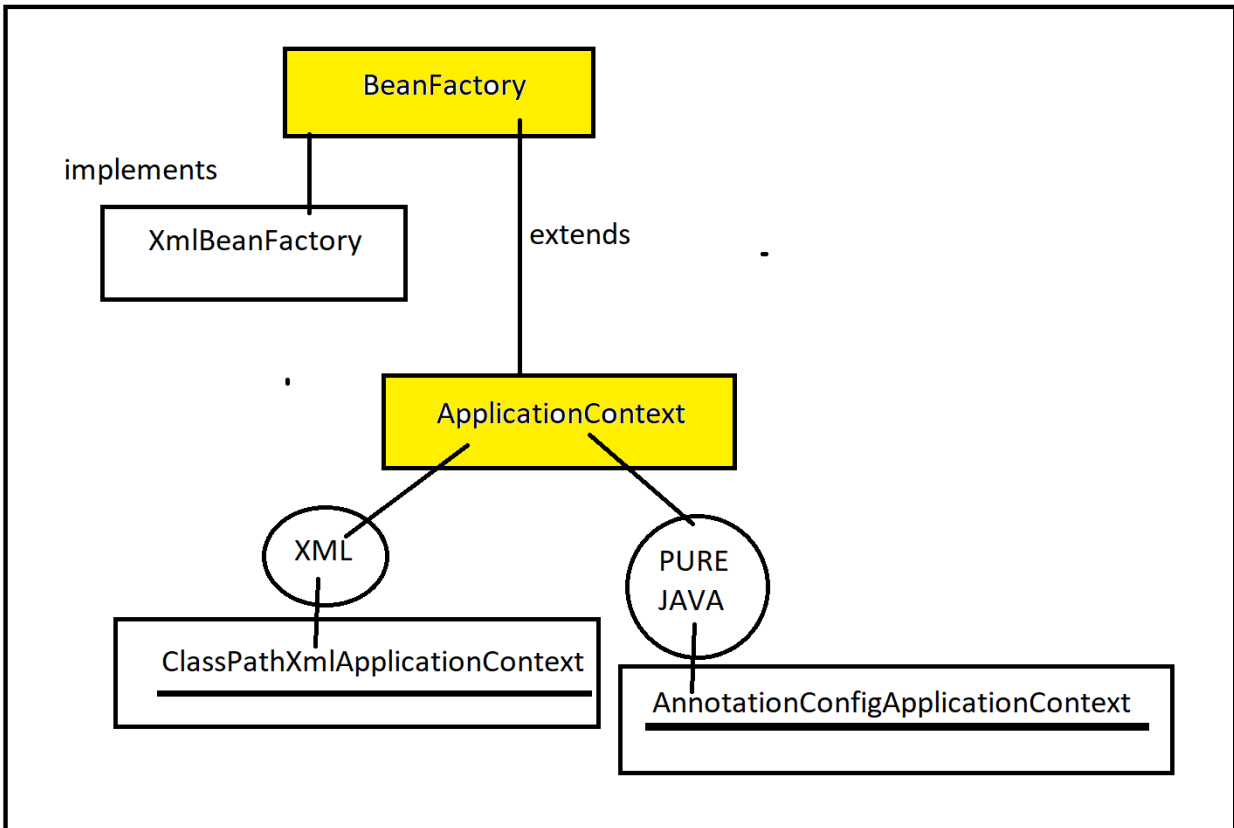
### **1.BeanFactory (Deprecated) (Interface) 3.X**

## 2.ApplicationContext(Latest) (Interface)

IOC: it is a design pattern (1990's, Hollywood Principle: don't call us, we will call you)







# SPRING FRAMEWORK

## DAY 3:

BeanFactory IOC Container

Impl: **XmlBeanFactory**

1. Lazy Loading
2. Resource Object

### 3.Less Features

## Spring Core Jars: (5.3.1)

- Spring core jar
- Spring beans jar
- Spring jcl jar
- Spring context jar
- Spring expression jar

Note: Interface 21(Old Name)

## ApplicationContext IOC Container

Impl: **ClassPathXmlApplicationContext**

1.Eager Loading

2.More Features

# SPRING FRAMEWORK

## DAY 4:

Bean Scope:

- Singleton (by default) : Object will be created only once and each time when you ask for object it will share same object
- Prototype (Object will create when u invoke getBean() and every time IOC container will provide a new Object)

```

package com.mainapp;

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");
        System.out.println("RAJU ACCOUNT CREATION");

        Account acc1=(Account)ac.getBean("acc");
        acc1.setUsername("RAJU123");
        acc1.setPassword("123456");
        acc1.setFullName("RAJU KK");
        acc1.setAddress("MUMBAI");

        System.out.println(acc1);

        System.out.println("KAJU ACCOUNT CREATION");

        Account acc2=(Account)ac.getBean("acc");
        acc2.setUsername("KAJU123");
        acc2.setPassword("223456");
        acc2.setFullName("KAJU KK");
        acc2.setAddress("HYDRABAD");

        System.out.println(acc2);

        System.out.println("****BOTH ACCOUNT DETAILS****");
        System.out.println(acc1);
        System.out.println(acc2);
    }
}
package com.mainapp;

public class Account {

    private String username;
    private String password;
    private String fullName;
    private String address;

    public Account() {
        System.out.println("ACC INSTANTIATION");
    }

    public String getUsername() {
        return username;
    }
}

```

```

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getFullName() {
        return fullName;
    }

    public void setFullName(String fullName) {
        this.fullName = fullName;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Account [username=" + username + ", password=" + password + ",
fullName=" + fullName + ", address="
            + address + "]";
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util" xsi:schemaLocation="
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions
here -->

    <bean id="acc" class="com.mainapp.Account" scope="prototype" > </bean>

</beans>

```



# Dependency Injection through XML

->Dependency injection term is used to refer injecting a dependent object of a class into another class.

1.Setter Injection

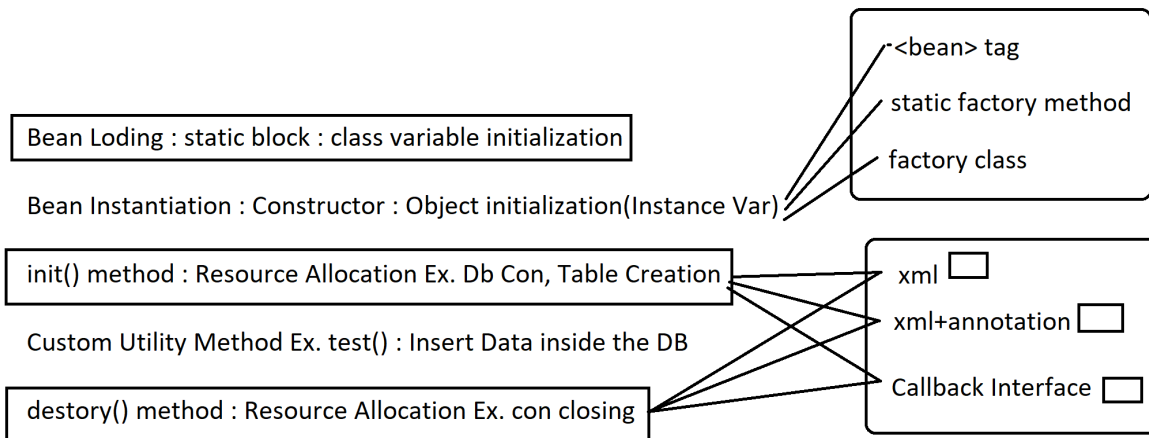
2.Constructor Injection

```
Controller{  
    private Service service; //Dependency  
    public Controller(){  
        service=new Service(); //Constructor Dependency Injection  
    }  
    public void doPost(){  
        service.test(); //NullPointerException  
    }  
}  
public Service{  
    test(){ }  
}
```

## **SPRING FRAMEWORK**

### **DAY 5:**

# Bean Life Cycle



->it refers to the series of steps a bean goes through from loading to destruction

## init method through pure xml

in case of prototype scope you need to call destroy method explicitly

```
package com.mainapp;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("bean.xml");

        EmployeeBean bean1=(EmployeeBean) ac.getBean("emp");
        bean1.test();

        EmployeeBean bean2=(EmployeeBean) ac.getBean("emp");
        bean2.test();

        EmployeeBean bean3=(EmployeeBean) ac.getBean("emp");
        bean3.test();

        ac.close();
    }
}
package com.mainapp;
public class EmployeeBean {

    static
    {
```

```

        System.out.println("BEAN LOADING");
    }
    public EmployeeBean()
    {
        System.out.println("BEAN INSTANTIATION");
    }

    public void myXmlInit() {
        System.out.println("RESOURCE ALLOCATION");
    }

    public void test() {
        System.out.println("CUSTOM METHOD");
    }

    public void myXmlDestroy() {
        System.out.println("RESOURCE DE-ALLOCATION");
    }
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util" xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
        http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions here -->

    <bean
        id="emp"
        class="com.mainapp.EmployeeBean"
        init-method="myXmlInit"
        destroy-method="myXmlDestroy"
        scope="prototype"
    > </bean>

</beans>

```

For Annotation and Callback Interface you need on extra jar aop.jar

PostConstruct ( Java Specification )

PreDestroy ( Java Specification )

From Java 1.9 these annotation has been remove and since java 1.9 onwards you have to download external jar javax annotation jar

# SPRING FRAMEWORK

## DAY 6:

### CallBack Interface ( Less Common )

#### IntializingBean DisposableBean

```
package com.mainapp;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");

        EmployeeBean bean1=(EmployeeBean) ac.getBean("emp");
        bean1.test();

        ac.close();
    }
}
package com.mainapp;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;

public class EmployeeBean implements InitializingBean,DisposableBean {

    static {
        System.out.println("BEAN LOADING");
    }
}
```

```

    public EmployeeBean() {
        System.out.println("BEAN INSTANTIATION");
    }

    public void myXmlInit() {
        System.out.println("RESOURCE ALLOCATION-XML");
    }

    @PostConstruct
    public void myAnnoInit() {
        System.out.println("RESOURCE ALLOCATION-ANNO");
    }

    public void test() {
        System.out.println("CUSTOM METHOD");
    }

    //init method
    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("RESOURCE ALLOCATION-CALL");
    }

    //destroy
    @Override
    public void destroy() throws Exception {
        System.out.println("RESOURCE DE-ALLOCATION-CALL");
    }

    public void myXmlDestroy() {
        System.out.println("RESOURCE DE-ALLOCATION-XML");
    }

    @PreDestroy
    public void myAnnoDestroy() {
        System.out.println("RESOURCE DE-ALLOCATION-ANNO");
    }
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util" xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
        http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions
here -->

<bean

```

```

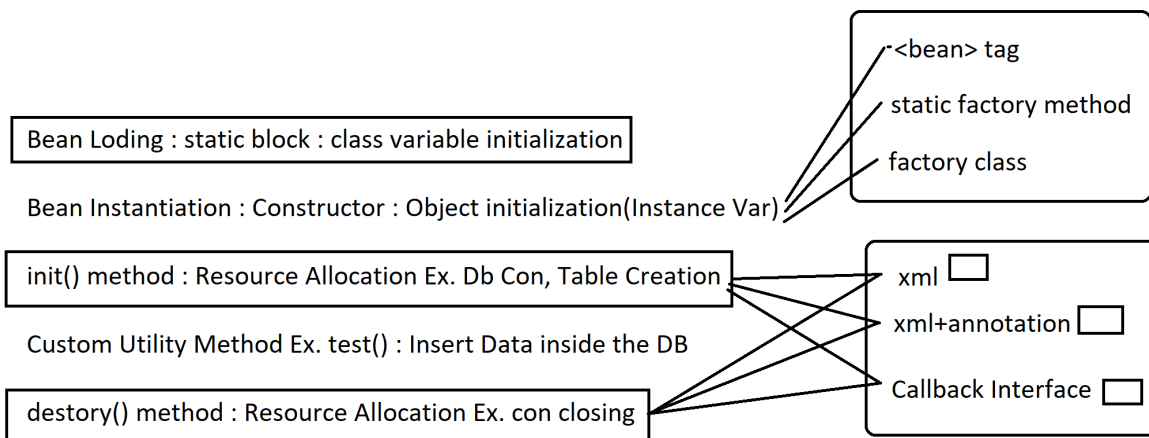
    id="emp"
    class="com.mainapp.EmployeeBean"
    init-method="myXmlInit"
    destroy-method="myXmlDestroy"
  > </bean>
</bean>

  <bean
    class="org.springframework.context.annotation.CommonAnnotationBeanPostProcessor"
  > </bean>
</beans>

```

## Note:

RESOURCE ALLOCATION-ANNO  
 RESOURCE ALLOCATION-CALL  
 RESOURCE ALLOCATION-XML



# Bean Instantiation:

## Simple <bean> Tags

### Static Factory Method

```

package com.mainapp;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

```

```

        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("bean.xml");

        EmployeeBean bean1=(EmployeeBean) ac.getBean("emp");
        bean1.test();

        ac.close();
    }
}
package com.mainapp;

import java.util.Scanner;

public class EmployeeBean {

    private String companyCode;

    static {
        System.out.println("BEAN LOADING");
    }

    public EmployeeBean(String companyCode) {
        this.companyCode=companyCode;
        System.out.println("BEAN INSTANTIATION");
    }

    public void test() {
        System.out.println("CUSTOM METHOD: "+companyCode);
    }

    public static EmployeeBean customObject() {
        Scanner scanner = new Scanner(System.in);
        System.out.println("ENTER KEY");

        EmployeeBean employeeBean;
        int key=scanner.nextInt();
        if(key==1212) {
            employeeBean=new EmployeeBean("QWERTY");
        }
        else {
            employeeBean=new EmployeeBean("PQRS");
        }

        return employeeBean;
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util" xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
        http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions here -->

```

```

<!-- <bean
    id="emp"
    class="com.mainapp.EmployeeBean"
>

```

```

<constructor-arg name="companyCode" value="QWERTY">
    enter key: correct->QWERY otherwise PQRS
</constructor-arg>

```

```

</bean> -->

<bean
  id="emp"
  class="com.mainapp.EmployeeBean"
  factory-method="customObject"
>
</bean>

<bean
  class="org.springframework.context.annotation.CommonAnnotationBeanPostProcessor"
>
</bean>

</beans>

```

# Factory class

```

package com.mainapp;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("bean.xml");

        EmployeeBean bean1=(EmployeeBean) ac.getBean("emp");
        bean1.test();

        ac.close();
    }
}
package com.mainapp;
import java.util.Scanner;
public class EmployeeBean {

    private String companyCode;

    static {
        System.out.println("BEAN LOADING");
    }

    public EmployeeBean(String companyCode) {
        this.companyCode=companyCode;
        System.out.println("BEAN INSTANTIATION");
    }

    public void test() {
        System.out.println("CUSTOM METHOD: "+companyCode);
    }
}
package com.mainapp;
import java.util.Scanner;
public class ObjectFactoryClass {

    public EmployeeBean customObject() {
        Scanner scanner = new Scanner(System.in);
        System.out.println("ENTER KEY");

        EmployeeBean employeeBean;
        int key=scanner.nextInt();
        if(key==1212) {
            employeeBean=new EmployeeBean("QWERTY");

```



```

    }
    else {
        employeeBean=new EmployeeBean("PQRS");
    }
    return employeeBean;
}
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util" xsi:schemaLocation="
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions here -->

    <bean id="fbean" class="com.mainapp.ObjectFactoryClass" ></bean>

    <bean
    id="emp"
    class="com.mainapp.EmployeeBean"
    factory-bean="fbean"
    factory-method="customObject"
    >
    </bean>

    <bean
        class="org.springframework.context.annotation.CommonAnnotationBeanPostProcessor"
    >
    </bean>

</beans>

```

# SPRING FRAMEWORK

## DAY 7:

## BeanPostProcessor(Interface)

- Bean Loading
- Bean Instantiation

```
postProcessBeforeInitialization(); //LOG
```

- init()  
    postProcessAfterInitialization(); //LOG
- Custom method
- destroy()

# SPRING FRAMEWORK

## DAY 8:

### Bean Inheritance

<bean> : 5 property

<bean>: child

<bean>: child

<bean>: child

# SPRING FRAMEWORK

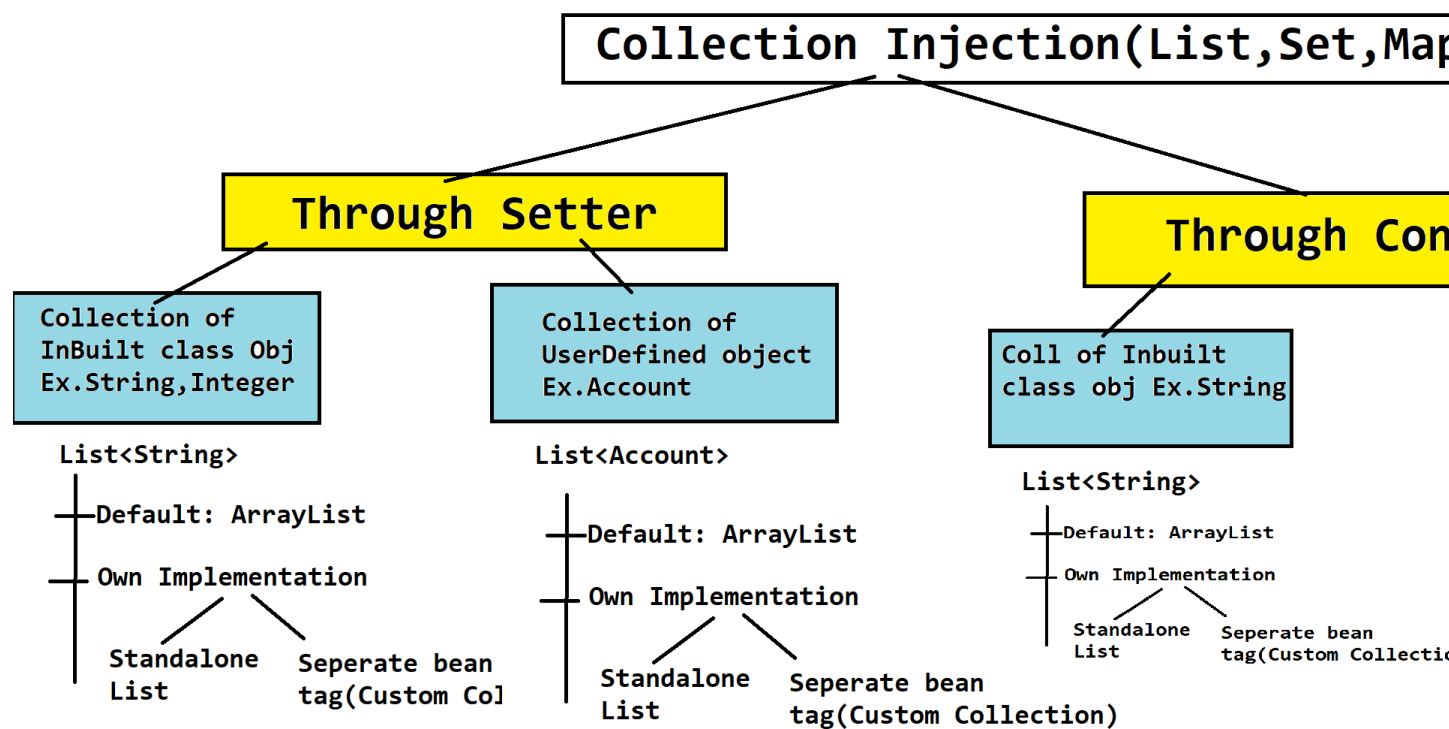
## DAY 9:

# Dependency Injection in XML

1.setter injection <property> Ex. Primitive,String(value)  
UserDefinedObj(ref)

2.constructor injection <constructor-arg> Ex.  
Primitive,String(value) UserDefinedObj(ref)

## Collection Injection



## List injection:

# 1. List of inbuilt class object Ex. List<String> listOfBooks; (Default Impl : ArrayList)

Through Setter:

```
package com.test;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");
        employee.setEid(11);
        employee.setEname("Raju");
        employee.setEaddress("add1");

        System.out.println(employee);
        System.out.println(employee.getDefaultListOfBooks().getClass());
    }
}
```

```
package com.test;
import java.util.List;

public class Employee {

    //USER
    private int eid;
    private String ename;
    private String eaddress;

    private List<String> defaultListOfBooks; //DEPENDENCY

    public Employee() {
        // TODO Auto-generated constructor stub
    }

    public int getEid() {
        return eid;
    }

    public void setEid(int eid) {
        this.eid = eid;
    }

    public String getEname() {
        return ename;
    }

    public void setName(String ename) {
        this.ename = ename;
    }

    public String getEaddress() {
        return eaddress;
    }

    public void setEaddress(String eaddress) {
        this.eaddress = eaddress;
    }

    public List<String> getDefaultListOfBooks() {
```

```

        return defaultListOfBooks;
    }

    public void setDefaultListOfBooks(List<String> defaultListOfBooks) {
        this.defaultListOfBooks = defaultListOfBooks;
    }

    @Override
    public String toString() {
        return "Employee [eid=" + eid + ", ename=" + ename + ", eaddress=" + eaddress + ",
defaultListOfBooks="
                + defaultListOfBooks + "];"
    }
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean definitions here -->

    <bean id="employee" class="com.test.Employee" >
        <property name="defaultListOfBooks">
            <list>
                <value>JAVA</value>
                <value>PYTHON</value>
                <value>AI</value>
            </list>
        </property>
    </bean>
</beans>

```

## Through Constructor:

employee.java

```

public Employee(List<String> defaultListOfBooks) {
    super();
    this.defaultListOfBooks = defaultListOfBooks;
}

```

bean.xml

```

<bean id="employee" class="com.test.Employee" >
    <constructor-arg name="defaultListOfBooks">
        <list>
            <value>JAVA</value>
            <value>PYTHON</value>
            <value>AI</value>
        </list>
    </constructor-arg>
</bean>

```

```
        </constructor-arg>
    </bean>
```

## Own Implementation

1. StandAlone List (for setter you can use <property> tag inside employee <bean>)

```
    <bean id="employee" class="com.test.Employee">
        <constructor-arg name="defaultListOfBooks" ref="list">
</constructor-arg>

    </bean>

    <!-- STANDALONE LIST -->
    <util:list id="list" list-class="java.util.LinkedList">
        <value>JAVA</value>
        <value>PYTHON</value>
        <value>AI</value>
    </util:list>
```

2. Separate Bean Tag (Custom Collection)

```
    <bean id="employee" class="com.test.Employee">
        <constructor-arg name="defaultListOfBooks" ref="list" >
</constructor-arg>
    </bean>

    <bean id="list" class="java.util.LinkedList">
        <constructor-arg>
            <list>
                <value>JAVA</value>
                <value>PYTHON</value>
                <value>PHP</value>
            </list>
        </constructor-arg>
    </bean>
```

Note:

```
//      ArrayList<String> arrayList = new ArrayList<String>();
//      arrayList.add("test1");
//      arrayList.add("test2");
```

```
//          arrayList.add("test3");
//
//          LinkedList<String> linkedList = new LinkedList<String>(arrayList);
//          System.out.println(linkedList);

//          LinkedList<String> linkedList = new
LinkedList<String>(Arrays.asList("TEST1","TEST2","TEST3"));
//          System.out.println(linkedList);
```

## 2.List of user defined class object Ex.

List<Account> listOfAccounts;

(Default Impl : ArrayList)

Through Setter:

```
package com.test;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Launch {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");
        employee.setEid(11);
        employee.setEname("Raju");
        employee.setEaddress("add1");

        System.out.println(employee);
        System.out.println(employee.getDefaultListOfAccounts().getClass());
    }
}
package com.test;

public class Account {

    //IOC
    private int an;
    private String aname;

    public Account() {
        // TODO Auto-generated constructor stub
    }

    public Account(int an, String aname) {
        super();
        this.an = an;
        this.aname = aname;
    }

    public int getAn() {
        return an;
    }

    public void setAn(int an) {
        this.an = an;
    }
}
```

```

        public String getAname() {
            return aname;
        }

        public void setName(String aname) {
            this.aname = aname;
        }

        @Override
        public String toString() {
            return "Account [an=" + an + ", aname=" + aname + "]";
        }
    }
}
package com.test;
import java.util.List;

public class Employee {

    //USER
    private int eid;
    private String ename;
    private String eaddress;

    //IOC
    private List<Account> defaultListOfAccounts; //DEPENDENCY

    public Employee() {
        // TODO Auto-generated constructor stub
    }

    public Employee(List<Account> defaultListOfAccounts) {
        super();
        this.defaultListOfAccounts = defaultListOfAccounts;
    }

    public int getEid() {
        return eid;
    }

    public void setEid(int eid) {
        this.eid = eid;
    }

    public String getEname() {
        return ename;
    }

    public void setName(String ename) {
        this.ename = ename;
    }

    public String getEaddress() {
        return eaddress;
    }

    public void setEaddress(String eaddress) {
        this.eaddress = eaddress;
    }

    public List<Account> getDefaultListOfAccounts() {
        return defaultListOfAccounts;
    }

    public void setDefaultListOfAccounts(List<Account> defaultListOfAccounts) {
        this.defaultListOfAccounts = defaultListOfAccounts;
    }

    @Override
    public String toString() {
        return "Employee [eid=" + eid + ", ename=" + ename + ", eaddress=" + eaddress + ",
defaultListOfAccounts="
                                + defaultListOfAccounts + "]";
    }
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"

```



```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:util="http://www.springframework.org/schema/util"
xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/util http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

<bean id="acc1" class="com.test.Account">
  <constructor-arg name="an" value="9090" ></constructor-arg>
  <constructor-arg name="aname" value="bank1" ></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
  <constructor-arg name="an" value="8080" ></constructor-arg>
  <constructor-arg name="aname" value="bank2" ></constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
  <property name="defaultListOfAccounts">
    <list>
      <ref bean="acc1" />
      <ref bean="acc2" />
    </list>
  </property>
</bean>

</beans>

```

## Through Constructor:

```

<bean id="acc1" class="com.test.Account">
  <constructor-arg name="an" value="9090" ></constructor-arg>
  <constructor-arg name="aname" value="bank1" ></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
  <constructor-arg name="an" value="8080" ></constructor-arg>
  <constructor-arg name="aname" value="bank2" ></constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
  <constructor-arg name="defaultListOfAccounts">
    <list>
      <ref bean="acc1" />
      <ref bean="acc2" />
    </list>
  </constructor-arg>
</bean>

```

## Own implementation:

1. StandAlone list: (for Setter you can use  
 <property> tag inside employee <bean> tag

```

<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090" ></constructor-arg>
    <constructor-arg name="aname" value="bank1"
></constructor-arg>
</bean>

    <bean id="acc2" class="com.test.Account">
        <constructor-arg name="an" value="8080" ></constructor-arg>
        <constructor-arg name="aname" value="bank2"
></constructor-arg>
</bean>

    <bean id="employee" class="com.test.Employee">
        <constructor-arg name="defaultListOfAccounts" ref="list"
></constructor-arg>
</bean>

    <util:list id="list" list-class="java.util.LinkedList" >
        <ref bean="acc1" />
        <ref bean="acc2" />
    </util:list>

```

## 2. Seperate Bean Tag (Custom Collection)

```

<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090" ></constructor-arg>
    <constructor-arg name="aname" value="bank1"
></constructor-arg>
</bean>

    <bean id="acc2" class="com.test.Account">
        <constructor-arg name="an" value="8080" ></constructor-arg>
        <constructor-arg name="aname" value="bank2"
></constructor-arg>
</bean>

    <bean id="employee" class="com.test.Employee">
        <constructor-arg name="defaultListOfAccounts" ref="list"
></constructor-arg>
</bean>

    <bean id="list" class="java.util.LinkedList" >
        <constructor-arg>

```

```
<list>
    <ref bean="acc1" />
    <ref bean="acc2" />
</list>
</constructor-arg>
</bean>
```

# SPRING FRAMEWORK

## DAY 10:

## Dependency Injection in XML

### Collection Injection

### Set Injection:

#### 1. Set of predefined class Object Ex. Set<String>

- By default it will inject **LinkedHashSet** (Order: Preserved) : setter/getter
- Own impl( Ex. HashSet )
  - a.StandAlone Set : setter/getter
  - b.Seperate bean tag : setter/getter

#### 2. Set of user defined class Object Ex. Set<Account>

- By default it will inject **LinkedHashSet** (Order: Preserved) : setter/getter
- Own impl( Ex. HashSet )

- a.StandAlone Set : setter/getter
- b.Seperate bean tag : setter/getter

## Program:

```
package com.test;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");

        employee.setId(18);
        employee.setName("raju");
        employee.setAddress("add1");

        System.out.println(employee);

        System.out.println(employee.getSetOfDefaultBooks().getClass());

    }
}
package com.test;
import java.util.Set;

public class Employee {

    //USER
    private int id;
    private String name;
    private String address;

    //IOC
    private Set<String> setOfDefaultBooks;

    public Employee() {
```

```

    }

    public Employee(Set<String> setOfDefaultBooks) {
        super();
        this.setOfDefaultBooks = setOfDefaultBooks;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public Set<String> getSetOfDefaultBooks() {
        return setOfDefaultBooks;
    }

    public void setSetOfDefaultBooks(Set<String> setOfDefaultBooks) {
        this.setOfDefaultBooks = setOfDefaultBooks;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ",
address=" + address + ", setOfDefaultBooks="
        + setOfDefaultBooks + "]\n";
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/util
           http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

    <bean id="employee" class="com.test.Employee">
        <property name="setOfDefaultBooks">
            <set>
                <value>JAVA</value>
                <value>JAVAABC</value>
                <value>ABCJAVA</value>
                <value>JAVABBB</value>
                <value>JAYYYVA</value>
                <value>JPPPAVA</value>
            </set>
        </property>
    </bean>
</beans>

```

## Through constructor:

```

<bean id="employee" class="com.test.Employee">
    <constructor-arg name="setOfDefaultBooks">
        <set>
            <value>JAVA</value>
            <value>JAVAABC</value>
            <value>ABCJAVA</value>
            <value>JAVABBB</value>
            <value>JAYYYVA</value>
            <value>JPPPAVA</value>
        </set>
    </constructor-arg>
</bean>

```

Own impl:

1. StandAlone:

Through constructor

```
<bean id="employee" class="com.test.Employee">
    <constructor-arg name="setOfDefaultBooks" ref="set">
    </constructor-arg>
</bean>

<util:set id="set" set-class="java.util.HashSet">
    <value>JAVA</value>
    <value>JAVAABC</value>
    <value>ABCJAVA</value>
    <value>JAVABBB</value>
    <value>JAYYYVA</value>
    <value>JPPPAVA</value>
</util:set>
```

Through Setter:

```
<bean id="employee" class="com.test.Employee">
    <property name="setOfDefaultBooks" ref="set">
    </property>
</bean>

<util:set id="set" set-class="java.util.HashSet">
    <value>JAVA</value>
    <value>JAVAABC</value>
    <value>ABCJAVA</value>
    <value>JAVABBB</value>
    <value>JAYYYVA</value>
    <value>JPPPAVA</value>
</util:set>
```

2. Separate Bean Tag(Custom Collection)

```

<bean id="employee" class="com.test.Employee">
    <property name="setOfDefaultBooks" ref="set">
    </property>
</bean>

<bean id="set" class="java.util.HashSet">
    <constructor-arg>
        <set>
            <value>JAVA</value>
            <value>JAVAABC</value>
            <value>ABCJAVA</value>
            <value>JAVABBB</value>
            <value>JAYYYVA</value>
            <value>JPPPAVA</value>
        </set>
    </constructor-arg>
</bean>

```

## Note:

```

<bean id="employee" class="com.test.Employee">
    <property name="setOfDefaultBooks" ref="set">
    </property>
</bean>

<bean id="set" class="java.util.HashSet">
    <constructor-arg>
        <list>
            <value>JAVA</value>
            <value>JAVAABC</value>
            <value>ABCJAVA</value>
            <value>JAVABBB</value>
            <value>JAYYYVA</value>
            <value>JPPPAVA</value>
        </list>
    </constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
    <property name="setOfDefaultBooks" ref="set">
    </property>

```



```

</bean>

<bean id="set" class="java.util.HashSet">
    <constructor-arg>
        <array>
            <value>JAVA</value>
            <value>JAVAABC</value>
            <value>ABCJAVA</value>
            <value>JAVABBB</value>
            <value>JAYYYVA</value>
            <value>JPPPAVA</value>
        </array>
    </constructor-arg>
</bean>

```

```

=====

=====

```

## Set of user defined class object

### 1.default impl(setter)

```

package com.test;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");

        employee.setId(18);
        employee.setName("raju");
        employee.setAddress("add1");

        System.out.println(employee);
        System.out.println(employee.getSetOfAccounts().getClass());
    }
}

```

```

    }
}
package com.test;
import java.util.Set;

public class Employee {

    //USER
    private int id;
    private String name;
    private String address;

    //IOC
    private Set<Account> setOfAccounts;

    public Employee() {

    }

    public Employee(Set<Account> setOfAccounts) {
        super();
        this.setOfAccounts = setOfAccounts;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }
}

```

```

    public void setAddress(String address) {
        this.address = address;
    }

    public Set<Account> getSetOfAccounts() {
        return setOfAccounts;
    }

    public void setSetOfAccounts(Set<Account> setOfAccounts) {
        this.setOfAccounts = setOfAccounts;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ",
address=" + address + ", setOfAccounts=" + setOfAccounts
        + "]";
    }
}
package com.test;

public class Account {

    //IOC
    private int an;
    private String aname;

    public Account() {
        // TODO Auto-generated constructor stub
    }

    public Account(int an, String aname) {
        super();
        this.an = an;
        this.aname = aname;
    }

    public int getAn() {
        return an;
    }

    public void setAn(int an) {
        this.an = an;
    }
}

```

```

    public String getAname() {
        return aname;
    }

    public void setAname(String aname) {
        this.aname = aname;
    }

    @Override
    public String toString() {
        return "Account [an=" + an + ", aname=" + aname + "];"
    }
}
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
        http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

    <bean id="acc1" class="com.test.Account">
        <constructor-arg name="an" value="9090"></constructor-arg>
        <constructor-arg name="aname"
value="bank1"></constructor-arg>
    </bean>

    <bean id="acc2" class="com.test.Account">
        <constructor-arg name="an" value="8080"></constructor-arg>
        <constructor-arg name="aname"
value="bank2"></constructor-arg>
    </bean>

    <bean id="employee" class="com.test.Employee">
        <property name="setOfAccounts">
            <set>
                <ref bean="acc1" />
                <ref bean="acc2" />
            </set>
        </property>
    </bean>
</beans>

```

## 2.default impl(constructor) : use constructor-arg tag with employee <bean>

```
<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
    <constructor-arg name="an" value="8080"></constructor-arg>
    <constructor-arg name="aname"
value="bank2"></constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
    <constructor-arg name="setOfAccounts">
        <set>
            <ref bean="acc1" />
            <ref bean="acc2" />
        </set>
    </constructor-arg>
</bean>
```

## MyOwn Impl( HashSet)

### 1.StandAlone Set (Through constructor)

```
<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
    <constructor-arg name="an" value="8080"></constructor-arg>
    <constructor-arg name="aname"
value="bank2"></constructor-arg>
```

```

</bean>

<bean id="employee" class="com.test.Employee">
    <constructor-arg name="setOfAccounts" ref="set">
    </constructor-arg>
</bean>

<util:set id="set" set-class="java.util.HashSet">
    <ref bean="acc1" />
    <ref bean="acc2" />
</util:set>

```

For setter use property tag with employee bean

## 2. Seperate Bean Tag (constructor)

```

<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
    <constructor-arg name="an" value="8080"></constructor-arg>
    <constructor-arg name="aname"
value="bank2"></constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
    <constructor-arg name="setOfAccounts" ref="set">
    </constructor-arg>
</bean>

<bean id="set" class="java.util.HashSet">
    <constructor-arg>
        <set>
            <ref bean="acc1" />
            <ref bean="acc2" />
        </set>
    </constructor-arg>

```

`</bean>`

For setter use property tag with employee bean

# MAP INJECTION:

## 1. Map of predefined class Object Ex. Map<String,String>

- By default it will inject **LinkedHashMap** (Order: Preserved) : setter/getter
- Own impl( Ex. HashMap )
  - a.StandAlone Map : setter/getter
  - b.Separate bean tag : setter/getter

## 2. Set of user defined class Object Ex. Map<String,Account>

- By default it will inject **LinkedHashMap** (Order: Preserved) : setter/getter
- Own impl( Ex. HashMap )
  - a.StandAlone Map : setter/getter
  - b.Separate bean tag : setter/getter

**Program:** Map of predefined class Object Ex.

Map<String,String>

## Default impl(Setter)

```
package com.test;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
```

```

public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");

        employee.setId(18);
        employee.setName("raju");
        employee.setAddress("add1");

        System.out.println(employee);
        System.out.println(employee.getMapOfBooks().getClass());
    }
}
package com.test;
import java.util.Map;
public class Employee {

    //USER
    private int id;
    private String name;
    private String address;

    //IOC
    private Map<String,String> mapOfBooks;

    public Employee() {

    }

    public Employee(Map<String, String> mapOfBooks) {
        super();
        this.mapOfBooks = mapOfBooks;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {

```



```

        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public Map<String, String> getMapOfBooks() {
        return mapOfBooks;
    }

    public void setMapOfBooks(Map<String, String> mapOfBooks) {
        this.mapOfBooks = mapOfBooks;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ",
address=" + address + ", mapOfBooks=" + mapOfBooks + "]\n";
    }
}

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:util="http://www.springframework.org/schema/util"
xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/util
    http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

    <bean id="employee" class="com.test.Employee">
        <property name="mapOfBooks">
            <map>

```

```

        <entry key="a1" value="java1" ></entry>
        <entry key="a2" value="java2" ></entry>
        <entry key="a3" value="java3" ></entry>
        <entry key="a4" value="java4" ></entry>
    </map>
</property>
</bean>

```

```
</beans>
```

Through constructor: use constructor-arg tag with employee bean

## Own impl:

### 1.StandAlone Map: (Setter)

```

<bean id="employee" class="com.test.Employee">
    <property name="mapOfBooks" ref="map">
    </property>
</bean>

<util:map id="map" map-class="java.util.HashMap" >
    <entry key="a1" value="java1"></entry>
    <entry key="a2" value="java2"></entry>
    <entry key="a3" value="java3"></entry>
    <entry key="a4" value="java4"></entry>
</util:map>

```

Through constructor: use constructor-arg tag with employee bean

### 2.Seperate Bean Tag: (Setter)

```

<bean id="employee" class="com.test.Employee">
    <property name="mapOfBooks" ref="map">
    </property>
</bean>

```

```

<bean id="map" class="java.util.HashMap">
    <constructor-arg>
        <map>
            <entry key="a1" value="java1"></entry>
            <entry key="a2" value="java2"></entry>
            <entry key="a3" value="java3"></entry>
            <entry key="a4" value="java4"></entry>
        </map>
    </constructor-arg>
</bean>

```

Through constructor: use constructor-arg tag with employee bean

## 2. Set of user defined class Object Ex.

### Map<String,Account>

#### 1.default (Setter)

Key: predefined : key

Key: userDefined : key-ref

Value: predefined : value

Value: userDefined: value-ref

```

package com.test;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
public class Launch {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext ac = new
ClassPathXmlApplicationContext("bean.xml");
        Employee employee= (Employee) ac.getBean("employee");
    }
}

```

```

        employee.setId(18);
        employee.setName("raju");
        employee.setAddress("add1");

        System.out.println(employee);
        System.out.println(employee.getMapOfAccounts().getClass());
    }
}
package com.test;
import java.util.Map;
public class Employee {

    //USER
    private int id;
    private String name;
    private String address;

    //IOC
    private Map<String,Account> mapOfAccounts;    //< key=preDefined ,
value=userDefined >

    public Employee() {

    }

    public Employee(Map<String, Account> mapOfAccounts) {
        super();
        this.mapOfAccounts = mapOfAccounts;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

```

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public Map<String, Account> getMapOfAccounts() {
        return mapOfAccounts;
    }

    public void setMapOfAccounts(Map<String, Account> mapOfAccounts)
{
        this.mapOfAccounts = mapOfAccounts;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ",
address=" + address + ", mapOfAccounts=" + mapOfAccounts
        + "]";
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

```

```

    <bean id="acc1" class="com.test.Account">
        <constructor-arg name="an" value="9090"></constructor-arg>
        <constructor-arg name="aname"
value="bank1"></constructor-arg>
    </bean>

```

```

    <bean id="acc2" class="com.test.Account">
        <constructor-arg name="an" value="8080"></constructor-arg>

```

```

        <constructor-arg name="aname"
value="bank2"></constructor-arg>
    </bean>

    <bean id="employee" class="com.test.Employee">
        <property name="mapOfAccounts">
            <map>
                <entry key="a1" value-ref="acc1"></entry>
                <entry key="a2" value-ref="acc2"></entry>
            </map>
        </property>
    </bean>
</beans>

```

Through constructor: use constructor-arg tag with employee bean

## Own Impl:

### 1.StandAlone(Setter)

```

<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

    <bean id="acc2" class="com.test.Account">
        <constructor-arg name="an" value="8080"></constructor-arg>
        <constructor-arg name="aname"
value="bank2"></constructor-arg>
    </bean>

    <bean id="employee" class="com.test.Employee">
        <property name="mapOfAccounts" ref="map">
            </property>
    </bean>

```

```

<util:map id="map" map-class="java.util.HashMap">
    <entry key="a1" value-ref="acc1"></entry>
    <entry key="a2" value-ref="acc2"></entry>
</util:map>

```

Through constructor: use constructor-arg tag with employee bean

## 2. Seperate Bean Tag(Custom Collection)

```

<bean id="acc1" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

<bean id="acc2" class="com.test.Account">
    <constructor-arg name="an" value="8080"></constructor-arg>
    <constructor-arg name="aname"
value="bank2"></constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee">
    <property name="mapOfAccounts" ref="map">
    </property>
</bean>

<bean id="map" class="java.util.HashMap">
    <constructor-arg>
        <map>
            <entry key="a1" value-ref="acc1"></entry>
            <entry key="a2" value-ref="acc2"></entry>
        </map>
    </constructor-arg>
</bean>

```

Through constructor: use constructor-arg tag with employee bean

Exceptional Case:

Key: Integer (**LinkedHashMap** only, **map-class** wont work)

=====

=====

# **SPRING FRAMEWORK**

## **DAY 11:**

### **Dependency Injection in XML**

### **Autowiring through XML**

->It is a way of dependency injection in which dependency will inject automatically

Modes:

1.No (Manually injection)

2.byName (Setter)

3.byType (Setter)

4.constructor ( Constructor )



Not Suitable for Complex Configuration

Not Readable

## bean.xml

```
<bean id="eid" class="com.test.Employee"  
autowire="byName">
```

```
</bean>
```

```
<bean id="account" >
```

```
<util:list id="listofbooks" class="linkedlist">
```

```
<util:list id="listofbooks2" class="vector">
```

## byname:

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:util="http://www.springframework.org/schema/util"  
  xsi:schemaLocation="  
    http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd  
    http://www.springframework.org/schema/util  
http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean  
definitions here -->
```

```
  <bean id="account" class="com.test.Account">  
    <constructor-arg name="an" value="9090"></constructor-arg>  
    <constructor-arg name="aname"  
value="bank1"></constructor-arg>
```

```

    </bean>

    <bean id="adhaar" class="com.test.Adhaar">
        <constructor-arg name="adhaarNo"
value="11119090"></constructor-arg>
        <constructor-arg name="adhaarName"
value="raju"></constructor-arg>
    </bean>

    <bean id="listOfBooks" class="java.util.LinkedList" >
        <constructor-arg>
            <list>
                <value>JAVA1</value>
                <value>JAVA2</value>
                <value>JAVA3</value>
            </list>
        </constructor-arg>
    </bean>

    <bean id="employee" class="com.test.Employee" autowire="byName">

</bean>

</beans>

```

In case of **byname** if there is two bean available and we are looking for a single bean then in such case we can disqualify a bean by **autowire-candidate="false"** , still it will be available for **manual injection and autowire byname injection**

```

<bean id="bean1a" class="com.test.Account">
    <constructor-arg name="an" value="9090"></constructor-arg>
    <constructor-arg name="aname"
value="bank1"></constructor-arg>
</bean>

    <bean id="bean1b" class="com.test.Account">
        <constructor-arg name="an" value="90902"></constructor-arg>
        <constructor-arg name="aname"
value="bank2"></constructor-arg>
</bean>

        <bean id="bean1c" class="com.test.Account">
            <constructor-arg name="an" value="90903"></constructor-arg>
            <constructor-arg name="aname"
value="bank3"></constructor-arg>
</bean>

            <bean id="bean1d" class="com.test.Account">
                <constructor-arg name="an" value="90904"></constructor-arg>
                <constructor-arg name="aname"
value="bank4"></constructor-arg>
</bean>

                <bean id="bean1e" class="com.test.Account">
                    <constructor-arg name="an" value="90905"></constructor-arg>
                    <constructor-arg name="aname"
value="bank5"></constructor-arg>
</bean>

                    <bean id="bean1f" class="com.test.Account" primary="true">
                        <constructor-arg name="an" value="90906"></constructor-arg>
                        <constructor-arg name="aname"
value="bank6"></constructor-arg>
</bean>

                        <bean id="bean2" class="com.test.Adhaar">
                            <constructor-arg name="adhaarNo"
value="11119090"></constructor-arg>
                            <constructor-arg name="adhaarName"
value="raju"></constructor-arg>
</bean>

                            <bean id="listbean1" class="java.util.LinkedList" >
                                <constructor-arg>
                                    <list>

```

```

        <value>JAVA1</value>
        <value>JAVA2</value>
        <value>JAVA3</value>
    </list>
</constructor-arg>
</bean>

<bean id="employee" class="com.test.Employee" autowire="byType">
<!--Account , Adhaar , List -->
</bean>

```

Mode3: constructor (ByType->ByName) :  
Through constructor

Exception: Constructor Common: 1fail: All fail

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:util="http://www.springframework.org/schema/util"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/util
        http://www.springframework.org/schema/util/spring-util.xsd"> <!-- bean
definitions here -->

    <bean id="bean1a" class="com.test.Account">
        <property name="an" value="90901"></property>
        <property name="aname" value="bank1"></property>
    </bean>

    <bean id="bean1b" class="com.test.Account">
        <property name="an" value="90902"></property>
        <property name="aname" value="bank2"></property>
    </bean>

```

```

<bean id="bean1c" class="com.test.Account">
    <property name="an" value="90903"></property>
    <property name="aname" value="bank3"></property>
</bean>

<bean id="bean1d" class="com.test.Account">
    <property name="an" value="90904"></property>
    <property name="aname" value="bank4"></property>
</bean>

<bean id="bean1e" class="com.test.Account">
    <property name="an" value="90905"></property>
    <property name="aname" value="bank5"></property>
</bean>

<bean id="account" class="com.test.Account">
    <property name="an" value="90906"></property>
    <property name="aname" value="bank6"></property>
</bean>

<bean id="adhaar" class="com.test.Adhaar">
    <property name="adhaarNo" value="11119090"></property>
    <property name="adhaarName" value="raju"></property>
</bean>

<bean id="list1" class="java.util.LinkedList" >
    <constructor-arg>
        <list>
            <value>JAVA1</value>
            <value>JAVA2</value>
            <value>JAVA3</value>
        </list>
    </constructor-arg>
</bean>

<bean id="listOfBooks" class="java.util.LinkedList" >
    <constructor-arg>
        <list>
            <value>JAVA1</value>
            <value>JAVA2</value>
            <value>JAVA3</value>
        </list>
    </constructor-arg>
</bean>

```

```
<bean id="employee" class="com.test.Employee"
autowire="constructor"> <!--Account , Adhaar , List -->
</bean>
</beans>
```

# SPRING FRAMEWORK

## DAY 12:

Till now we have seen XML approach:

1. In enterprise application, separating configuration from code using xml can make it easier to manage dependencies and it can put your configuration at one place.
2. If you want to keep your spring configuration outside the source code (better separation concern)
3. If you are working with legacy application then it is better to maintain legacy

## THEN WHY SHOULD WE LEARN PURE JAVA APPROACH (ANNOTATION)

1.It is preferred in many **modern application** due to its simplicity, readability, easy to use.

```
@Service
public class Service{
    @Autowired
    private Dao dao; //Setter
}
=====
<bean id="dao" class="com.dao.Dao" ></bean>
<bean id="service" class="com.service.Service" >
    <property name="dao" ref="dao" >
</bean>
=====
```

## BEST PRACTICE?

- Always try to go with hybrid Approach ( XML+ANNOTATION)
- Use Annotation for bean definitions and dependency injection
- Use XML for environment specific configuration

=====

=====

## With Annotation we will learn:

- 1.configuration
- 2.How to create an Object( Simple & Custom logic)
- 3.Lazy and Eager
- 4.Scope
- 5.Bean Life Cycle
- 6.Bean Post Processor
- 7.Dependency Injection & SpEL

```
package com.test;
import
org.springframework.context.annotation.AnnotationConfigApplicationCont
ext;
import com.config.MyConfig;

public class Launch {

    public static void main(String[] args) {

        AnnotationConfigApplicationContext ac=new
AnnotationConfigApplicationContext(MyConfig.class);
        Employee bean1 = ac.getBean(Employee.class);
        bean1.test();
        ac.close();

//        Employee bean2 = ac.getBean(Employee.class);
//        bean2.test();
    }
}
```



```
package com.test;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import org.springframework.context.annotation.Lazy;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;
```

```
//@Component
//@Lazy
//@Scope(scopeName = "prototype")
//@Scope("prototype")
public class Employee {

    static {
        System.out.println("BEAN LOADING");
    }

    public Employee() {
        System.out.println("BEAN INSTANTIATION");
    }
    @PostConstruct
    public void init() {
        System.out.println("BEAN INIT");
    }
    @PreDestroy
    public void destroy() {
        System.out.println("BEAN DESTROY");
    }

    public void test() {
        System.out.println("METHOD CALLED");
    }
}
```

```
package com.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

import com.test.Employee;
```

```

@Configuration
@ComponentScan(basePackages = {"com.test","com.postprocessor"})
public class MyConfig {

    @Bean
    public Employee employeeBean() {

        System.out.println("CUSTOM LOGIC");
        return new Employee();

    }

}

//@ComponentScan(basePackages = {"com.test","com.xyz"})
package com.postprocessor;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.stereotype.Component;
@Component
public class BeanPostProcessorImpl implements BeanPostProcessor {
    @Override
    public Object postProcessBeforeInitialization(Object bean, String
beanName) throws BeansException {
        // TODO Auto-generated method stub
        System.out.println("before initialization");
        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(Object bean, String
beanName) throws BeansException {
        // TODO Auto-generated method stub
        System.out.println("after initialization");
        return bean;
    }

}

```

# SPRING FRAMEWORK

## DAY 13:

Dependency Injection with Annotation ( Pure Java ): **Autowiring ( ByType ) + SPEL**

### 1.Setter Injection

### 2.Constructor Injection

### 3.Field Injection

```
package com.mainapp;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.config.MyConfig;
import com.controller.MyController;
import com.entity.Employee;

public class Launch {

    public static void main(String[] args) {

        AnnotationConfigApplicationContext ac = new
AnnotationConfigApplicationContext(MyConfig.class);
        Employee empBean = ac.getBean(Employee.class);
        empBean.setEid(11);
        empBean.setEname("Raju");
        empBean.setEaddress("Add1");
        empBean.setEsalary(1000);

        MyController controllerBean = ac.getBean(MyController.class);
        String res = controllerBean.doPost(empBean);
        System.out.println(res);

    }
}

package com.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
```

```
import com.entity.Employee;
import com.service.MyService;
```

```
@Component
```

```
public class MyController {

    @Autowired //FIELD INJECTION
    private MyService myService;

    public String doPost(Employee empBean) {

        return myService.add(empBean);

    }

}
```

```
package com.service;
import com.entity.Employee;
```

```
public interface MyService {

    String add(Employee empBean);

}
```

```
package com.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;
import com.entity.Employee;
import com.repository.MyRepository;
```

```
@Component
```

```
public class MyServiceImpl implements MyService {
```

```
    @Autowired
    @Qualifier("repo1") //FIELD INJECTION
    private MyRepository myRepository;
```

```
//    @Autowired
//    @Qualifier("repo2") //SETTER INJECTION
//    public void setMyRepository(MyRepository myRepository) {
//        System.out.println("*****");
//        this.myRepository = myRepository;
//    }
```

```
//    @Autowired //CONSTRUCTOR INJECTION
//    public MyServiceImpl(@Qualifier("repo2") MyRepository myRepository) {
//        super();
//        System.out.println("*****");
//        this.myRepository = myRepository;
//    }
```

```
@Override
public String add(Employee empBean) {
```

```

        return myRepository.add(empBean);
    }
}

```

```

package com.repository;
import com.entity.Employee;
public interface MyRepository {

    String add(Employee empBean);

}

```

```

package com.repository;
import java.util.List;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import com.entity.Employee;

```

```

@Component("repo1")
public class MyRepositoryImpl1 implements MyRepository {

    //    @Value("1000")
    @Value("#{2+1000}")    //SPEL    #{ }
    private int testid;

    @Value("ABCD")
    private String testString;

    //    @Value("#{list}")
    @Value("#{list[0]}")
    private List<String> testList;

    @Override
    public String add(Employee empBean) {

        System.out.println(empBean+" saved successfully "+testid+" | "+testString+" | "+testList);
        return "SUCCESS1";
    }
}

```

```

package com.repository;
import org.springframework.stereotype.Component;
import com.entity.Employee;
@Component("repo2")
public class MyRepositoryImpl2 implements MyRepository {

    @Override
    public String add(Employee empBean) {

        System.out.println(empBean+" saved successfully");
        return "SUCCESS2";
    }
}

```

```
}
```

```
package com.entity;
import org.springframework.stereotype.Component;
@Component
public class Employee {

    private int eid;
    private String ename;
    private String eaddress;
    private int esalary;

    public Employee() {
        // TODO Auto-generated constructor stub
    }

    public int getId() {
        return eid;
    }

    public void setId(int eid) {
        this.eid = eid;
    }

    public String getName() {
        return ename;
    }

    public void setName(String ename) {
        this.ename = ename;
    }

    public String getAddress() {
        return eaddress;
    }

    public void setAddress(String eaddress) {
        this.eaddress = eaddress;
    }

    public int getSalary() {
        return esalary;
    }

    public void setSalary(int esalary) {
        this.esalary = esalary;
    }

    @Override
    public String toString() {
        return "Employee [eid=" + eid + ", ename=" + ename + ", eaddress=" +
eaddress + ", esalary=" + esalary + "]";
    }
}
```

```
package com.config;
import java.util.Arrays;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages =
{"com.entity","com.controller","com.service","com.repository"})
public class MyConfig {

    @Bean
    public List<String> list(){
        return Arrays.asList("JAVA1","JAVA2");
    }
}
```

# SPRING FRAMEWORK

## DAY 14:

# Spring JDBC (Database Interaction)

- Mostly Used : Hibernate(JPA implementation)

Spring JDBC sits between JDBC and Hibernate providing a balance control and convenience

## Why Not Just JDBC?

1. Boilerplate code
2. Manual work
3. Manual Exp Handling
4. Difficult transaction management
5. ResultSet Processing

Adv:

- > Fast Performance when it comes to bulk operation
- > You can optimize SQL statement ( More Control )

## Why Not Just Hibernate?

1. Caching mechanism (Performance overhead)
2. Not ideal for batch processing
3. Less control over SQL statement

Adv:

- > auto create, version, polymorphic, embedded, caching
- > Enterprise Application (Complex App)
- > Database independency ( HQL )



USE CASE: ORM benefits (Hibernate)

USE CASE: Speed/Performance

Spring JDBC: it reduces lot of lines of code and still your application perform almost similar as JDBC

JDBC:

- Connection Get
- SQL
- PreparedStatement
- ResultSet
- Pojo Object
- ArrayList add
- Return list

Spring JDBC

- return queryForList(SQL)
-

## CRUD OPERATION:

To interact with DB SpringJDBC provides  
JdbcTemplate

1.xml 2.annotation

To work with Spring JDBC you will need two Jars

- 1.Spring JDBC
  - 2.Spring TX
- 
- ReadAll: List<Map> : byDefault
  - ReadAll: List<Pojo> : Mapping
  - ReadSingle: Pojo : Mapping

# **S**PRING **F**RAMEWORK **DAY 15:**

## **Spring MVC (Model View Controller)**

What is MVC?

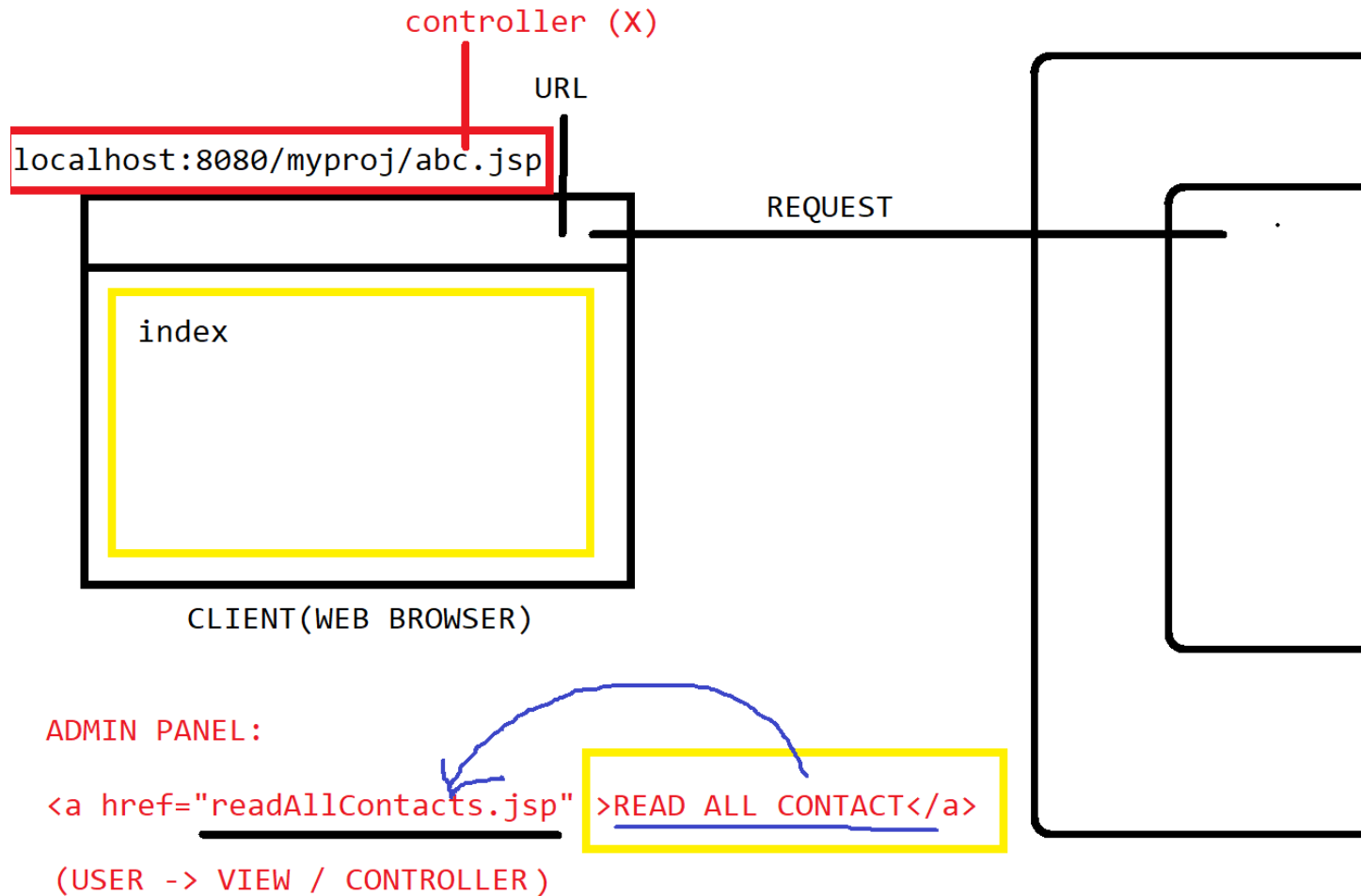
- >MVC stands for Model View Controller
- >It is a design Pattern or architecture

->It is used to separate an application into three interconnected component, this separation helps to manage the complexity of a large application

MODEL : It holds application Data

VIEW: Through which user interacts

CONTROLLER: who interacts with UI



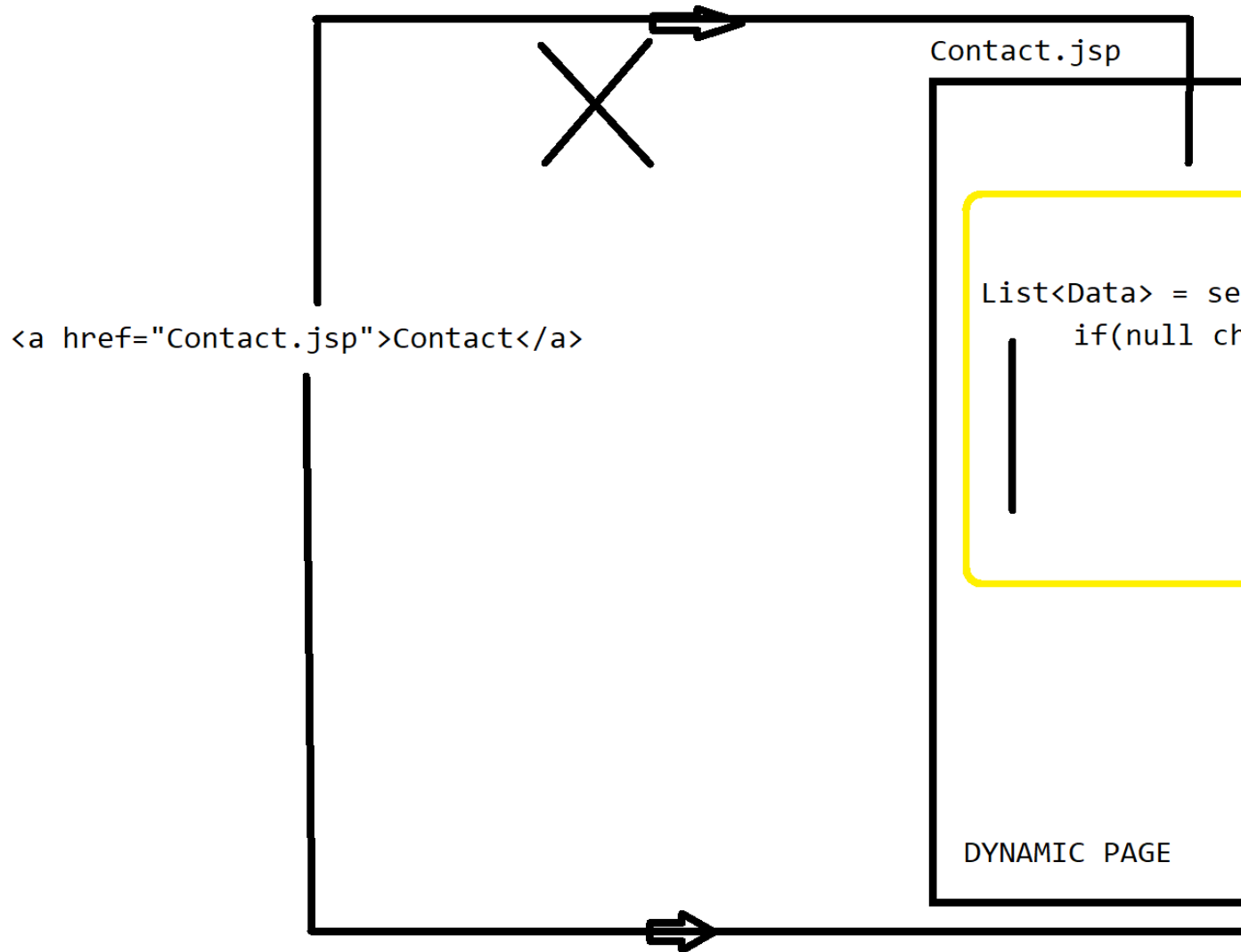
## 1.MVC architecture Flow:

- USER SENDS A REQUEST (BUTTON, `<a>`, URL )
- CONTROLLER RECEIVES THE REQUEST AND INTERACT WITH MODEL
- SEND data to Service Layer ( Business Logic)
- Controller send the response to the view.

RULE: IN MVC you cannot call the view directly,  
the view must be rendered through controller

REQUEST----->Controller----->DynamicPage(JSP  
)

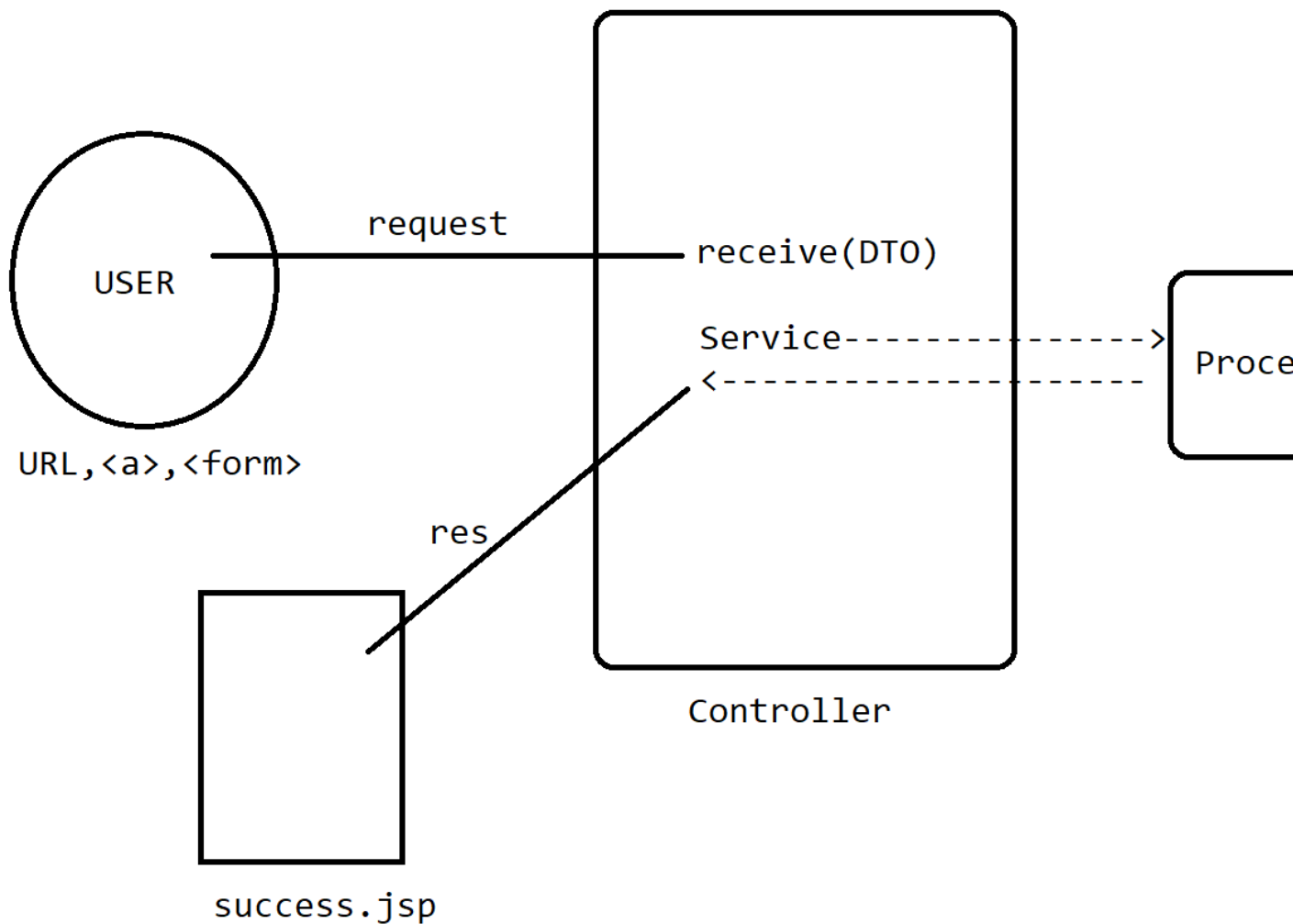
Note: you can access static page directly



What is Spring MVC?

->Spring MVC is module of spring fr that follows MVC design pattern to build a web application.

->BENEFITS: To simplify web application development



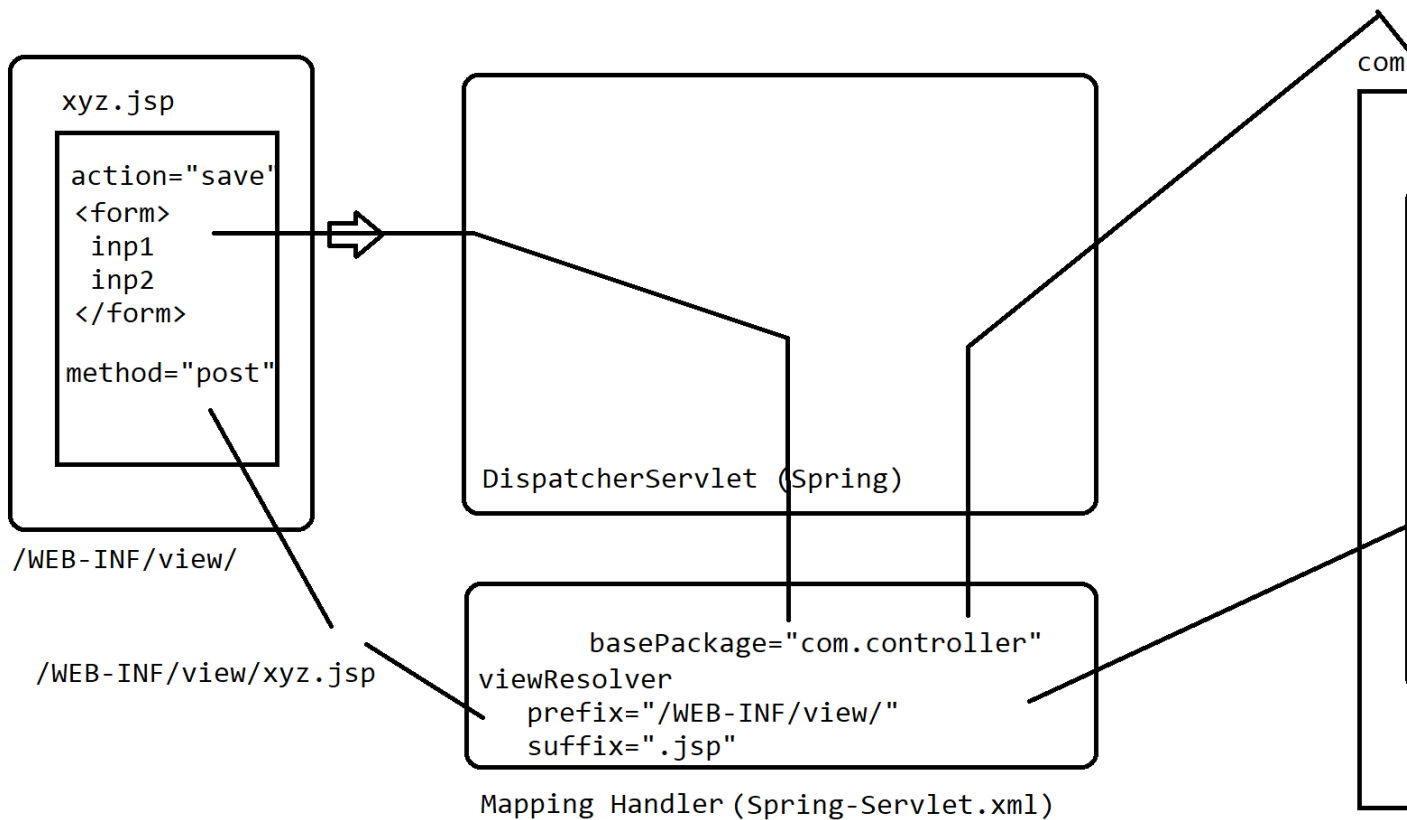
How spring MVC works internally:

1. `DispatcherServlet` ( FRONT CONTROLLER ) : it acts as a front controller and it can handle all requests
2. `Controller` ( `@Controller` )
3. `Model`



## 4.ViewResolver

## 5.Mapping Handler



## SUMMURY:

### 1.MVC ARCH

### 2.SEPERATION, SIMPLYFY

### 3.YOU MUST NOT CALL DYNAMIC PAGE DIRECTLY

4.REQUEST→CONTROLLER→DYNAMICPAGE(VIEW  
)

5.REQUEST->FRONT CONTROLLER(DISPATCHER  
SERVLET)->YOUR CONTROLLER

# **SPRING FRAMEWORK**

## **DAY 16:**

### **Spring MVC (Model View Controller)**

# Implementation of Spring MVC

1.Spring web

2.Spring webmvc

STATIC PAGE: directly call

DYNAMIC PAGE: user->controller->dynamic

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:mvc="http://www.springframework.org/schema/mvc"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/mvc/spring-mvc.xsd">
```

href="/save" : localhost:8080/save

href="save" : localhost:projName/save

## SUMMURY:

Where to place your dynamic page: ans ->  
inside WEB-INF

How to call a controller using <a>

- href="/save" : localhost:8080/save
- href="/save" : localhost:projName/save

How to give multiple path to a method  
(path={"", ""})

How to handle multiple controllers (  
@RequestMapping)

# **SPRING**

# **FRAMEWORK**

## **DAY 17:**

### **Spring MVC (Model View Controller)**

#### **How to send data to the Controller**

1. `HttpServletRequest` ( RARE CASE)

2. `@ModelAttribute` (MOSTLY)

### 3.@RequestParam( SPECIFIC CASE)

MODEL: which can hold application data  
(DTO,Entity,Model(Inbuilt))

```
package com.controller;
import javax.servlet.http.HttpServletRequest;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import com.dto.EmployeeDTO;

@Controller
public class MyController {

    @GetMapping(path={"insert"})
    public String insertView() {

        return "insert";

    }

    // @PostMapping(path={"insert"})
    // public String insertData(HttpServletRequest request) {
    //
    //     String id = request.getParameter("id");
    //     String name = request.getParameter("name");
    //
    //     System.out.println(id+"-"+name);
    //
    //     return "insert";
    //
    // }

    // @PostMapping(path={"insert"})
    // public String insertData( @ModelAttribute EmployeeDTO
    employeeDTO) {
    //
    //
    //     System.out.println(employeeDTO.getId()+"-"+employeeDTO.getName());
    // }
```

```
//
//      return "insert";
//
//  }

@PostMapping(path={"insert"})
public String insertData( @RequestParam("id") int id) {

    System.out.println(id);
    return "insert";

}
}
```

## HOW TO GET RID OF RESUBMISSION PROBLEM

You can use **POST** REDIRECT GET pattern

POST----->controller(sop)----->Redirect----->  
 Brower----->NewRequest: /insert (GET)

Model: when we don't use redirect

# **S**PRING

# **F**RAMEWOR**K**

## **DAY 18:**

### **SpringBoot**

- >It is a tool or Project developed to simplify Java Development
- >It is built on the top of Spring Framework
- >It is an extension of Spring framework
- >It was developed by Pivotal Software (Aquired by VMware) and released in APRIL 2014



1.It supports Convention over Configuration Approach ( LESS CODE )

2.It is based on Spring Framework (CORE MVC Benefits : LESS CODE)

3.You need not to write any configuration( NO CONFIGURATION XML FILE )

- URL **USER PASS** DRIVER UPDATE SHOW\_SQL  
(application.properties OR application.yml)

4.SpringBoot can able to download and place JARS automatically in your system and it will only configure jar inside your project ( INTERNALLY SPRING USES MAVEN TOOL )

5.Springboot offers embedded servers (Tomcat Server : 8080)

6.It is also called Framework of Framework

SUMUP: SPRINGBOOT SUPPORTS RAPID APPLICATION DEVELOPMENT ( RAD )

**PRE REQUISITES:**

- 1.CORE & ADV
- 2.HIBERNATE
- 3.SPRING CORE & MVC
- 4.MAVEN

## MAVEN TOOL

- >It is an **Automation and Management Tool** which is used to manage Dependencies (Jars)
- >You can work with maven with simple java project
- >We can interact with maven through POM.xml ( Project Object Model )
- >Maven first check inside local Repository and if jar is not available then it will download

```
<dependencies>
```

```
<!--
```

```
https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
```

```
<dependency>
```

```
<groupId>com.mysql</groupId>  
  
<artifactId>mysql-connector-j</artifactId>  
  
<version>8.3.0</version>  
  
                                </dependency>  
                                </dependencies>
```

MAVEN download jars from REMOTE REPOSITORY (mvnrepository.com) and place your jars inside LOCAL REPOSITORY (C:\Users\DELL\.m2\repository) and then jars will be configured inside your project

- SpringBoot : 2.7.X ( 18 )
- Java version: 8

# How to create Spring boot Project

1.Eclipse ( Plugins )

2.STS ( Spring tool suite )

3.Spring Initializr ( web )

- Download STS: <https://spring.io/tools>
- Extract->shortcut create
- Select a workspace->Launch

# SPRING

# FRAMEWORK

# DAY 19:

# SpringBoot

- 1.Springboot Introduction
- 2.How to SETUP ( STS & Eclipse )
- 3.How to create Springboot project
- 4.How to change VERSION of Springboot and Java
- 5.How to change default Server ( TOMCAT )
- 6.How to Interact with Database

Note: If you don't need to implement client server architecture then you need not to add **spring web** dependency.

- If I use **Spring Web** then it will automatically configure tomcat server in our project?
- But we can change server as per our use

|                        |                  |
|------------------------|------------------|
| Suitable FOR           | Lightweight      |
| PERFORMANCE            | EMBEDDED SUPPORT |
| TOMCAT(APACHE)         | Heavy            |
| Monolithic application | SPRINGBOOT       |
| GOOD                   |                  |
| JETTY(ECLIPSE)         | lightweight      |
| MICROSERVICES          | MANUALLY         |
| GOOD                   |                  |

# How to Interact with Database

## 1.JPA ( Java Persistence API )

It is a specification : it provides set of  
interfaces related to the ORM operations

## 2.Hibernate

- Hibernate provides implementation of **JPA** to interact with databases: ( **PORTABILITY** )
- Hibernate have own NATIVE API to interact with databases (MORE POWERFUL THAN HBN-JPA-IMPL)
- @id : javax.persistence / org.hibernate

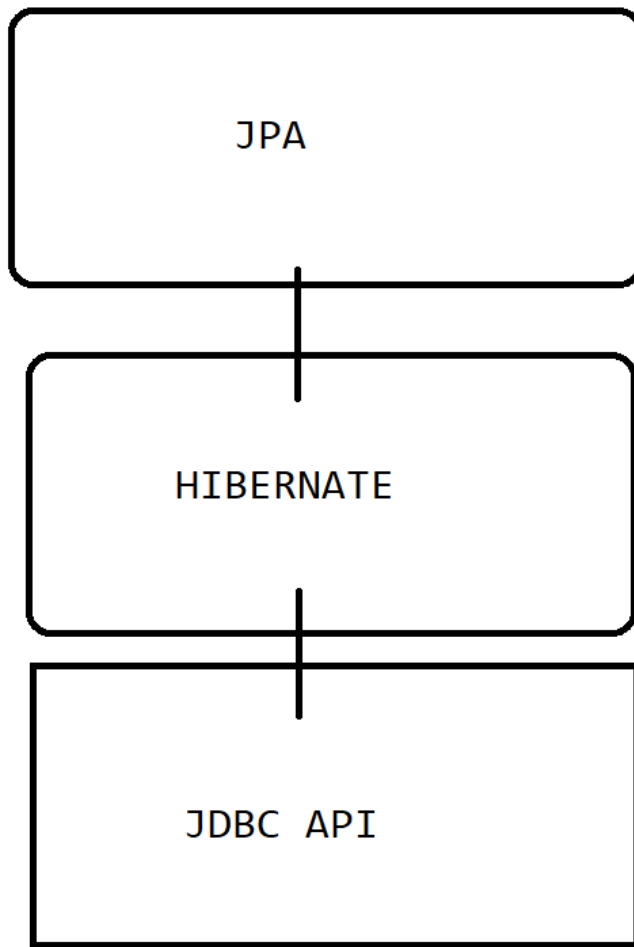
## COMMON JPA IMPLEMENTATIONS

1.Hibernate

2.EclipseLINK

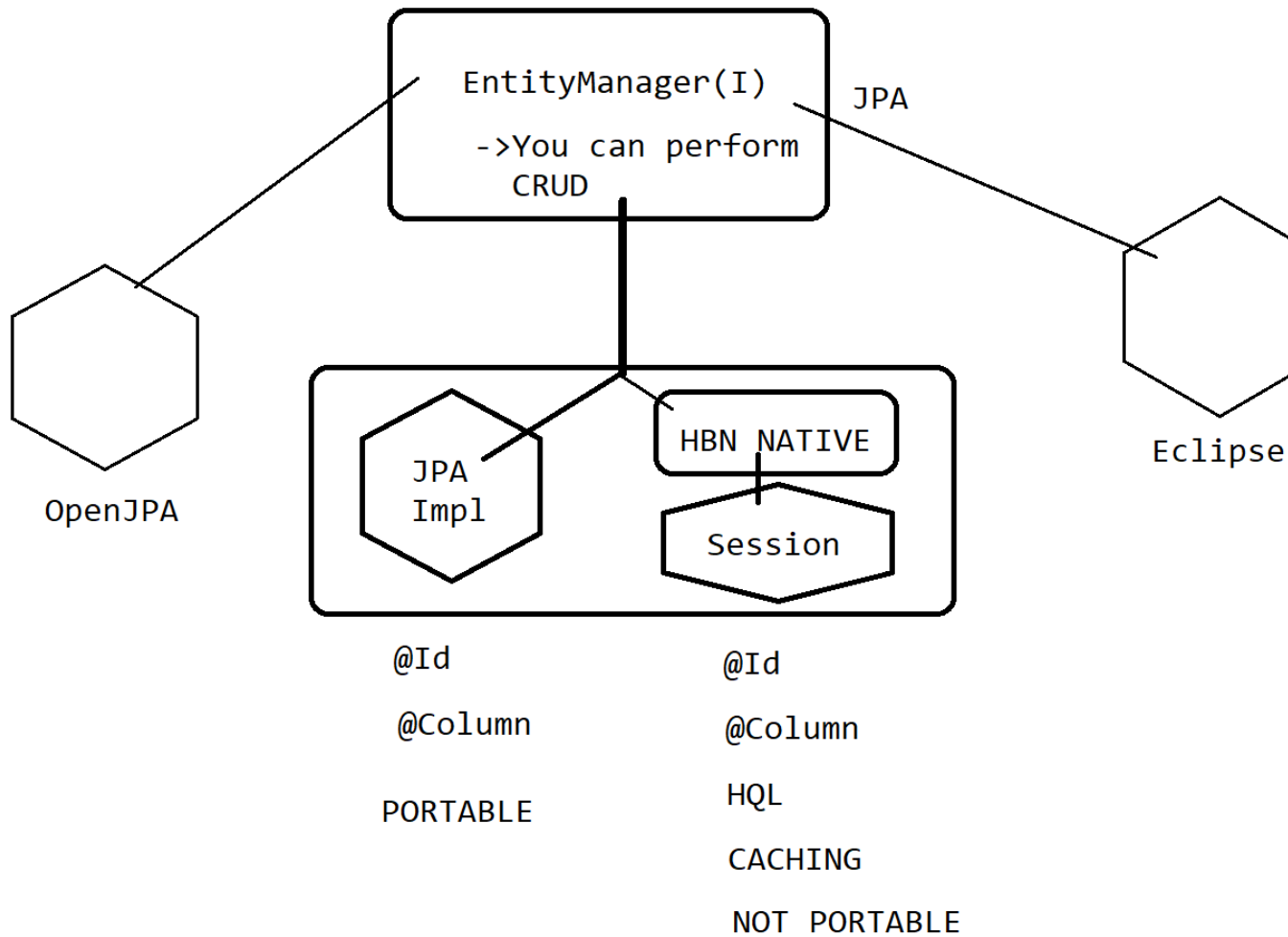
3.OPENJPA

3.DATANucleus



When you are working with Springboot, you might use hibernate or Eclipse Link or any other ways to interact with database that is why Spring boot internally **uses JPA as a specification** and it uses **hibernate as an implementation**





While working with springboot, you can use JPA and Hibernate as its implementation but for that you need to do many thing like manually autowiring, creating methods and all.

To simplify JPA work springboot created **SPRING DATA JPA** (A PART OF SPING DATA FAMILY) on the top of JPA:

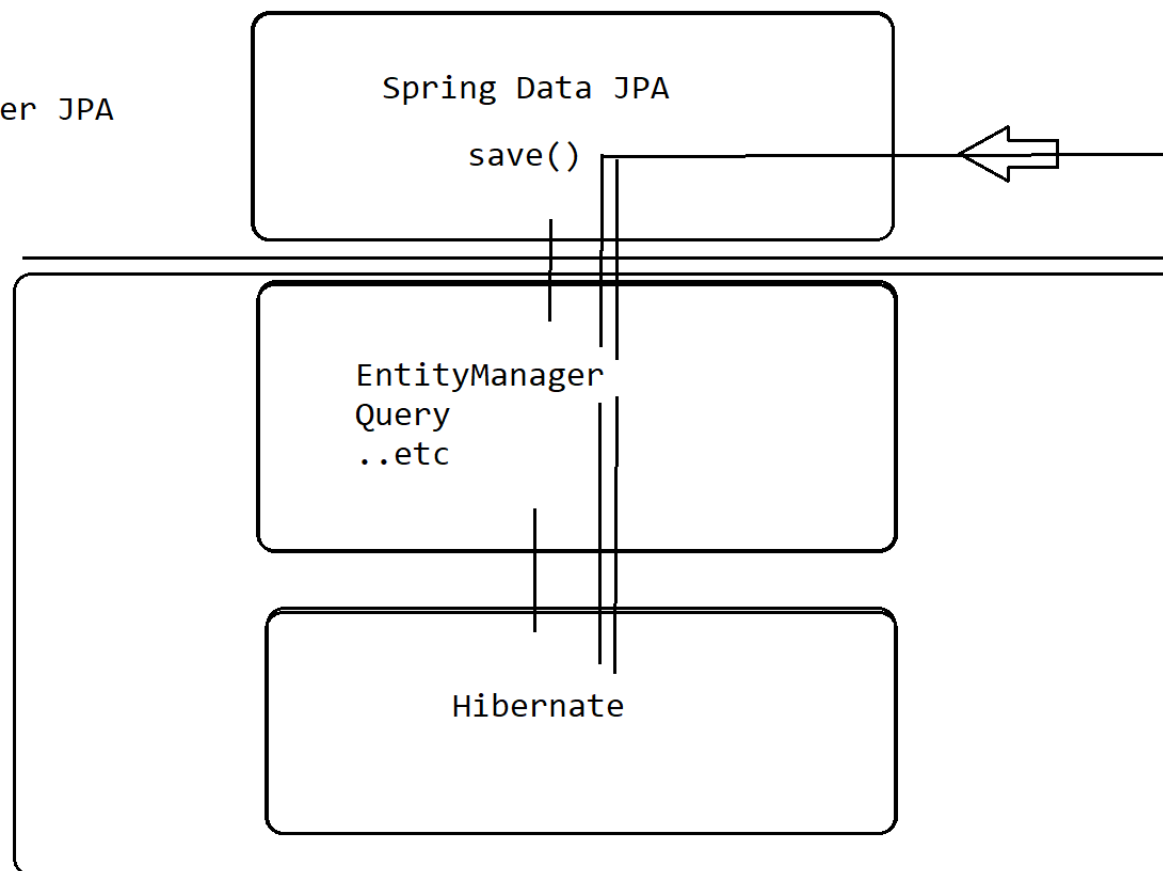
1.CrudRepositoy(I) basic features :

JPA->Hibernate

2.JpaRepository(I) adv features : :

JPA->Hibernate

IT provides  
abstraction over JPA



**NOTE: Your packages must be inside the base package**

# **S**PRING

# **F**RAMEWOR**K**

## **DAY 20:**

### **SpringBoot**

#### **SPRING DATA JPA**

->It is a part of Spring Data Family designed to simplify database interaction by reducing boilerplate code. It provides abstraction over JPA and integrates seamlessly with Hibernate, EclipseLink and other JPA providers.

Spring Data JPA provides two main interfaces

1. CrudRepository<T, ID>

1. JpaRepository<T, ID> (extends  
CrudRepository)

### **CrudRepository:**

- It provides basic CRUD operation
- Suitable for simple database operation

### **JpaRepository:**

- It provides advanced Database operations like Pagination, Batch processing, sorting..etc
- Suitable for simple database operation

Spring DATA JPA automatically implements method following a **specific naming convention**

Methods like **findByFieldNameContaining** are called **Derived Query Method** and springboot will automatically implement it so you need not to write your custom query

- **findByFieldName**
- **findByFieldNameContaining**
- **findByFieldNameStartingWith**
- **findByFieldNameEndingWith**
- **findByFieldNameGreaterThan**

Here's a list of the possible derived query methods you can use in Spring Data JPA following the naming conventions for various CRUD operations, including delete and read functionalities:

### Read/Select Operations

1. **findBy[Property]**  
Example: `List<Employee> findByName(String name);`  
(Find by exact match)
2. **findBy[Property]Containing**  
Example: `List<Employee> findByNameContaining(String name);`  
(Find by a substring match)
3. **findBy[Property]StartingWith**  
Example: `List<Employee> findByNameStartingWith(String prefix);`  
(Find by prefix match)
4. **findBy[Property]EndingWith**  
Example: `List<Employee> findByNameEndingWith(String suffix);`  
(Find by suffix match)
5. **findBy[Property]IsNull**  
Example: `List<Employee> findByNameIsNull();`  
(Find where the property is null)

6. **findBy[Property]IsNotNull**  
Example: `List<Employee> findByIdNameIsNotNull();`  
(Find where the property is not null)
  7. **findBy[Property]In**  
Example: `List<Employee> findByIdIn(List<Integer> ids);`  
(Find by a collection of values)
  8. **findBy[Property]NotIn**  
Example: `List<Employee> findByIdNotIn(List<Integer> ids);`  
(Find by values that are not in a collection)
  9. **findBy[Property]GreaterThan**  
Example: `List<Employee> findByIdGreaterThan(int id);`  
(Find by a property greater than a value)
  10. **findBy[Property]GreaterThanOrEqualTo**  
Example: `List<Employee> findByIdGreaterThanOrEqualTo(int id);`  
(Find by a property greater than or equal to a value)
  11. **findBy[Property]LessThan**  
Example: `List<Employee> findByIdLessThan(int id);`  
(Find by a property less than a value)
  12. **findBy[Property]LessThanOrEqualTo**  
Example: `List<Employee> findByIdLessThanOrEqualTo(int id);`  
(Find by a property less than or equal to a value)
  13. **findBy[Property]Between**  
Example: `List<Employee> findByIdBetween(int start, int end);`  
(Find by a range)
  14. **findBy[Property]Not**  
Example: `List<Employee> findByIdNameNot(String name);`  
(Find where the property is not equal to a value)
  15. **findBy[Property]Like**  
Example: `List<Employee> findByIdNameLike(String name);`  
(Find by pattern match)
  16. **findBy[Property]OrderBy[Property]Asc**  
Example: `List<Employee> findByIdAgeOrderByAgeAsc();`  
(Find by a property and order results ascending)
  17. **findBy[Property]OrderBy[Property]Desc**  
Example: `List<Employee> findByIdAgeOrderByAgeDesc();`  
(Find by a property and order results descending)
-

## Delete Operations

1. **deleteBy[Property]**  
Example: `void deleteByName(String name);`  
(Delete by exact match)
  2. **deleteBy[Property]IsNull**  
Example: `void deleteByNameIsNull();`  
(Delete where the property is null)
  3. **deleteBy[Property]IsNotNull**  
Example: `void deleteByNameIsNotNull();`  
(Delete where the property is not null)
  4. **deleteBy[Property]In**  
Example: `void deleteByIdIn(List<Integer> ids);`  
(Delete by a collection of values)
  5. **deleteBy[Property]NotIn**  
Example: `void deleteByIdNotIn(List<Integer> ids);`  
(Delete by values that are not in a collection)
  6. **deleteBy[Property]GreaterThan**  
Example: `void deleteByIdGreaterThan(int id);`  
(Delete where the property is greater than a value)
  7. **deleteBy[Property]LessThan**  
Example: `void deleteByIdLessThan(int id);`  
(Delete where the property is less than a value)
  8. **deleteBy[Property]Between**  
Example: `void deleteByIdBetween(int start, int end);`  
(Delete by a range)
  9. **deleteBy[Property]Not**  
Example: `void deleteByNameNot(String name);`  
(Delete where the property is not equal to a value)
- 

## Other Operations

1. **existsBy[Property]**  
Example: `boolean existsByName(String name);`  
(Check if the property exists)
2. **countBy[Property]**  
Example: `long countByName(String name);`  
(Count occurrences by exact match)



3. **findFirstBy[Property]**

Example: `Optional<Employee> findFirstByName(String name);`  
(Find the first match based on the given property)

4. **findTopBy[Property]**

Example: `Optional<Employee> findTopByName(String name);`  
(Find the top result)

---

### Additional Functions

1. **findBy[Property]NotLike**

Example: `List<Employee> findByNameNotLike(String name);`  
(Find where the property doesn't match a pattern)

2. **findBy[Property]NotBetween**

Example: `List<Employee> findByIdNotBetween(int start, int end);`  
(Find values that are not in a specified range)

These are the most common derived query methods you can use in Spring Data JPA based on naming conventions. You can combine these methods as needed for more complex queries.

# SPRING FRAMEWORK DAY 21:

# SpringBoot

1.inbuilt method

2.derived query method ( user defined : convention )

3.custom method ( SQL ) : select / non select

4.custom method ( HQL ) : select / non select (insert not supported)

```
@Transactional
@Modifying
@Query(value = "insert into
employee(id,name) values(:id,:name)" ,
nativeQuery = true)
public int insert( @Param("id") int
id, @Param("name") String name);
```

( This way of insertion you can use in both  
CrudRepository and JpaRepository )

In case of non select operation you must use  
@Modifying and @Transactional with @Query

```
10 public interface MyJpaRepository extends JpaRepository<Employee, Integer> {
11
12     @Transactional
13     @Modifying
14     @Query(value = "insert into employee(id, name) values(?, ?)")
15     public int insert( @Param("id") int id, @Param("name") String name);
16
17     @Transactional
18     @Modifying
19     @Query(value = "delete from employee where id = ?")
20     public int delete( @Param("id") int id);
21
22     @Transactional
23     @Modifying
24     @Query(value = "update employee set name = ? where id = ?")
25     public int delete(@Param("name") String name, @Param("id") int id);
26
27 }
```

# **S**PRING

# **F**RAMEWORK

## **DAY 22:**

### **SpringBoot**

How to work with MVC in  
Springboot ( web application )

**1.JSP**

**2.THYMELEAF**

# How to work with JSP in springboot

(JSP): VIEW

Private int id

Private String ename;

.java----->.class

Three Extra Dependencies

1.validation (DATA VALIDATION)

2.lombok (GENERATES BOILERPLATE CODE)

3.tomcat jasper (JSP SUPPORT)

ONE PLUGIN:

---

## Eclipse Enterprise Java and Web Developer T



Enables Enterprise Java Bean, Java Enterprise Application, (JSF), Java Server Pages (JSP), Java... [\*\*more info\*\*](#)

by [The Eclipse Foundation](#), EPL  
[xml](#) [html](#) [CSS](#) [js](#) [jsp](#)

★ 1805



Installs: **1.32M** (12,090 last month)

---

webapp : static

WEB-INF : Dynamic

# **SPRING**

# **FRAMEWORK**

## **DAY 23:**

### **SpringBoot**

**->SPRING BOOT MVC**

**CRUD**

**->DATA VALIDATION**

**->MODELMAPPER**

# How to validate data through Annotation(Hibernate Validator)

1.primitive

2.Object

3.CharSequence Object  
( String )

EMPLOYEE DATA: id name address salary date  
ucode(application)



**S**PRING

**F**RAMWORK

**DAY 24:**

**File Uploading**

**S**PRING

**F**RAMWORK

**DAY 25:**

**Thymeleaf rec**

**provided ( FROM**

**ANOTHER BATCH )**

# **S**PRING

# **F**RAMEWOR**K**

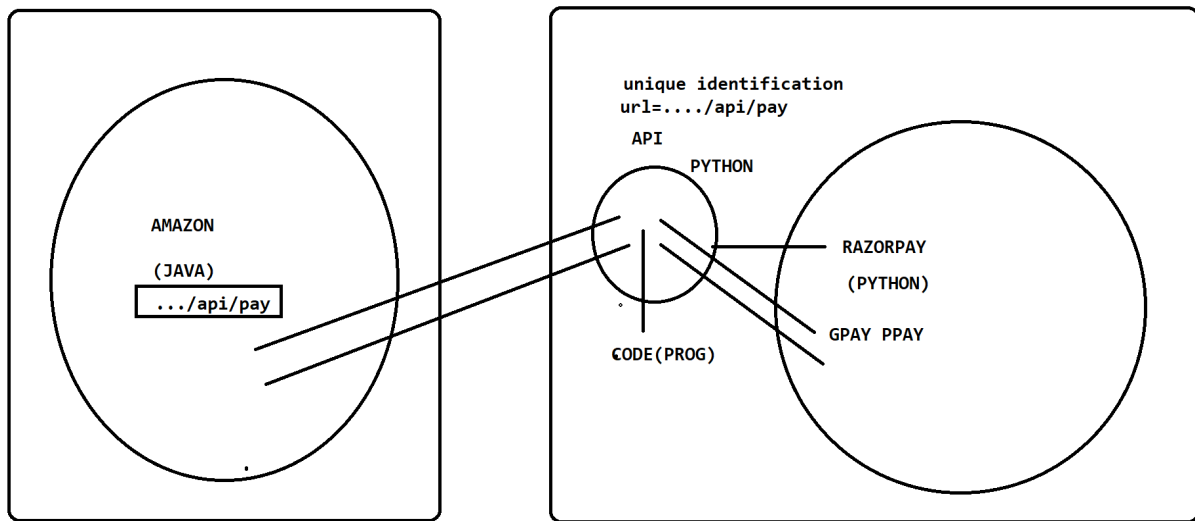
## **DAY 26:**

# **REST API**

## **WHAT IS AN API?**

- API is a set of rules and protocols that allow different software application to communicate each other.
- API is just a piece of CODE  
(Libraries, Packages, Classes, Interfaces)

- API acts like a bridge between two softwares, enabling them to exchange the data
- API is language agnostic
- API provides endpoint to connect with them



## WHAT IS REST?

REST is an architectural style to create a network application given by DR ROY FIELDING in 2000

IT provides set of principles to create a network application.

REST is language agnostic

If an API is created by following the principles of REST then that API is called REST API

REST API/RESTFUL API/RESTFUL SERVICES

DR ROY FIELDING: DOCTORAL DISSERTATION

# **PRINCIPLES OF REST:**

## **1.CLIENT SERVER ARCHITECTURE**

- The client(FE) and server(BE) are separate
- Server handles request and the client only need to know the API endpoints

## **2.STATELESSNESS**

- Each request from the client to the server must contain all the necessary information.
- Server does not store client data (SESSION DATA)between request call

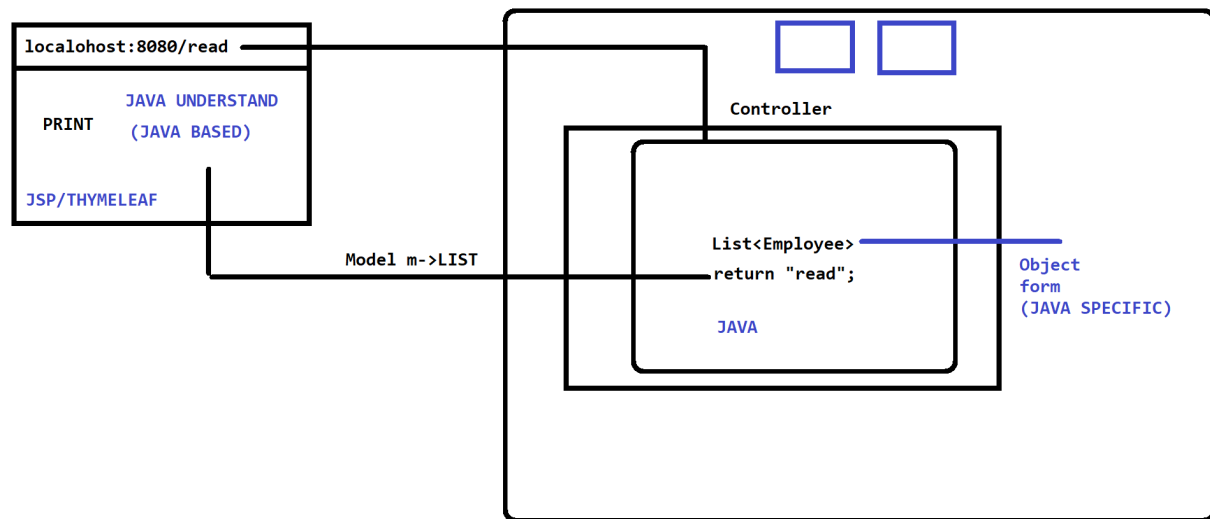
### **3.RESOURCE BASED**

Everything in a REST API is treated as a RESOURCE (read,product,user...etc)

Resources are identified by a unique URL(SINGLE or MULTIPLE)

## **WHY WE SHOULD CREATE AN API?**

How monolithic application works?



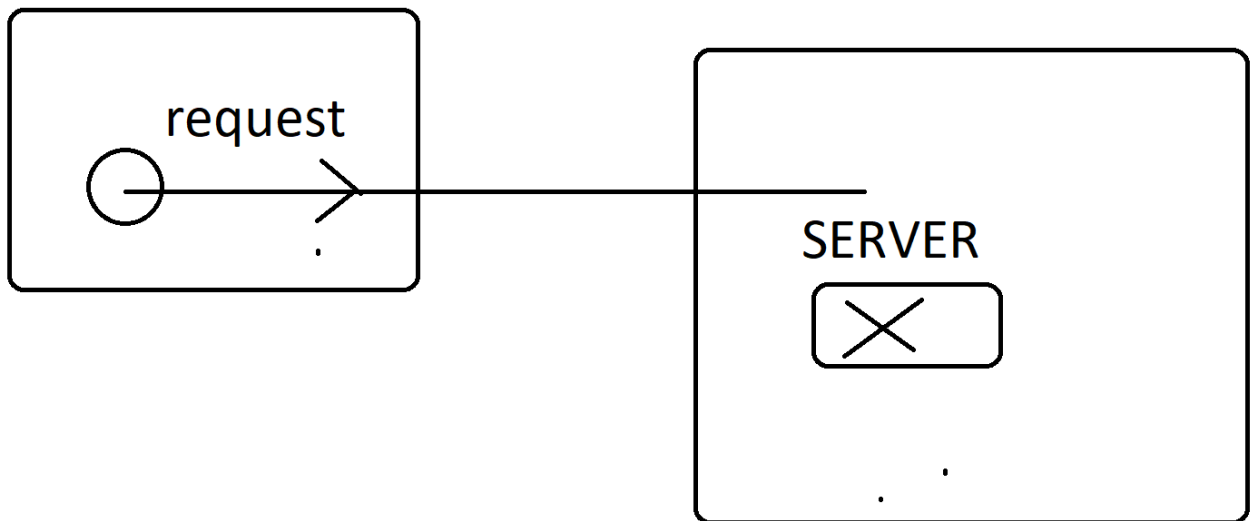
**Monolithic application:** containerized on a single server

Ex.Portfolio-->WAR-->HOST

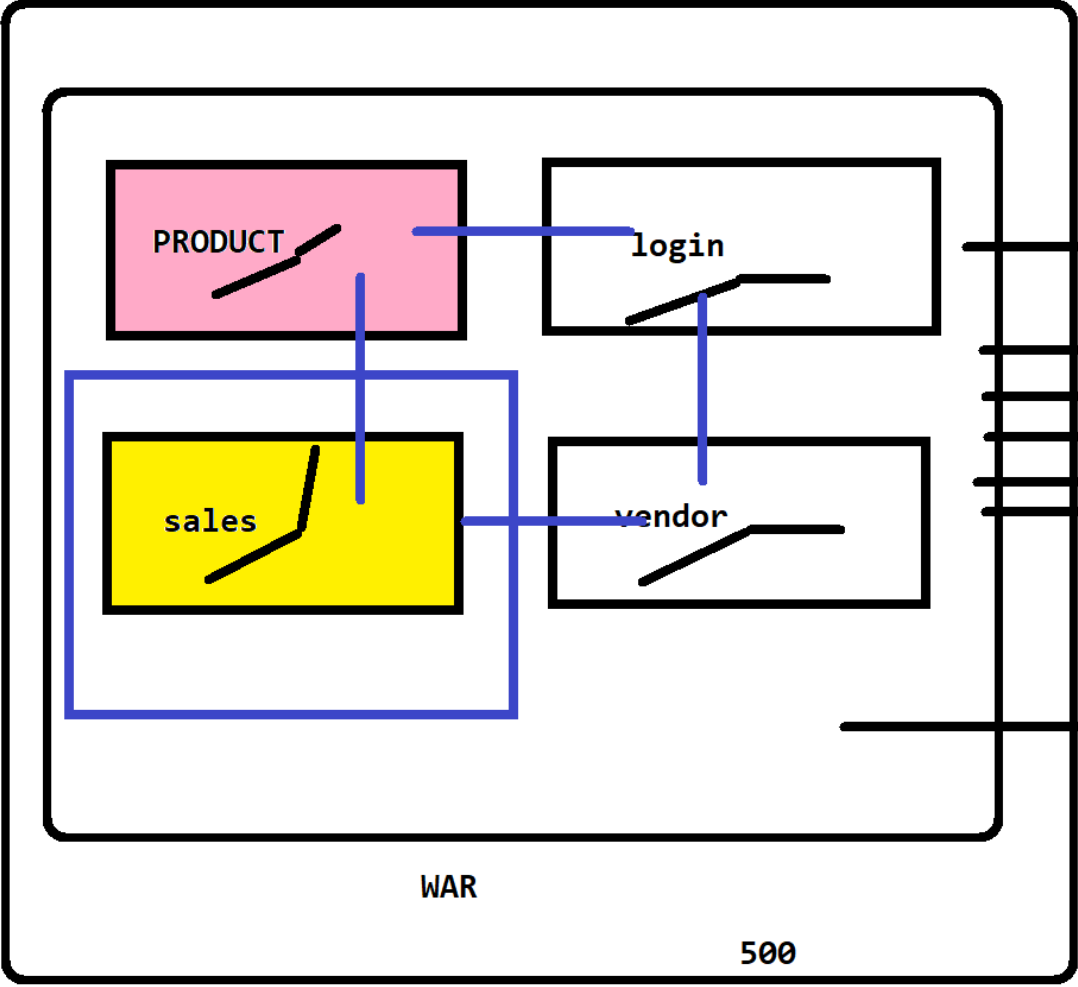
1. Everything in a single server
2. Mostly preferred Java Based FE technologies ( JSP, THYMELEAF)
3. FE----->BE ( JAVA KNOWS HOW TO GET DATA FROM THE CLEINT(FORM,URL,<a>)
4. BE----->FE ( IN JSP OR THYMELEAF U CAN EASILY ITERATE JAVA BASED DATA)
5. SUITABLE FOR SMALL SCALE APPLICATION

DISADVANTAGES:

**1.PROBLEM WHILE SCALING (TIGHTLY COUPLED)**

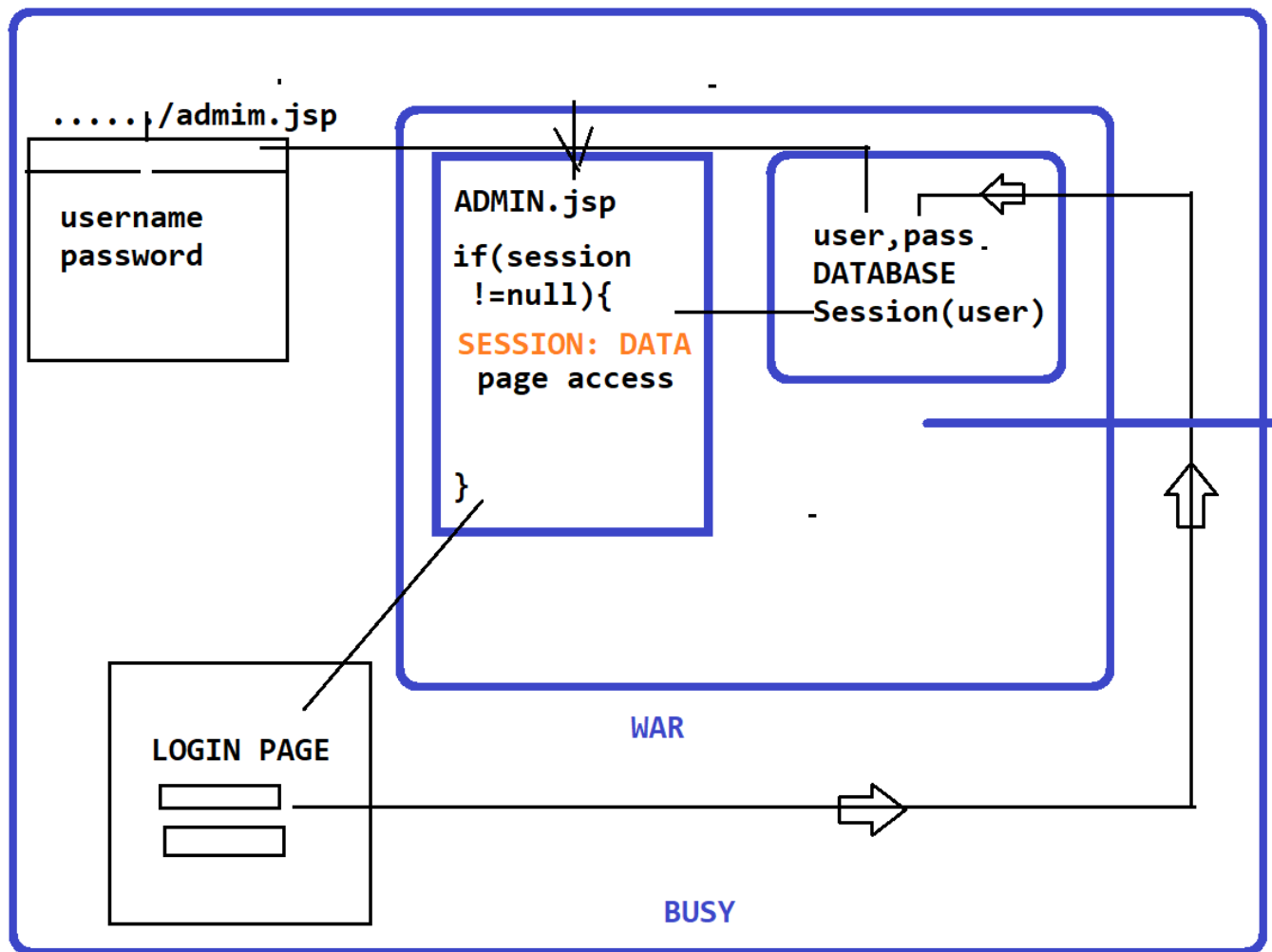






9RS/1DAY OFFE

NGINX -> LOAD BAL



# **S**PRING

# **F**RAMEWORK

## **DAY 27:**

# **REST API**

# **JSON:**

**JAVA SCRIPT OBJECT NOTATION ( JSON) is a lightweight data exchange format, it is easy for human to write and read and easy for machine to interpret and it is language agnostic so it is**

supported by multiple programming languages.

```
package com.codehunt.controller;
import java.util.Map;
import
org.springframework.beans.factory.annotation.Au
towired;
import
org.springframework.stereotype.Controller;
import
org.springframework.web.bind.annotation.GetMapp
ing;
import
org.springframework.web.bind.annotation.Respons
eBody;

import com.codehunt.service.MyService;

@Controller
public class MyController {

    @Autowired
    private MyService myService;

    @ResponseBody
    @GetMapping("api/readsales")
```

```
    public Map<Integer, Integer>
readSalesData(){

    return myService.salesData();
}
}
```

```
package com.codehunt.service;
import java.util.HashMap;
import java.util.Map;

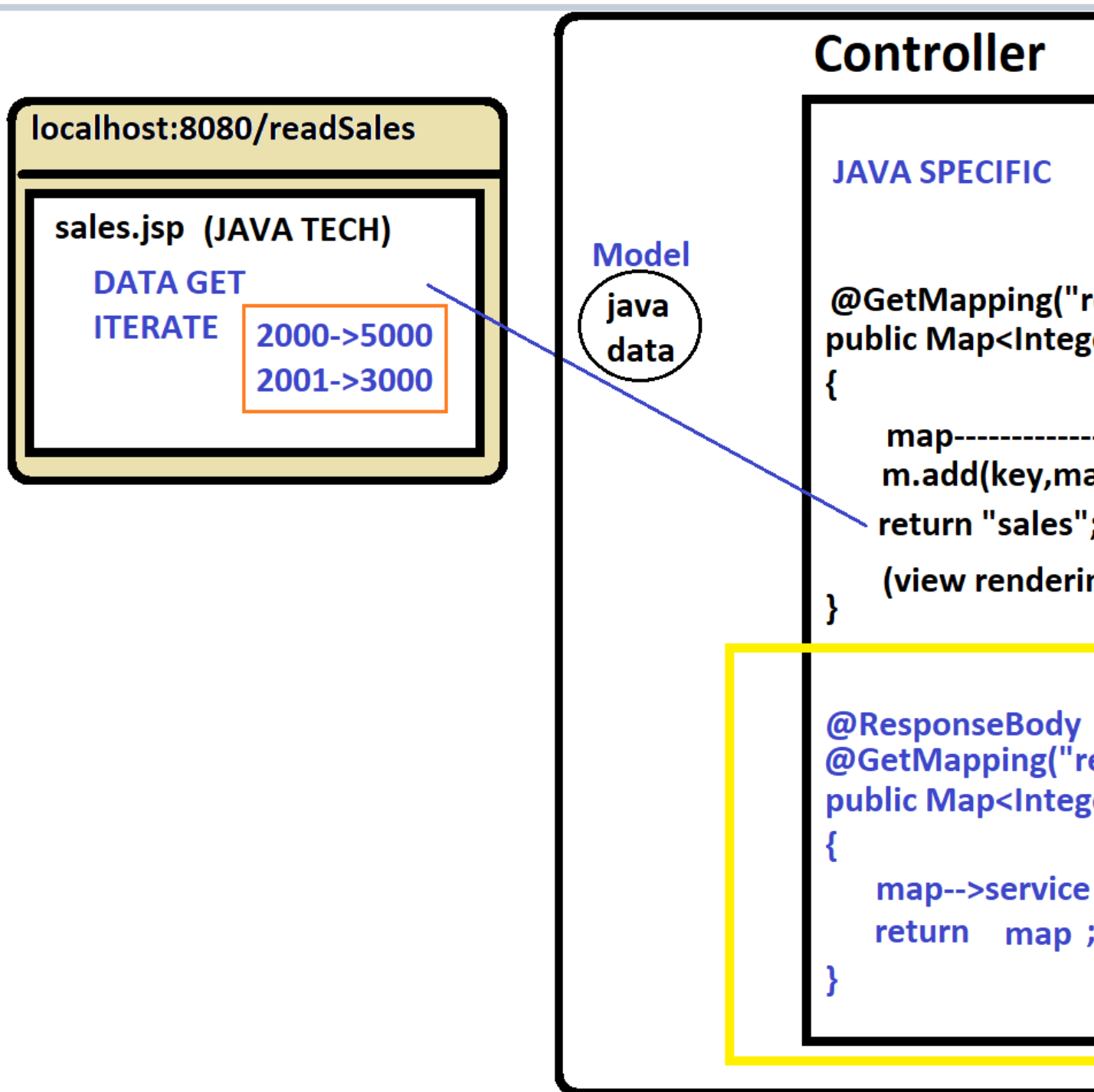
import org.springframework.stereotype.Service;
@Service
public class MyService {

    public Map<Integer, Integer> salesData() {

        Map<Integer, Integer> map = new
HashMap();
        map.put(1996, 2000);
        map.put(1997, 1000);
        map.put(1998, 9000);
        map.put(1999, 5000);
        map.put(2000, 2000);
        map.put(2001, 1000);
        map.put(2002, 7000);

        return map;
    }
}
```

}



- `import requests`
- `import matplotlib.pyplot as plt`
- `url="http://localhost:8080/api/readsales"`
- `response=requests.get(url)`
- `sales=response.json()`
- `year=list(sales.keys())`
- `sale=list(sales.values())`
- `plt.figure(figsize=(8,5))`
- `plt.plot(year,sale,label="SALES")`
- `plt.xlabel("YEARS")`
- `plt.ylabel("SALES")`
- `plt.title("CAR SALES DATA")`
- `plt.legend()`
- `plt.grid(True)`
- `plt.show()`

**S**PRING

**F**RAMEWORK

**DAY 28:**

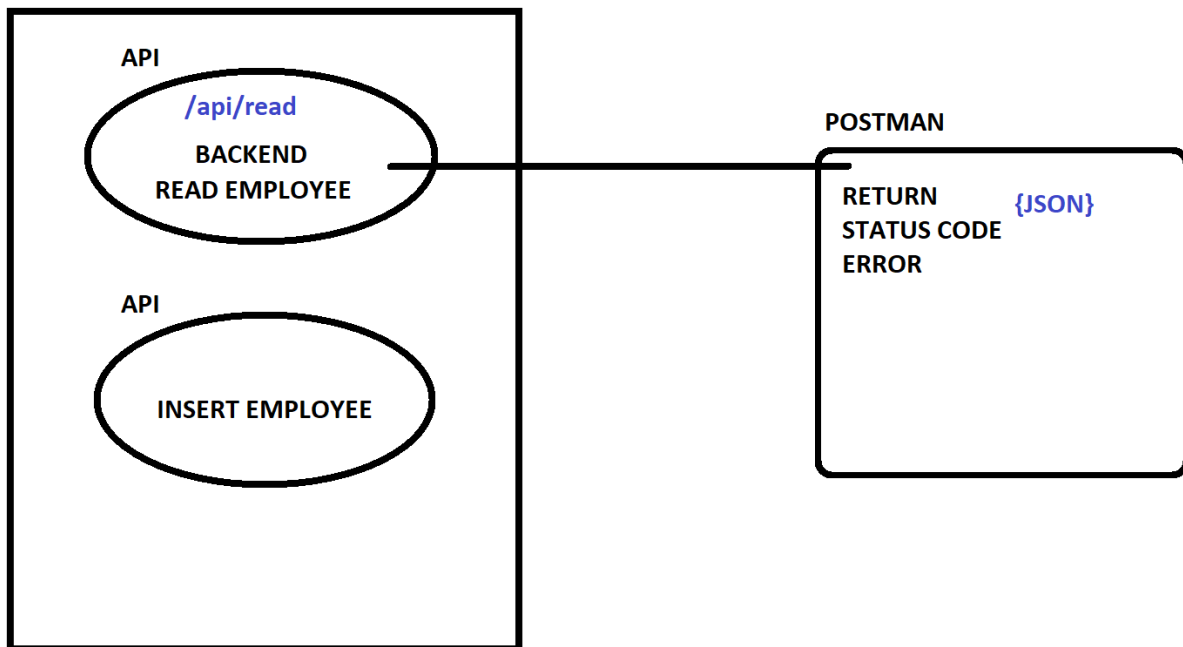
**REST API**

**API(CREATE)-----**

**----->TEST(POST**

**MAN)**





# 1.@RestController

```
package com.codehunt.controller;
import
org.springframework.beans.factory.annotation.Au
towed;
import
org.springframework.web.bind.annotation.GetMapp
ing;
import
org.springframework.web.bind.annotation.RestCon
troller;
import com.codehunt.service.MyService;
@RestController
public class MyController {
```

```
@Autowired
private MyService myService;

@GetMapping("api/readtest")
public String test(){

    return myService.read();

}
```

## INSERT:

(POSTMAN)----->CONTROLLER(@RequestBody DTO)

## INSERT:

- WHEN DATA IS SUCCESSFULLY INSERTED:  
201 (CREATED)
- WHEN DATA IS INVALID: 400 (BAD REQUEST)

- **WHEN DATA IS ALREADY EXIST : 409 CONFLICT**
- **OTHERWISE: 500 ( INTERNAL SERVER ERROR )**

## **READ:**

- **STATUS WITH SOME DATA : 200 ( OK )**
- **WHEN DATA IS NOT FOUND: 404 (NOT FOUND) : id--->valid--->id does not exist**
- **WHEN NO CONTENT FOUND: 204 ( NO CONTENT): id---->valid--->id exist--->but data not found**
- **WHEN AUTHENTICATION FAILED: 401 ( UNAUTHORIZED )**
- **WHEN LACK OF PERMISSION : 403 ( FORBIDDEN )**
- **WHEN PAGINATION: 206 ( PARTIAL CONTENT)**

## **DELETE:**

- **WHEN DATA IS NOT FOUND: 404 (NOT FOUND) : id--->valid--->id does not exist**
- **WHEN NO CONTENT FOUND: 204 ( NO CONTENT): id---->valid--->id exist--->but data not found**
- **STATUS WITH SOME DATA : 200 ( OK )**

@RequestBody -> JSON OBJECT

@PathVariable -> DATA FROM URL (DIRECT)

Ex. localhost:8080/api/delete/9

**S**PRING

**F**RAMEWOR**K**

**DAY 29:**

**REST API**

**UPDATE:**

**1.FULL OBJECT UPDATE**

**( use @PutMapping)**

**2.PARTIAL OBJECT  
UPDATE (use  
@PatchMapping)**

**S**PRING

**F**RAMWORK

**DAY 30:**

# REST API

## (MINI PROJECT)

- CRUD OPERATION
- DATA VALIDATION
- JPA REPO
- MODEL MAPPER

=====

INSERT ( @PostMapping ) :  
/employees

READ ( @GetMapping ) :  
/employees

UPDATE( @PutMapping):  
/employees/{id}

DELETE ( @DeleteMapping):

/employees/{id}

PARTIAL UPDATE ( @PatchMapping ):

/employees/{id}

# **S**PRING

# **F**RAMEWOR**K**

## **DAY 31:**

# **REST API**

PROJECT: READ SINGLE AND UPDATE ALL



**S**PRING

**F**RAMEWORK

**DAY 32:**

**REST API**

PROJECT: PARTIAL UPDATE VALIDATION

**S**PRING

**F**RAMEWOR**K**

**DAY 33:**

**REST API**

PROJECT: PARTIAL UPDATE JAVAX VALIDATION

**S**PRING

**F**RAMEWOR**K**

**DAY 34:**

**REST API**

PROJECT: PARTIAL UPDATE & DELETE

# SPRING

# FRAMEWORK

## DAY 35:

# Spring Security

## 1.What is Spring Security?

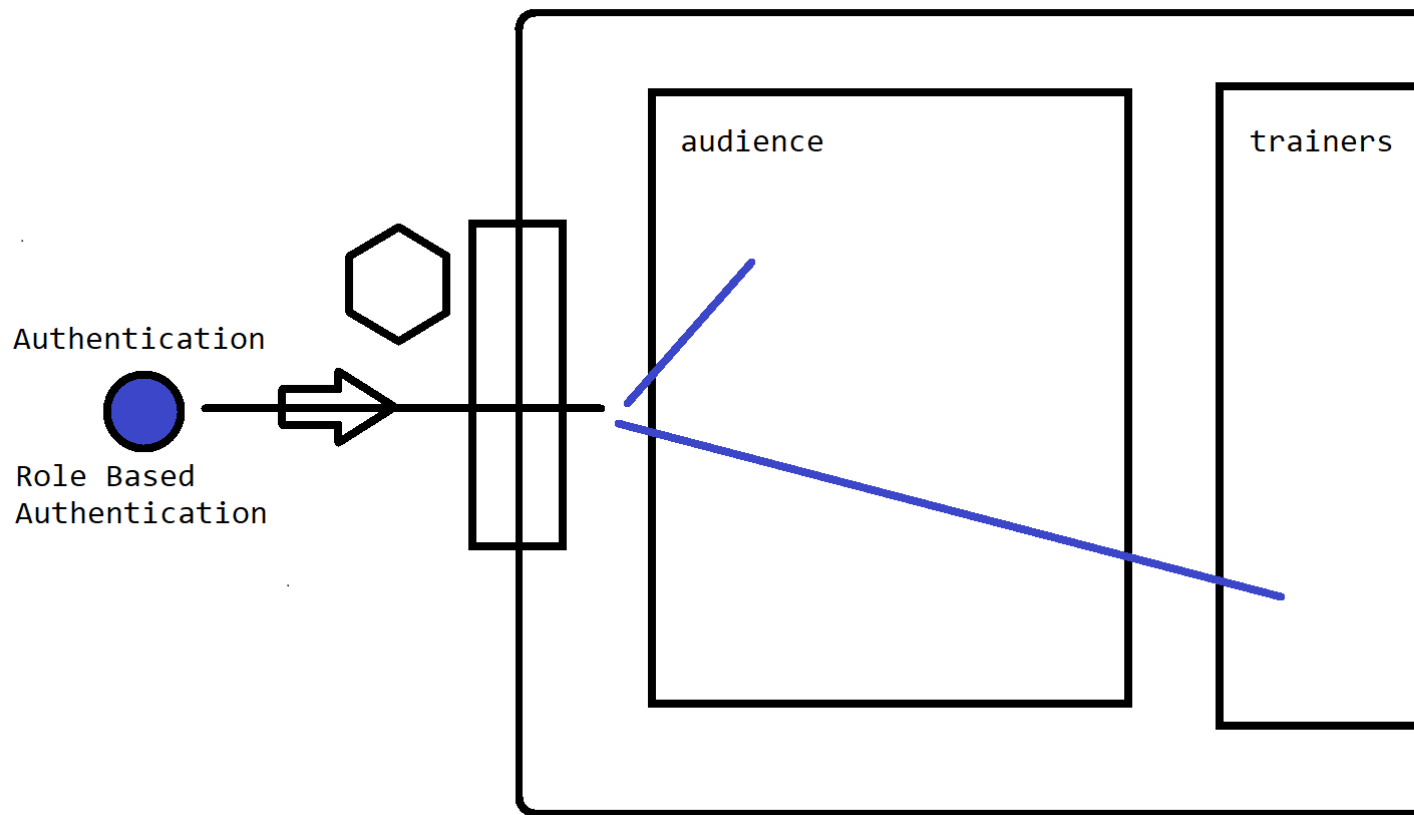
- Spring Security is a framework that provides authentication, authorization, and protection against common attacks.
- Developed by BEN ALEX 2003 (ACEGI SECURITY)

## **2.Need of Spring Security?**

- Hacking Attack
- Security Majors
- Easy Implementation

## **What is Authentication and Authorization?**

Authentication is a the process of verifying the identity of a user



## What is Authorization?

It is a process of determining what action a user is allowed to perform after authentication

localhost:8080/admin

admin e

admin e

ADMIN PANEL

employee

admin

admin

READ CONTACTS /read

DELETE CONTACTS /delete

**SPRING**

**FRAMEWORK**

**DAY 36:**

**Spring Security**

**PART-2**

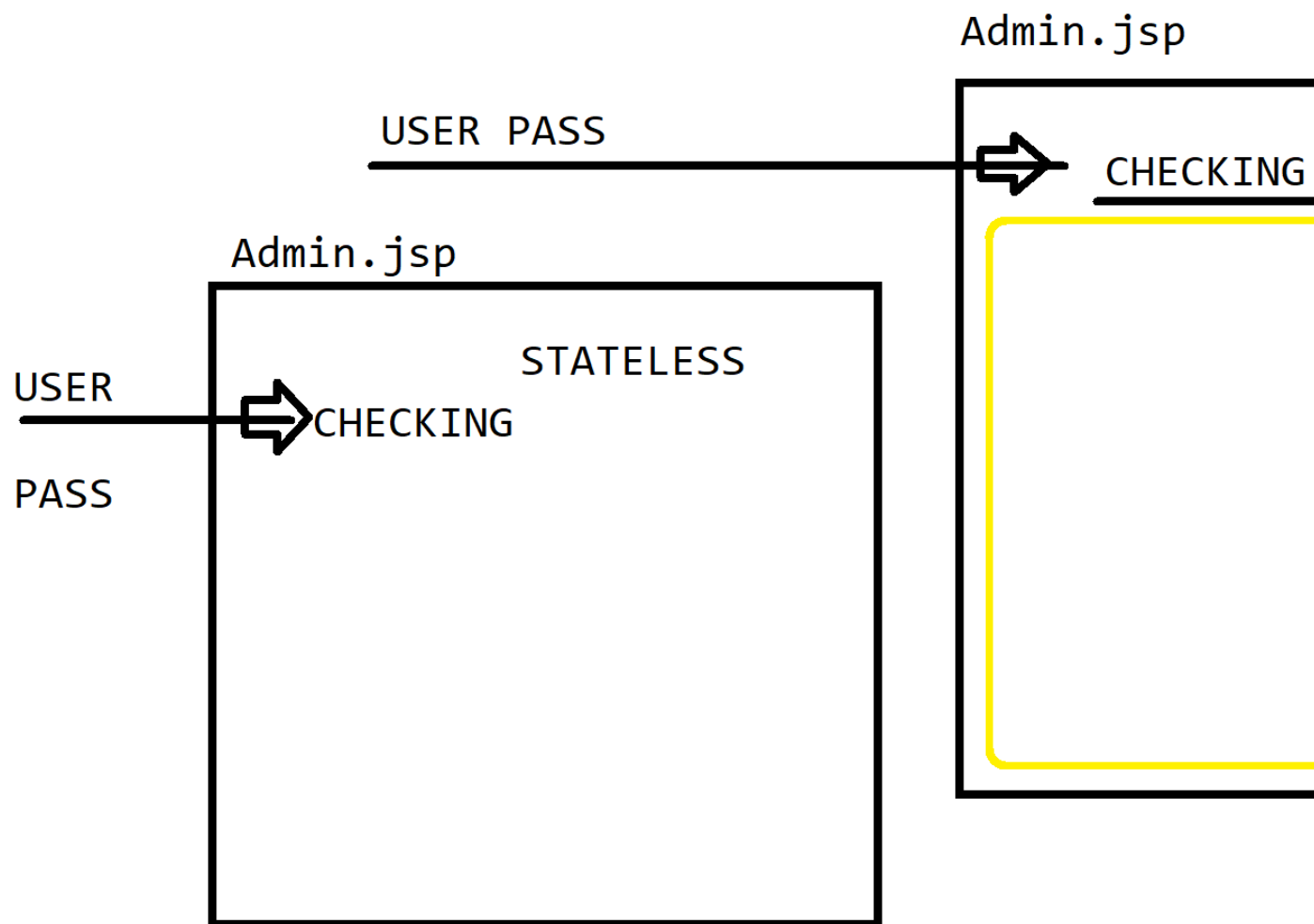
**Types of  
Authentication**



- 1. BASIC AUTHENTICATION**
- 2. FORM BASED AUTHENTICATION**
- 3. TOKEN BASED AUTHENTICATION(JWT)**
- 4. OAuth2 Authentication(GOOGLE LOGIN)**

# **BASIC AUTHENTICATI ON**

Basic authentication is a simple authentication mechanism where client send a **username and password** with every request.



**Basic authentication does not support logout because it is stateless (NO DATA MAINTAINED ON SERVER (SESSION))**

## **USE:**

### **1. SIMPLE WORK**

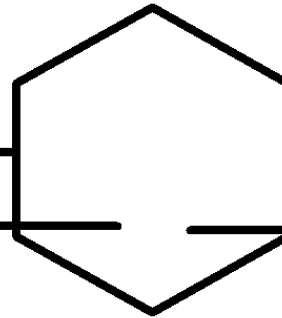
#### **CODEHUNT SOFTWARE TRAINING**

- **ONLINE EXAM PAGE:**
- **ONLINE NOTES:**

username  
password

localhost:8080/exam

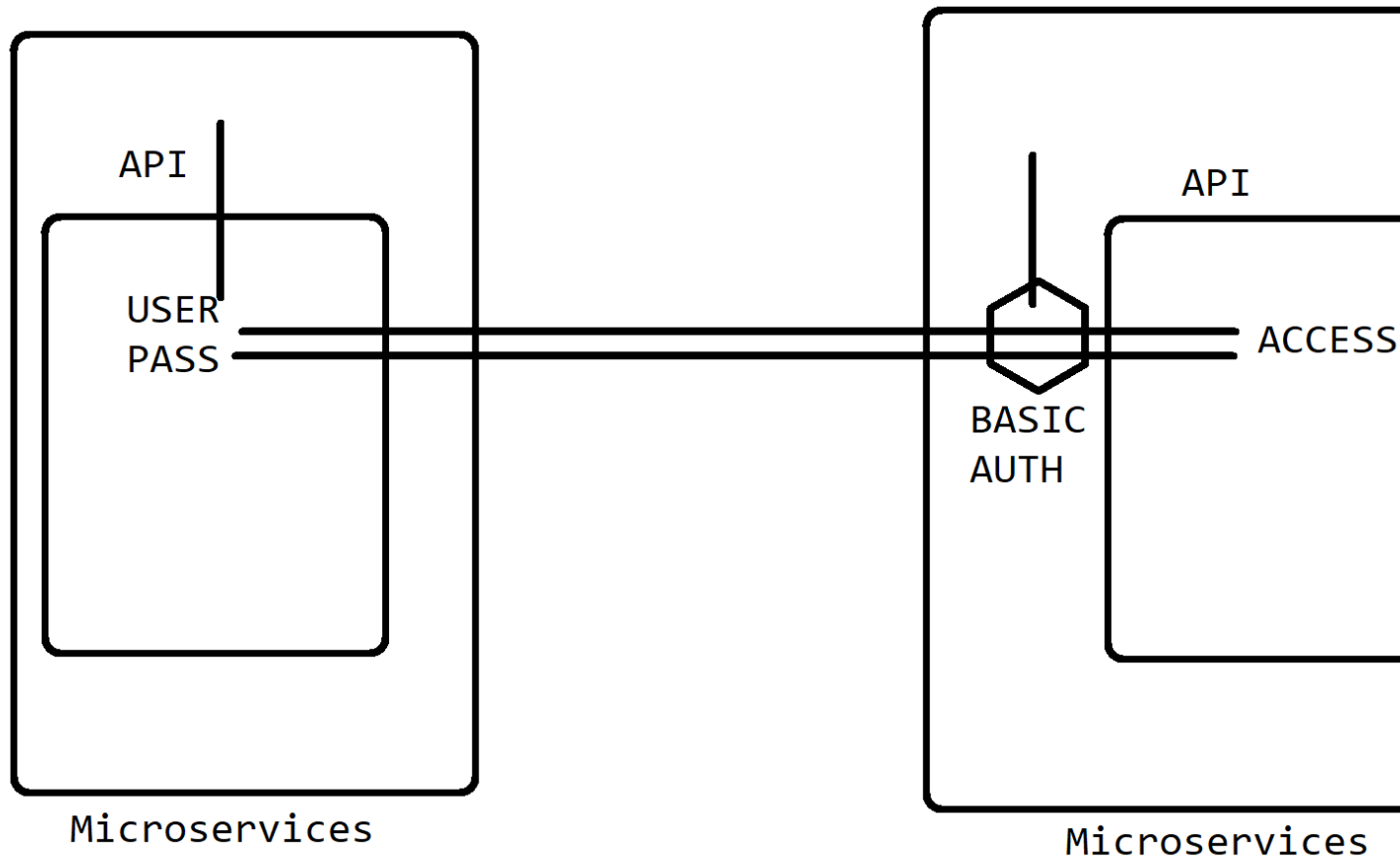
|                  |
|------------------|
| codehuntexampage |
| codehuntexampage |



BASIC AUTH

ONLINE  
NOTES

## 2. API COMMUNICATION ( MICROSERVICES )



## **BASIC AUTHENTICATION IS NOT PREFERRED FOR WEB APPS**

1. Credentials are sent in every request
2. No logout option
3. vulnerable to MIMT(MIDDLE IN THE MAN) attacks ( if not used over HTTPS )

# **PROJECT (**

# **BASIC**

# **AUTHENTICATI**

# **ON )**

localhost:8080/index

localhost:8080/admin

localhost:8080/member

admin

member car

IN MEMORY  
DATABASE

**S**PRING

**F**RAMEWORK

**DAY 37:**

**Spring Security**

**PART-3**

**Version:**



- **Springboot 2.7.X ----->Spring Security**

- 5.7.X----->JAVA 8**

- **Springboot 3.X ----->Spring Security 6.X**

When u put this dependency, automatically **form based authentication** has been applied

```
<dependency>
```

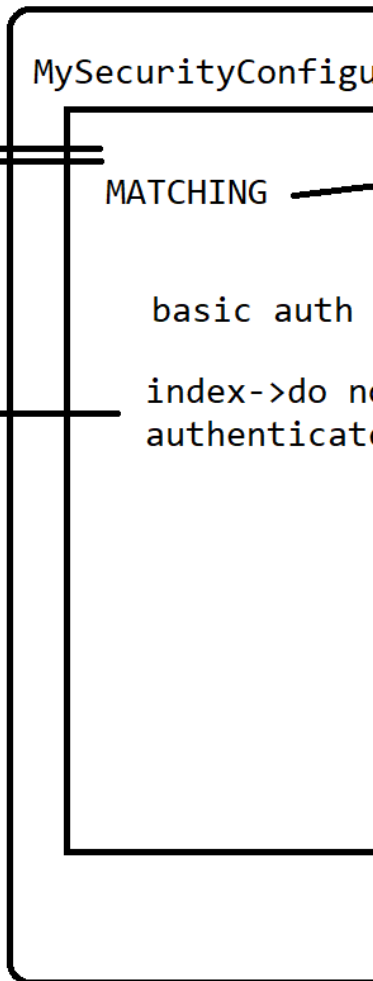
```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

- To switch from **form based authentication** to **basic authentication** you need to do some **configuration**
- For configuration in spring security 5.7 there is a deprecated class Called **WebSecurityConfigurerAdapter**
- Replaced by **SecurityFilterChain** (IN USE)

localhost:8080/admin  
USER PASS

localhost:8080/index



**S**PRING

**F**RAMWORK

**DAY 38:**

**Spring Security**

**PART-4**

# ROLE BASED AUTHENTICATION

First we should create two roles (**admin**, **member**) and we need to separate the credentials of both.

- **IN MEMORY AUTHENTICATION**
- **DATABASE AUTHENTICATION**

## **DEFINITION:**

- USER CREDENTIALS ARE STORED IN MEMORY (RAM) {HARDCODED IN THE APPLICATION}
- USER CREDENTIALS ARE STORED IN DATABASE AND RETRIEVE DYNAMICALLY

## **USE CASE:**

- SMALL APPLICATION, TESTING OR QUICK DEMO
- SUITABLE FOR LARGE SCALABLE APPLICATION

## **PERSIST:**

- NOT PERSIST, RESET WHEN THE APPLICATION RESTART
- USER DATA REMAINS IN THE DATABASE

## **SCALABLE:**

- NOT SCALABLE
- SCALABLE

## **SECURITY:**

- LESS MEMORY
- SECURED

## USER MANAGEMENT

- YOU CANNOT DYNAMICALLY ADD, REMOVE OR MODIFY USER
- CAN MANAGE USERS DYNAMICALLY

# IN MEMORY AUTHENTICATION

Note: Spring security will treat your **error.jsp** as a default error page

- When u try to access member page through admin credentials -> error.jsp
- When u cancel login -> error.jsp

- When u try to hit any URL which is not exist -> error.jsp
- If u are trying to access member but with wrong credentials -> TRY AGAIN

# **S**PRING

# **F**RAMWORK

# **DAY 39:**



# **Spring Security**

## **PART-5**

### **DATABASE**

### **AUTHENTICATION**

- USER CREDENTIALS ARE STORED IN DATABASE AND RETRIEVE DYNAMICALLY
- SETTING CREDENTIALS INSIDE THE DATABASE

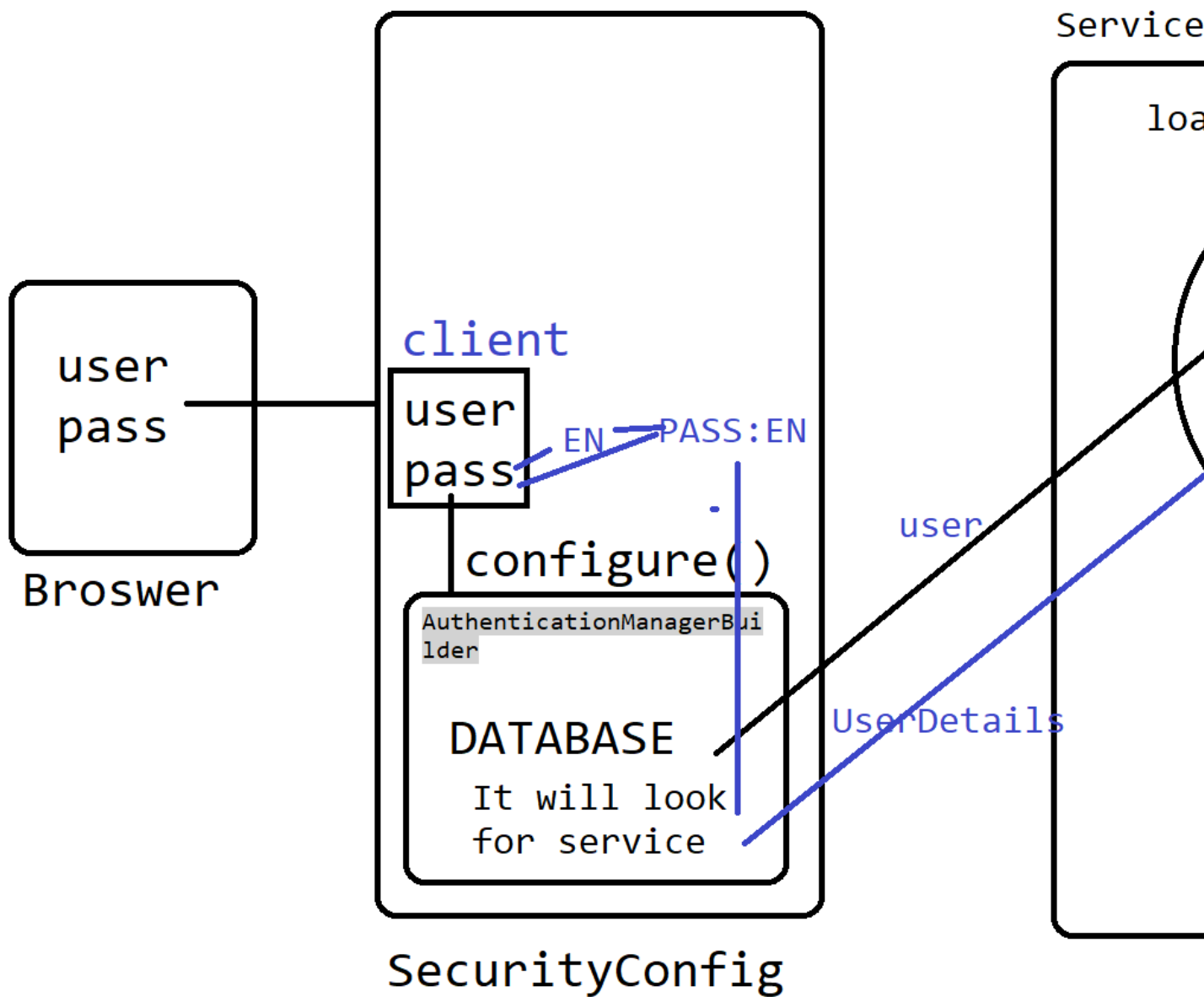
**S**PRING

**F**RAMEWOR**K**

**DAY 40:**

**Spring Security**

**PART-6**



```

@Override
public UserDetails
loadUserByUsername(String username)
throws UsernameNotFoundException {

```

```
        System.out.println(username);
        Optional<User> opt =
userRepo.findByUsername(username);
        if(opt.isPresent()) {
            User user = opt.get();
//username password role
            System.out.println(user);
            return new
UserDetailsImpl(user);
        }
        else {
            throw new
UsernameNotFoundException("USER NOT
FOUND");
        }
    }
```

# SPRING

# FRAMEWORK

## DAY 41:

# Spring Security

## PART-7

```
package com.codehunt;  
import  
org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import  
org.springframework.security.config.annotation.authentication  
n.builders.AuthenticationManagerBuilder;
```

```
import
org.springframework.security.config.annotation.web.builders.
HttpSecurity;
import
org.springframework.security.config.annotation.web.configura
tion.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configura
tion.WebSecurityConfigurerAdapter;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEnc
oder;

import com.codehunt.service.MyService;
@SuppressWarnings("deprecation")
@Configuration
@EnableWebSecurity
public class SecurityConfig extends
WebSecurityConfigurerAdapter {
```

```
    @Autowired
    private MyService myService;

    @Override
    protected void configure(HttpSecurity
http) throws Exception {

        http
            .authorizeHttpRequests()

            .antMatchers("/member").hasRole("MEMBER")

            .antMatchers("/admin").hasRole("ADMIN")
```

```
_____ .anyRequest().permitAll()  
_____ .and()  
_____ .httpBasic()  
_____ .and()  
_____ .exceptionHandling()  
_____ .accessDeniedPage("/error/403");  
  
}
```

```
@Override  
    protected void  
configure(AuthenticationManagerBuilder  
auth) throws Exception {
```

```
        //FROM DATABASE: MyService
```

```
auth.userDetailsService(myService  
e).passwordEncoder(bCryptPasswordEncoder());
```

```
//    auth  
//        .inMemoryAuthentication()  
//        .withUser("admin123")
```

```
//
.password(bCryptPasswordEncoder(
).encode("admin333"))
//      .roles("ADMIN")
//      .and()
//      .withUser("member123")
//
.password(bCryptPasswordEncoder(
).encode("member333"))
//      .roles("MEMBER");

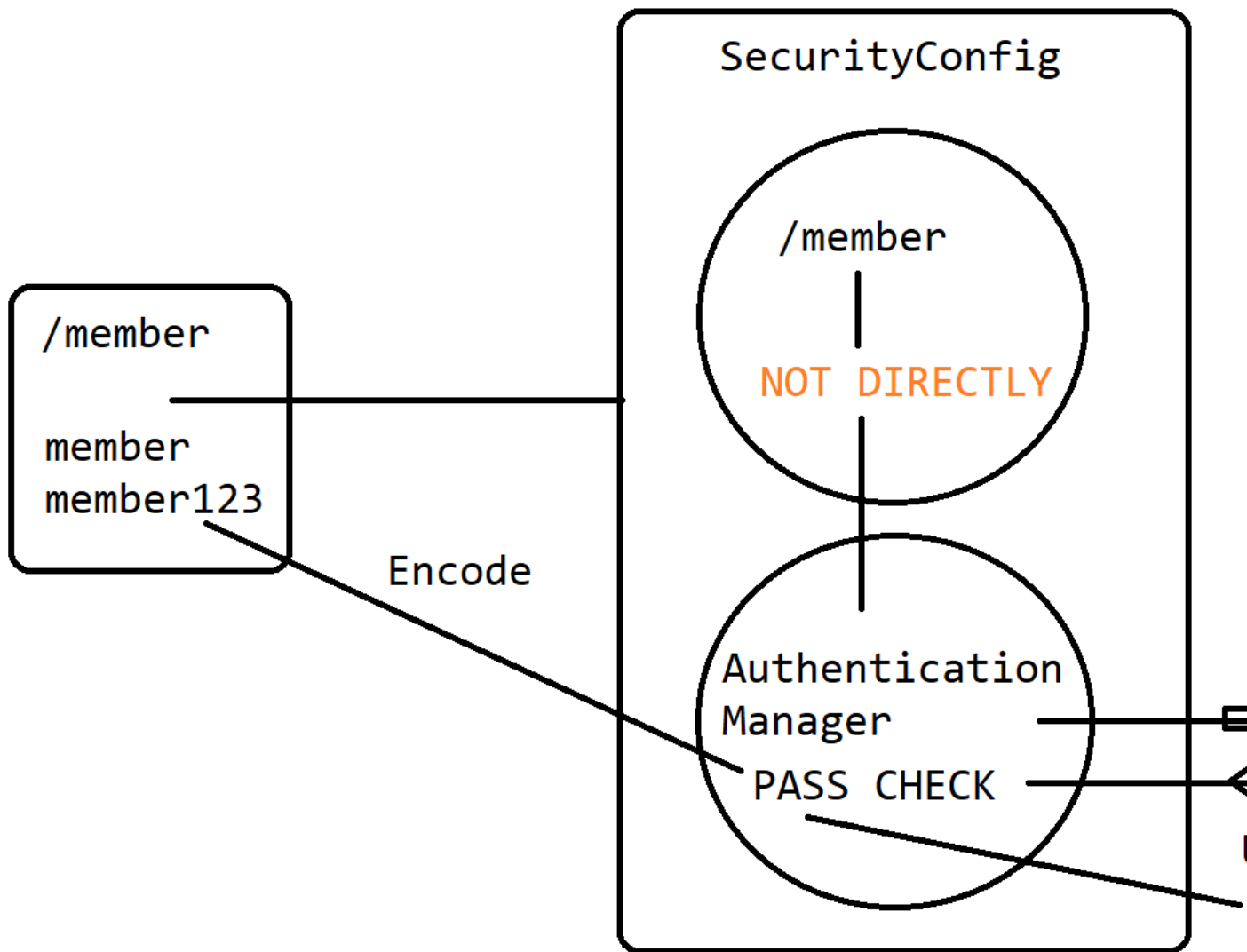
}

@Bean
public BCryptPasswordEncoder bCryptPasswordEncoder() {
    return new BCryptPasswordEncoder();
}
}
```

by **SecurityFilterChain**

❓ COMPONENT BASED CONFIGURATION





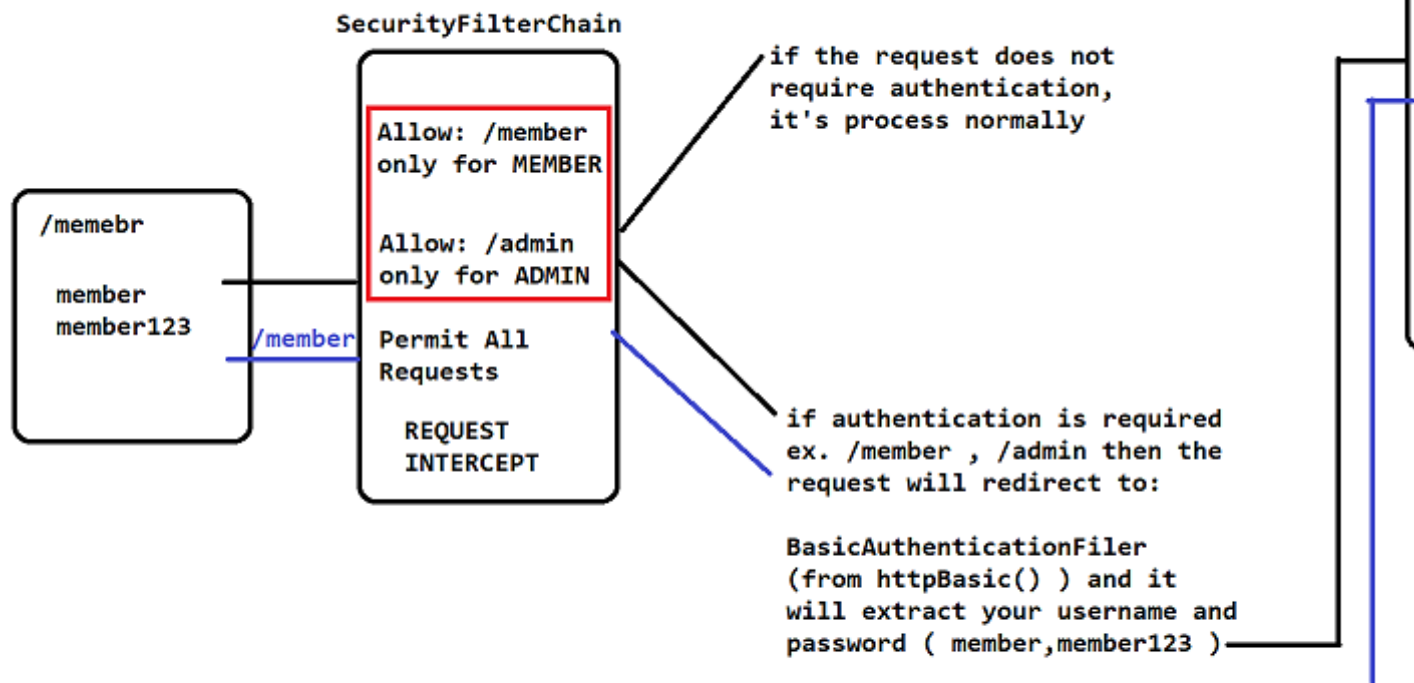
**S**PRING

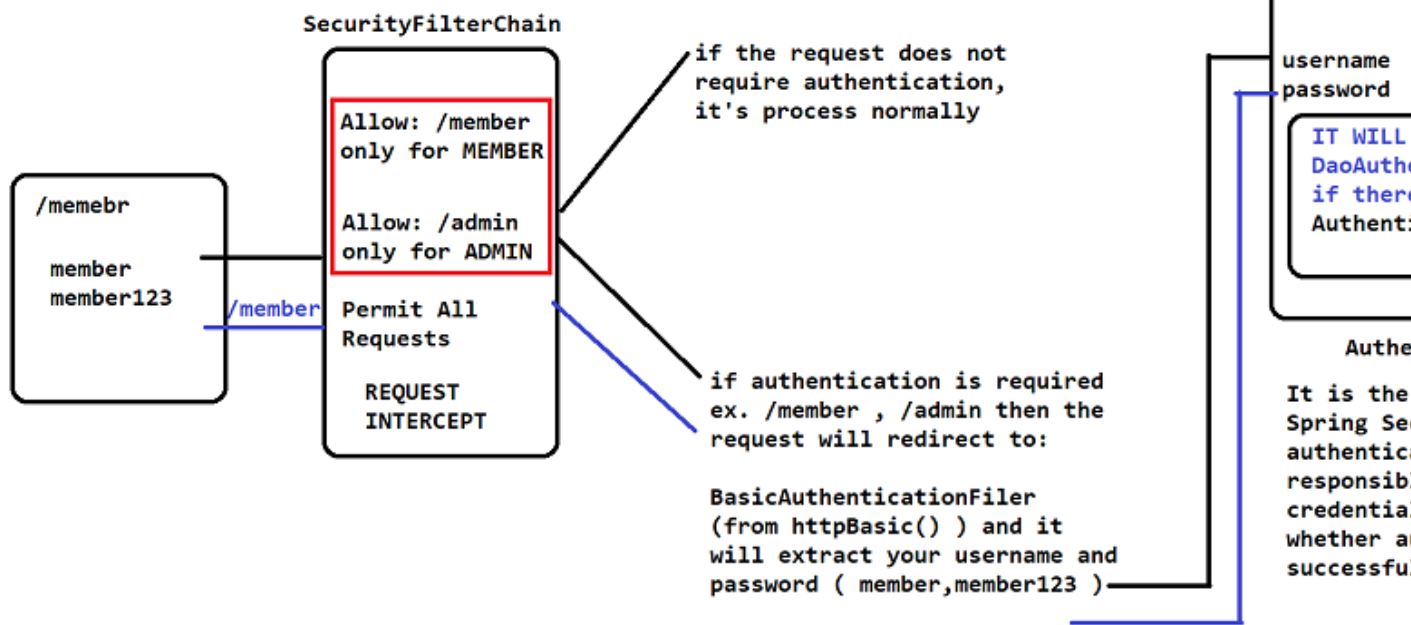
**F**RAMWORK

**DAY 42:**

**Spring Security**

**PART-8**





**S**PRING

**F**RAMWORK

**DAY 43:**

**Spring Security**

**PART-9**

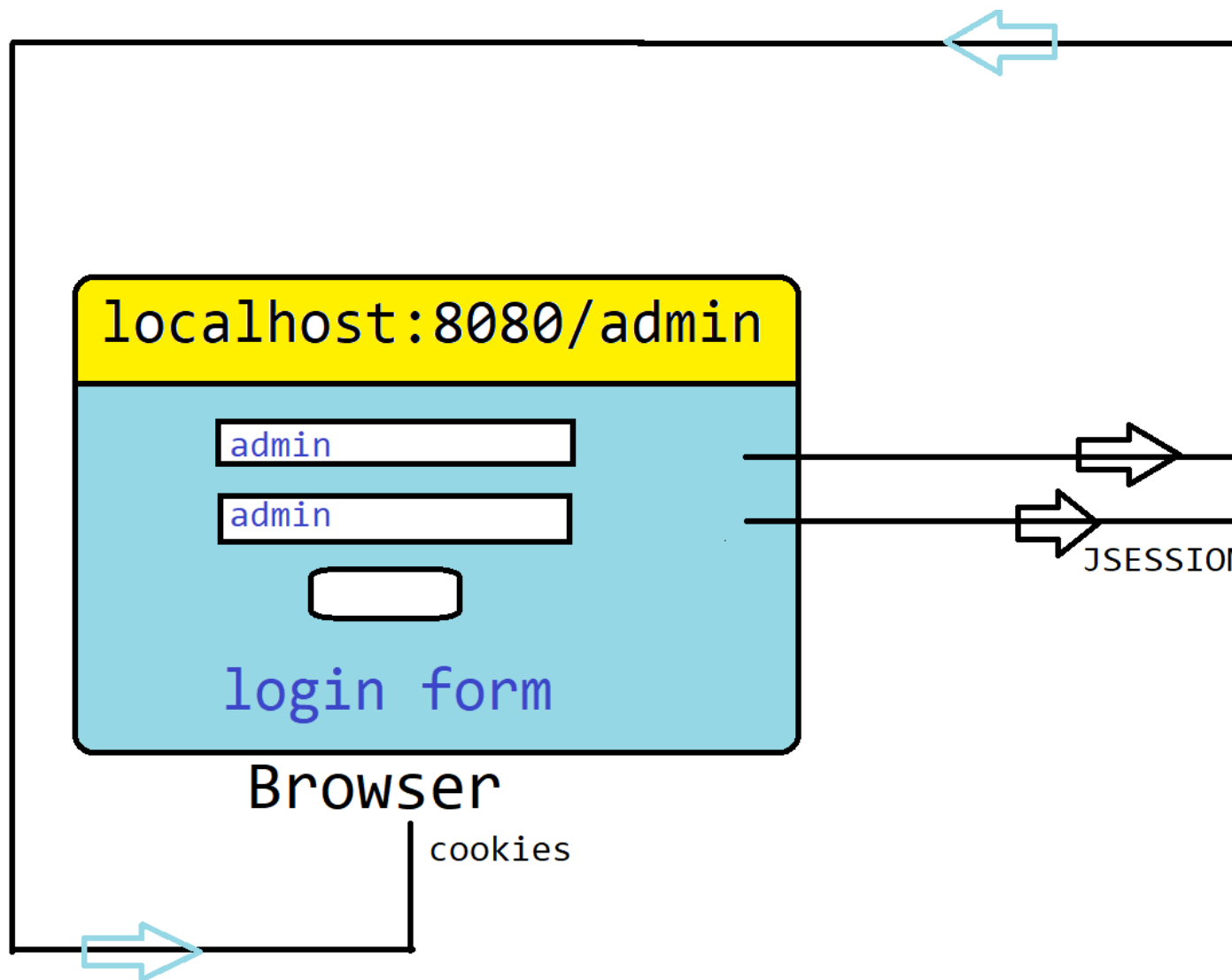
# **FORM BASED AUTHENTICATION (BY DEFAULT)**

**->WEB APPLICATION**

**->SESSION BASED**

**->HTML FORM**

**->**



Form-based authentication is a login mechanism where users provide their credentials (username & password) through an

HTML login form. The server processes the credentials, authenticates the user, and establishes a session for future requests.

## Explanation of Form-Based Authentication Flow:

When a user logs in with correct credentials, the server manages authentication using sessions. Let's break it down:

---

1.If credentials are correct, the server creates a session.

- When a user submits their username and password, the server verifies them against the



stored user details (e.g., database, in-memory authentication, etc.).

- If authentication is successful, the server creates a session for the user.
- This session stores user-specific data, such as authentication status, roles, and other necessary details.

Spring Security automatically creates a session after a successful login using HttpSession.

---

—

2. The session ID (JSESSIONID) is stored in a cookie.

- When the session is created, the server generates a unique Session ID (called JSESSIONID in Java web applications).
  - This JSESSIONID is then sent to the client (browser) as a cookie.
  - The browser stores this session ID and sends it back to the server with every request.
  - The server reads the JSESSIONID to identify the logged-in user.
- 

—

3. The user remains logged in as long as the session is valid.

- As long as the session is active, the user remains authenticated.
- Sessions expire due to:

Inactivity timeout

Manual logout (session.invalidate()).

Server restart (if session persistence is not enabled).

# **S**PRING

# **F**RAMWORK

# **DAY 44:**

# **Spring Security**

## **PART-10**

### **FORM BASED**

### **AUTHENTICATIO**

### **N**

FORM

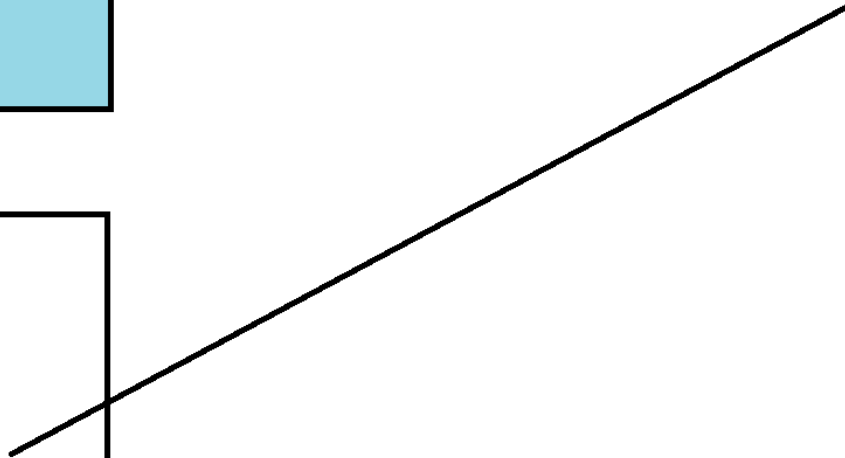
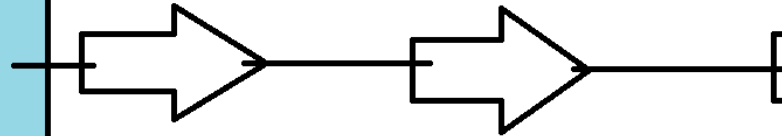
default form

username

password

CSRF TOKEN



**S**PRING

**F**RAMWORK

**DAY 45:**

**Spring Security**

**PART-11**

**ROLE BASED AUTH**

MEMBER ROLE----->member page

ADMIN ROLE----->admin page

- IN MEMORY (TRY)
- DATABASE

# **S**PRING

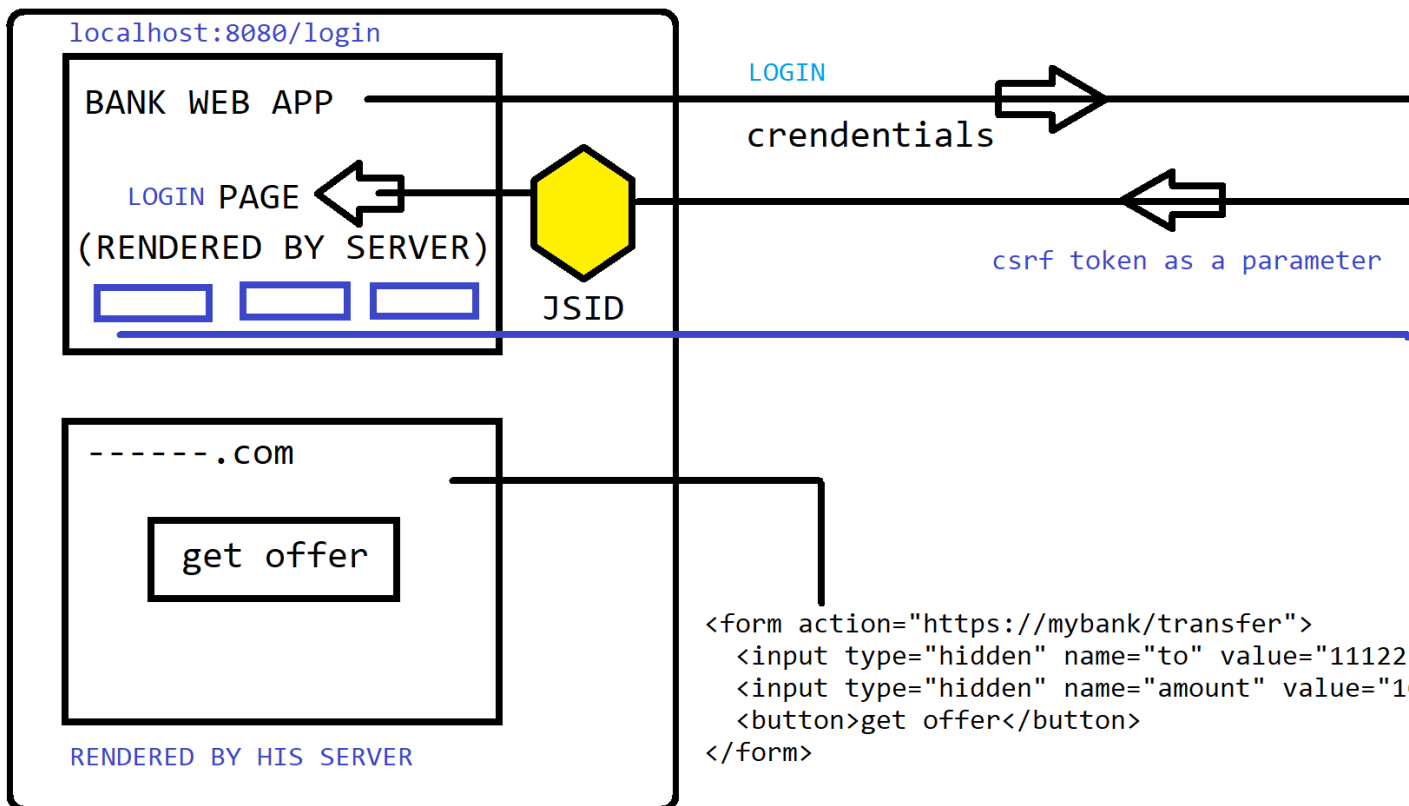
# **F**RAMWORK

## **DAY 46:**

# **Spring Security**

## **PART-12**

# CSRF TOKEN



CSRF (Cross-Site Request Forgery) is a type of attack that tricks a user into unknowingly

Executing unwanted actions on a web application where they're authenticated.



## **POINTS:**

- The server generates a unique token and includes it in every form.
- When a form is submitted, the token must be sent back.
- Server verifies that the token matches the one stored in the session.

## **NOTE:**

```
<input type="hidden" name="${_csrf.parameterName}"  
value="${_csrf.token}">
```

is server-side JSP code. It only works if the attacker is rendering a JSP from your server (e.g., bank.com/fake-form.jsp).

But here's the thing:

->Hacker can't render JSPs on your server

->A real attacker hosts a malicious page on their own domain, like -----.com

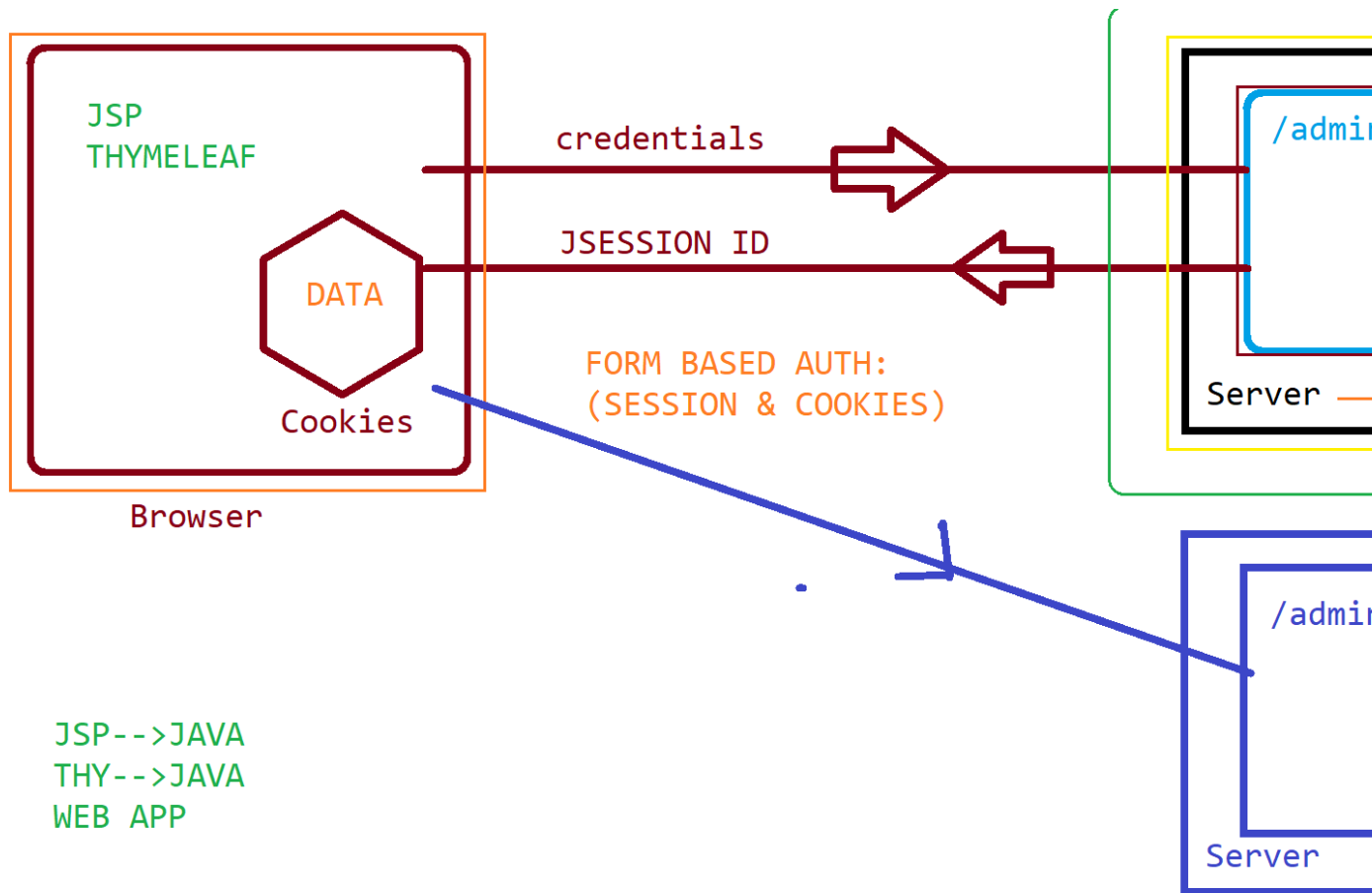
**S**PRING

**F**RAMEWOR**K**

**DAY 47:**

**Spring Security**

**PART-13**



**Problem with form based auth  
while working with large  
application and  
microservices(API BASED)**

Here are the major problems with form-based authentication when working with large applications and microservices (API-based):

### 1. Session-Based & Stateful

- Form-based auth uses HTTP sessions stored on the server.
- In a microservices environment, maintaining user sessions across multiple services becomes complex and non-scalable.

### 2. Not API Friendly

- Form login returns HTML pages, but APIs expect JSON responses.

### 3. Doesn't Work Well with Load Balancing

- In distributed systems, user requests go to multiple servers.

- Session info is stored per server; without sticky sessions, users can be logged out randomly.

#### 4. CSRF Overhead

- Session-based auth requires CSRF protection

#### 6. Poor Mobile App Compatibility

- Mobile apps can't easily handle HTML login pages and cookies.

### **Conclusion:**

**For micro services or large applications with APIs:**

->Avoid form-based authentication.

->Use JWT or OAuth2 — stateless, scalable,  
API-friendly.

**S**PRING

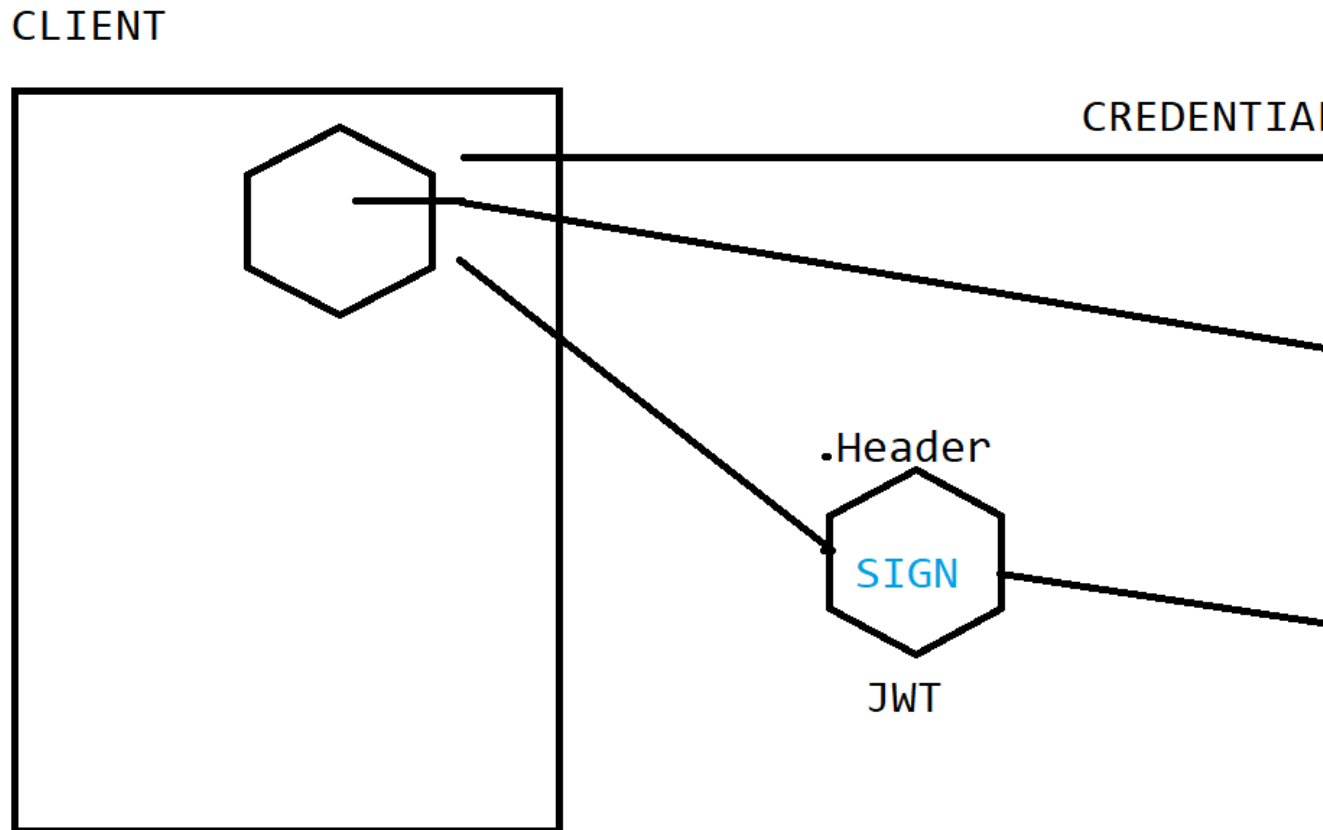
**F**RAMWORK

**DAY 48:**

**Spring Security**

**PART-14**

# JWT AUTHENTICATION



SPRING JWT AUTHENTICATION:

- A JSON Web Token (JWT) is a standard used to securely transmit information between a client (like a frontend application) and a server (the backend).
- It is commonly used to verify users' identities, authenticate them, and ensure safe communication between client and server.
- JWTs are mainly used in APIs to protect against unauthorized access.
- JWT (JSON Web Token) authentication is a stateless authentication (VVV IMP) {SCALABLE}

## How JWT token Works?

1. **User Logs In:** The client (browser) sends login credentials to the server.
2. **Server Generates JWT:** If credentials are valid, the server creates a JWT containing **user data** and **signs it with a secret key**.



- 3. Token Sent to Client:** The JWT is sent back to the client and stored (usually in localStorage or a cookie).
- 4. Client Sends Token in Requests:** For protected routes, the client includes the JWT in the Authorization header (Bearer Token).
- 5. Server Verifies and Responds:** The server verifies the token, extracts user info, and processes the request if valid.

**Server will reject the token if:**

- **The signature is invalid (i.e., not signed using your server's secret key).**
- **The token is expired.**

# STRUCTURE OF JWT

## Header:

Contains metadata like the type of token and the algorithm used to sign it:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

## Payload:

Contains the claims – i.e., user data like userId, username, roles, etc.

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "role": "admin",  
  "exp": 1714073232  
}
```

**Signature: Generated using the header,  
payload, and the secret key.**

---

# HOW THE SIGNATURE US FORMED?

## STEP1: ENCODING OF HEADER AND PAYLOAD

**Header:**

Contains metadata like the type of token and the algorithm used to sign it:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

**Header(Base64Url->encode) :**

**ewyfebfefhwhfsiaucfudwyftuewyfghwj**

**Payload:**

**Contains the claims – i.e., user data like userId, username, roles, etc.**

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "role": "admin",  
  "exp": 1714073232  
}
```

**Payload(Base64Url->encode) :**

**sqdlkncfgfeuywfbceuhfcxjerfxyregfcf**

## **STEP2: FORM THE MESSAGE**

message=

ewyfebfehwhfsiaucfudwyftuewyfghwj+”.”+  
sqdlkncfgfeuywfbceuhfcxjerfxyregfcf

ewyfebfehwhfsiaucfudwyftuewyfghwj.  
Sqdlkncfgfeuywfbceuhfcxjerfxyregfcf

## STEP3: GENERATE THE SIGNATURE BY USING THE ALGO ( HMACSHA256 )

Signature=HMACSHA256 (message, secret key)

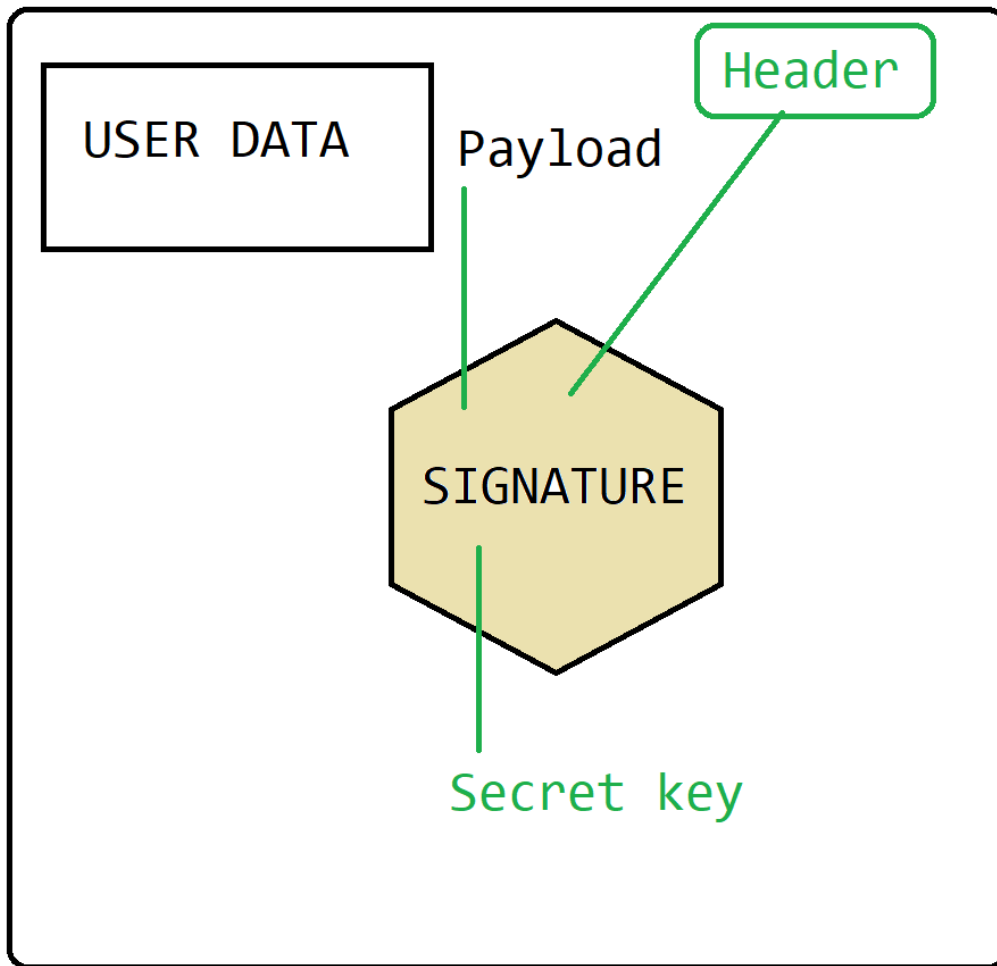
qjwhcnwaugfbcvuyrfguyewrfbcueyfcviu

# FULL JWT:

Ewyfebfehwhfsiaucfudwyftuewyfghwj.sqdlknc  
fgfeuywfbceuhfcxjerfxyregfcf.  
qjwhcnwaugfbcvuyrfguyewrfbcueyfcviu

---

# How the Signature is Formed:



eyJhbGciOiAiSFMyNTYiLC

eyJzdWIiOiAidXNlcjEyMy  
bGVzljogWyJST0xFX1VTR  
DB9.

s0K2yj1XZzNkM6tfCpb5A

JWT

**Here's the process, step-by-step:**

-----

**Combine the encoded header and payload:**

- 1.Header (Base64Url-encoded)**
- 2.Payload (Base64Url-encoded)**



**header:**

**eyJhbGciOiAiSFMyNTYiLCIAidHlwIjogIkpXVCJ9**

**payload:**

**eyJzdWliOiAidXNlcjEyMyIsICJ1c2VybmFtZSI6ICJzYWlmliwglJvbGVzIjogWyJST0xFX1VTRVIiXS  
wglmV4cCI6IDE3MTUwMDAwMDB9**

-----

**Form the message to be signed:**

**message = encoded\_header + "." +  
encoded\_payload**

**message =**

**eyJhbGciOiAiSFMyNTYiLCIAidHlwIjogIkpXVCJ9.  
eyJzdWliOiAidXNlcjEyMyIsICJ1c2VybmFtZSI6ICJzYWlmliwglJvbGVzIjogWyJST0xFX1VTRVIiXS  
wglmV4cCI6IDE3MTUwMDAwMDB9**

---

Generate the signature by applying the HMACSHA256 algorithm using the message and the secret key:

**signature = HMACSHA256(message,  
secret\_key)**

**Resulting in:**

**signature =**

**s0K2yj1XZzNkM6tfCpb5ADTDQ41XMf8a  
HUJ2ao4z2Fk**

---

**Example of the Full Token Again:**

**JWT (Header.Payload.Signature):**

eyJhbGciOiAiSFMyNTYiLCIAidHlwIjogIkpXVCJ9.

eyJzdWliOiAidXNlcjEyMyIsICJ1c2VybmFtZSI6ICJzYWlmliwglJvbGVzIjogWyJST0xFX1VTRVIiXS  
wglmV4cCI6IDE3MTUwMDAwMDB9.

s0K2yj1XZzNkM6tfCpb5ADTDQ41XMf8aHUI2a  
o4z2Fk

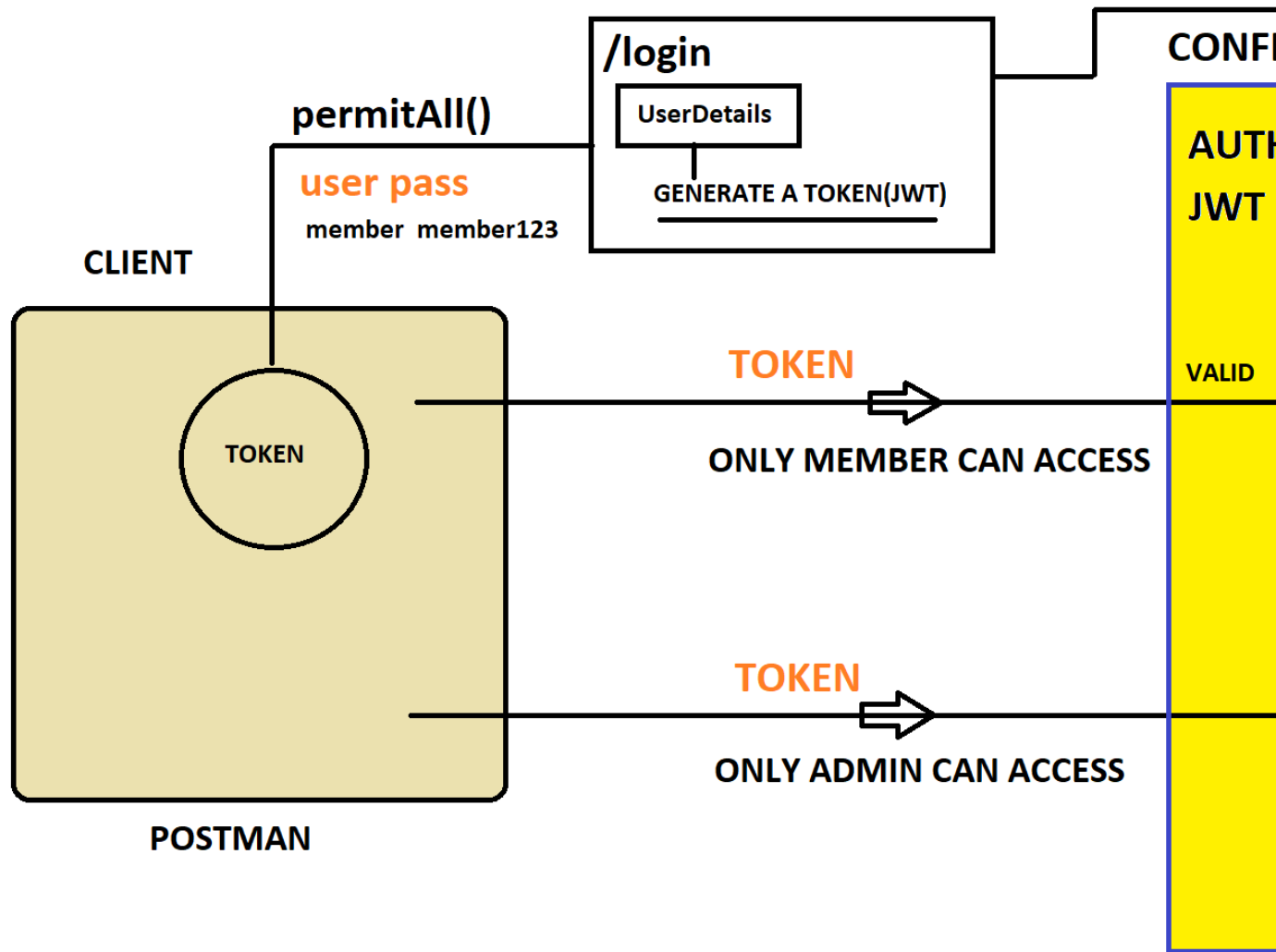
# **S**PRING

# **F**RAMWORK

# DAY 49:

# **Spring Security**

## **PART-15**



# Spring JWT Flow – First Time Login

## 1. Spring Security Configuration Loads

When the application starts, Spring Security loads your configuration.

**This defines:**

- Which endpoints are secured or open
- Custom handlers for auth/exception
- Your authentication manager setup

## **2. Login Request is Sent to /login**

- The client sends a POST request to /login with username & password.

## **3.Controller Handles Login**

Inside your controller (/login endpoint), you manually call:

**authenticationManager.authenticate(**

```
new  
UsernamePasswordAuthenticationToken(username,  
password)  
);
```

## 5. AuthenticationManager Flow

Internally, it triggers this chain:

ProviderManager →

DaoAuthenticationProvider →

UserDetailsServiceImpl

UserDetailsServiceImpl.loadUserByUsername(username) is called automatically.

It fetches the user from DB and returns a UserDetails object.

## 6. Successful Authentication

If credentials are valid:

- No exception is thrown
- You get a fully authenticated UserDetails object

## 7. Generate JWT Token

You call a `generateToken(userDetails)` method:

## 8. Return Token to Client

The compact token is sent back to the client in the response.

Client stores the token (usually in `localStorage` or cookies).

## NOTE:

**CSRF is only needed if: CSRF is not a concern in a stateless JWT-based API.**



- You're using form-based login
- Auth info is stored in cookies (i.e., session-based auth)



### **What does the secret key do in JWT?**

The secret key is used to digitally sign the JWT so that:

- The server can verify the token was not tampered with
- The server can trust the token's contents without storing it in a database

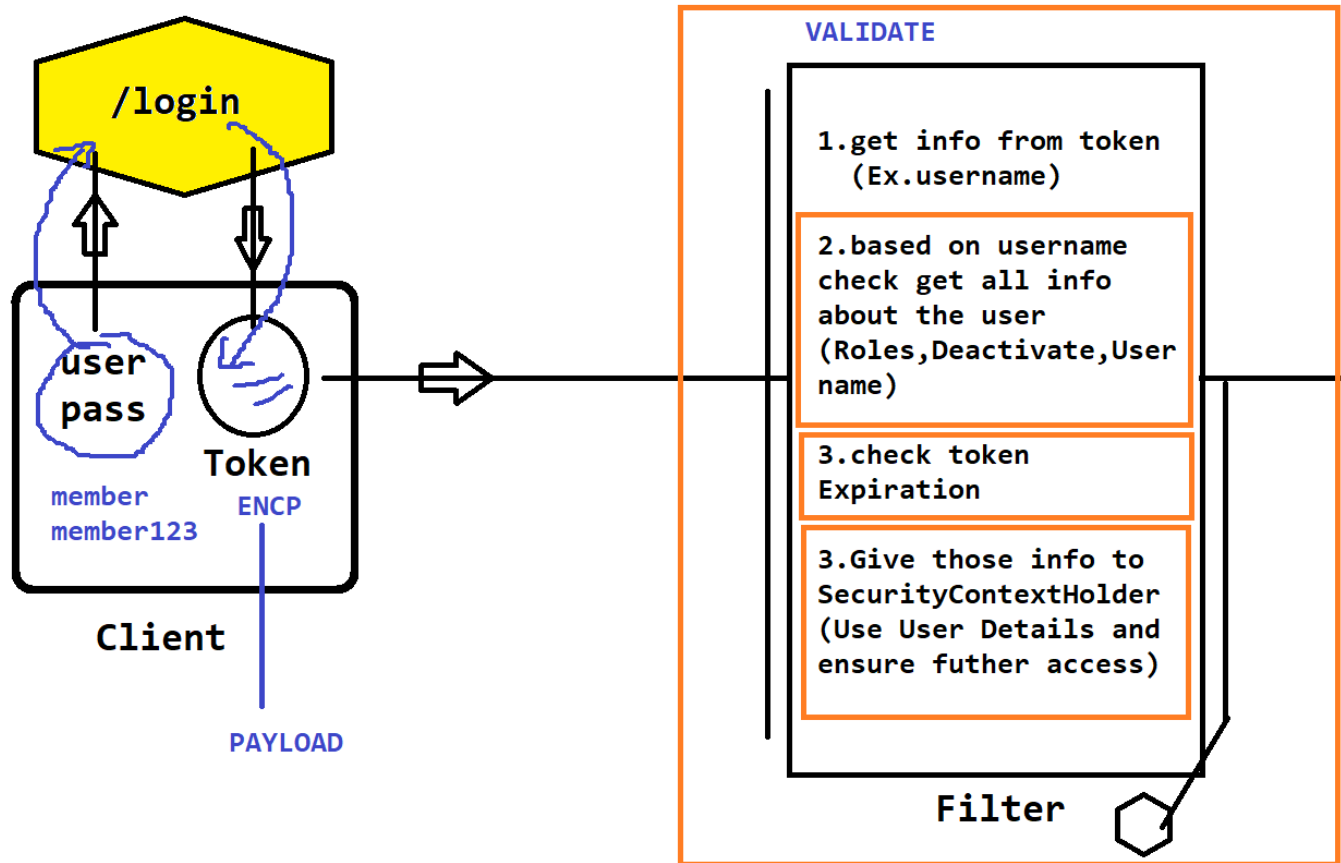
**S**PRING

**F**RAMWORK

**DAY 50:**

**Spring Security**

**PART-16**



## How JWT Token Works with Spring Security Filter – Full Flow

- After the user successfully logs in, the backend returns a JWT token. Now, whenever the user tries to access a secured API (e.g., /member), the following flow occurs:

## Request Flow via JwtFilter:

- The client sends an HTTP request to the secured API (/member) and includes the JWT token in the Authorization header:
- Since JwtFilter extends OncePerRequestFilter, it runs before your API controller logic.

## Token Extraction

### Inside the filter:

- Extract the token from the Authorization header.
- Remove the "Bearer " prefix to get the pure token:
- Extract Username from Token(The token contains a payload with information like username, roles, and expiry.)
- Now validate the token by checking:

1. The username in the token matches the username from DB
2. The token is not expired:

- Create

UsernamePasswordAuthenticationToken using UserDetails.

- Set this authentication into the SecurityContext:

1. SecurityContextHolder is used to store user authentication info and tell Spring Security that the user is logged in.
2. If it's not set, Spring Security blocks access to secured APIs.
3. This step ensures Spring Security knows that the user is authenticated.
4. It prevents future DB hits for the same request, improving performance.

## Final Step: Access Your API

If everything goes fine:

- The filter passes the request forward using `filterChain.doFilter(request, response);`
- The secured controller (e.g., `/member`) is now accessible.

## If Token is Invalid

If:

- Token is missing or malformed
- Token is expired
- Username doesn't match
- Any exception is thrown

➡ Spring Security will not allow the request to proceed to the controller.

➡ The user will receive an unauthorized (401) or forbidden (403) error.